COMPSCI 620 Reading Notes

Nikhil Anand

April 10, 2026

Contents

Part I

Reading Notes - February

Chapter 1

[Required] Understanding Understanding Source Code with Functional Magnetic Resonance Imaging

The paper investigates whether functional magnetic resonance imaging (fMRI) can be used to directly measure what happens in the brain when programmers understand source code. The broader aim is to build a scientific, cognitive basis for program comprehension so that programming tools, languages, and education can be improved.

1.1 Motivation

The main motivation of the paper is that we still don't fully understand what happens cognitively when programmers read and understand code, even though software underpins modern society. Researchers and educators want to improve programming languages, tools, and training, but most existing studies rely on indirect measures—such as performance, surveys, or think-aloud protocols—which are noisy, subjective, and often inconclusive.

But, why do we need a better understanding of program comprehension? Understanding program comprehension is not limited to theory building, but can have real downstream effects in improving education, training, and the design and evaluation of tools and languages for programmers. If direct measures of cognitive effort and difficulty could be obtained and correlated with programming activity, then researchers could identify and quantify which types of activities, segments of code, or kinds of problem solving are troublesome or improved with the introduction of a new language or tool.

The authors argue that to design better tools, languages, and educational approaches, we need direct, objective measurements of the mental processes involved in program comprehension. If researchers could measure cognitive effort and difficulty while someone reads code, they could identify which code structures are hardest to understand, evaluate new tools or languages more rigorously, and better train programmers.

To address this gap, the paper proposes using functional magnetic resonance imaging (fMRI) to observe brain activity during code comprehension. fMRI is widely used in cognitive neuroscience to study complex mental processes, so the authors aim to test whether it is feasible and meaningful to apply it to programming. Their broader goal is to lay the groundwork for a scientific understanding of program comprehension that could eventually answer broader questions, such as:

- What cognitive processes are involved in understanding code?

- How do languages, tools, or training methods affect comprehension?
- Can we predict or improve programming ability?

1.2 Methodology

1.2.1 fMRI

The authors use functional magnetic resonance imaging (fMRI) in a controlled experiment to measure brain activity while participants understand source code. fMRI is a non-invasive brain-imaging technique that measures changes in blood oxygenation, the BOLD (blood oxygen level dependent) signal, to infer neural activity. When a brain region becomes active, it consumes more oxygen, and this change in oxygenated vs. deoxygenated blood can be detected by the scanner. From these signals, researchers can determine which brain areas are active during a task.Because decades of neuroscience research have mapped many brain regions (Brodmann areas) to cognitive functions like language, memory, and attention, observing activation patterns allows researchers to infer which mental processes are involved in program comprehension.

1.2.1.1 Why was fMRI chosen?

The authors chose fMRI because:

- It provides direct physiological measurements of cognitive activity, unlike surveys or performance metrics.
- It allows identification of specific brain regions involved in understanding code.
- It is well established in cognitive neuroscience for studying complex behaviors like language comprehension.
- It can help build a scientific foundation for understanding programming cognition.

1.2.1.2 Challenges

1. fMRI does not measure neural activity directly—it measures the BOLD (blood oxygen level dependent) response, which reflects changes in blood oxygenation when a brain region becomes active. This signal takes a few seconds to appear after a cognitive process begins and peaks around 5 seconds before returning to baseline about 12 seconds after the task ends. Because of this delay, tasks must be carefully timed and long enough (often 30–120 seconds) so that activation builds up and can be measured reliably. Short or poorly timed tasks would produce weak or noisy signals.
2. To obtain statistically reliable results, fMRI studies requires multiple repetitions of similar tasks, averaging brain activity across trials and a careful comparison between experimental and control conditions. This means experiments must use tasks that are consistent in difficulty and duration, which limits how realistic or complex the tasks can be.
3. fMRI is extremely sensitive to movement. Even small movements of the head can introduce artifacts in the signal.

1.2.2 Structure of the Experiment

The study followed standard fMRI experimental design principles:

- Participants lay still in a scanner with their head stabilized to avoid motion artifacts.
- Code snippets were shown on a small mirrored screen.
- Tasks were repeated multiple times to average brain activity.

Each trial included:

1. comprehension task (60s)

2. rest period
3. syntax task (30s)
4. rest period

Rest periods establish a baseline so researchers can compare task-related activation. Participants indicated mentally when they finished a task (using a button) to minimize movement, and correctness was verified afterward.

1.3 Key Findings

1.3.1 Program comprehension activates a specific network of brain regions

The central result is that understanding source code produced a clear and consistent activation pattern in five brain regions, all in the left hemisphere - Brodmann Area 6, 21, 40, 44 and 47. These regions were significantly more active during program comprehension than during syntax-error detection, meaning they are associated specifically with understanding code rather than just reading it or visually scanning it.

1.3.1.1 Working memory and attention play a central role

Two of the activated regions, especially BA 6 and BA 40, are strongly linked to - working memory, divided attention and problem solving. This suggests that when programmers understand code, they must actively:

- Hold variable values and intermediate results in memory
- Track control flow (loops, conditions)
- Coordinate multiple pieces of information simultaneously

In other words, code comprehension heavily relies on cognitive resources similar to those used in reasoning and problem-solving tasks.

1.3.1.2 Strong involvement of language-processing regions

Another major finding is that three activated areas (BA 21, 44, and 47) are typically associated with language processing. This indicates that understanding code engages brain systems used for:

- Processing words and symbols
- Combining them into meaningful structures
- Interpreting semantics

The authors conclude that program comprehension shares similarities with both natural-language and artificial-language processing. This provides empirical support for earlier claims (e.g., Dijkstra) that language skills are important for programming.

1.3.1.3 Evidence for a bottom-up comprehension process

Because the experiment used short code snippets with obfuscated identifiers, it emphasized bottom-up comprehension (understanding code step by step rather than relying on prior knowledge). The activation pattern suggests that programmers:

- Read symbols and words
- Combine them into statements
- Integrate statements into larger semantic chunks
- Keep intermediate values in working memory

This supports a cognitive model in which code comprehension involves incremental integration and memory tracking.

1.3.2 Left-hemisphere dominance

All major activations occurred in the left hemisphere, which is typically associated with analytical reasoning and language. This reinforces the idea that program comprehension relies on structured, language-like cognitive processes.

1.3.3 Feasibility of using fMRI in software-engineering research

Beyond cognitive findings, an important result is methodological: The study demonstrates that fMRI can successfully be used to study program comprehension and produce meaningful activation patterns. This opens the door for future research on programming cognition using neuroscience methods.

1.4 Implications

The results of the paper open several important directions for future research and practical applications.

1.4.1 Study different types of program comprehension

This paper focused mainly on bottom-up comprehension (reading unfamiliar code line by line). A natural next step is to study other forms of comprehension, especially:

- Top-down comprehension: understanding code using prior knowledge, domain familiarity, or meaningful variable names
- Realistic large programs: moving beyond short snippets to more complex codebases
- Use of identifiers and domain knowledge: testing whether meaningful variable names act as "beacons" that change brain activation

Future experiments could compare obfuscated vs. meaningful code to see how memory and recognition affect comprehension.

1.4.2 Study code writing, not just code reading

This study only examined understanding code. Another major step is to study code generation or editing, which may involve different cognitive processes such as creativity and synthesis. Possible experiments include

- Compare brain activity during coding vs. reading
- Examine whether writing code activates right-hemisphere or creative regions
- Study subvocal speech or internal narration while coding

This would help build a more complete model of programming cognition.

1.4.3 Compare novice and expert programmers

A key open question is how expert programmers differ from average programmers. Future work could:

- Compare activation patterns of novices vs. experts
- Study whether experts show more efficient or reduced brain activation
- Identify cognitive traits linked to programming skill

Such research might help predict programming aptitude or guide training methods.

1.4.4 Evaluate programming languages and tools

The methodology could be used to evaluate whether certain language features or tools make code easier to understand. Possible experiments:

- Compare object-oriented vs. functional code
- Test different naming conventions or syntax styles
- Evaluate IDE tools or visualization systems
- Measure cognitive load across language designs

fMRI could help determine whether a language feature actually reduces mental effort rather than relying on subjective reports.

1.4.5 Compare code comprehension with natural language

Because the study found strong language-related brain activation, future work could compare:

- Code comprehension vs. reading natural language passages
- Programming languages vs. artificial grammars
- Multilingual programmers vs. monolingual programmers

This could clarify whether programming is more like language comprehension or mathematical reasoning.

1.4.6 Overall implication

The most important implication is that this paper establishes a new research direction: using neuroscience tools to study programming. The next steps involve scaling up the realism of tasks, comparing different kinds of programmers and programming activities, and using brain-based measurements to inform education, language design, and tool development.

Chapter 2

[Skim] The Effect of Poor Source Code Lexicon and Readability on Developer's Cognitive Load

The paper investigates how the quality of source code lexicon and readability influences developers' cognitive load during program comprehension. Program comprehension is a fundamental activity in software development, and a significant portion of development effort is spent understanding existing code before making changes. The authors argue that identifiers (e.g., variable and method names) and comments serve as critical carriers of domain knowledge, and their quality directly affects how easily developers can interpret code. While prior work has established links between identifier quality and overall software quality, there remains limited empirical evidence on how poor lexicon specifically impacts developers' mental effort during comprehension tasks .

To address this gap, the study explores the relationship between linguistic and structural properties of code and the cognitive load experienced by developers. Using techniques such as functional Near Infrared Spectroscopy (fNIRS) and eye tracking, the authors measure cognitive effort at a fine-grained level while participants perform comprehension and bug localization tasks. The work focuses particularly on "linguistic antipatterns"—poor naming and documentation practices—and examines how these, along with readability issues, affect developers' ability to process code. Overall, the paper aims to provide empirical evidence that poor source code lexicon and readability increase cognitive load, thereby negatively impacting program comprehension and developer efficiency .

2.1 Motivation

A substantial portion of software development effort is devoted to understanding existing code before making modifications, making program comprehension a central and costly activity in the software lifecycle. Prior research highlights that identifiers and comments act as key vehicles for embedding domain knowledge within code, enabling developers to reason about functionality and intent. Empirical studies have shown that the quality of identifiers correlates with overall software quality, suggesting that poor naming and documentation practices may hinder comprehension and increase maintenance effort [1].

Despite these insights, there is limited empirical evidence directly linking the quality of source code lexicon—particularly naming inconsistencies and poorly chosen identifiers—to developers' cognitive load during comprehension tasks. While software engineering research has extensively explored readability and structural metrics, the cognitive processes underlying how developers interpret code remain underexplored. Advances in cognitive neuroscience and program comprehension research have introduced techniques such as fMRI and eye tracking to study developers' mental processes, yet these approaches are often expensive or lack granularity [2, 3]. This creates a clear need for empirical, fine-grained investigation

into how linguistic aspects of code influence cognitive effort, motivating the study.

2.1.1 Background

The paper builds on the concept of linguistic antipatterns (LAs), which are recurring poor practices in naming, documentation, and identifier usage that negatively affect code understandability. These antipatterns include inconsistencies between method names and behavior, misleading identifiers, and contradictions between comments and implementation. Such issues can introduce confusion, forcing developers to spend additional cognitive effort reconciling discrepancies between code and its intended meaning [4].

In addition to linguistic factors, prior work has examined structural and readability metrics—such as cyclomatic complexity, lines of code, nesting depth, and adherence to coding conventions—as indicators of code comprehensibility. Studies have shown that these metrics correlate with how easily developers can understand and maintain code, with poorly structured code increasing comprehension difficulty [5]. Furthermore, research in cognitive measurement has explored the use of physiological signals to study program comprehension. Techniques such as fMRI have been used to identify brain regions involved in code understanding, while eye tracking has provided insights into developers' visual attention patterns during reading tasks [2, 6].

More recently, functional Near Infrared Spectroscopy (fNIRS) has emerged as a practical alternative for measuring cognitive load, offering a non-invasive and portable way to capture brain activity during real-world programming tasks. Unlike traditional methods, fNIRS enables fine-grained analysis of cognitive effort while maintaining ecological validity, making it suitable for studying how specific code features—such as lexicon quality and readability—affect developers' mental workload.

2.1.1.1 Linguistic Antipatterns

The paper builds upon prior work on linguistic antipatterns (LAs), which capture recurring deficiencies in the way identifiers, comments, and method names convey meaning in source code. These antipatterns arise when there is a mismatch between the natural language used in code and the actual behavior or intent of the program. Examples include misleading method names, inconsistent terminology, or contradictions between comments and implementation. Such issues can obscure program intent and force developers to spend additional effort reconciling semantic inconsistencies. Prior research has demonstrated that developers rely heavily on identifiers and comments as primary cues for understanding code, making the quality of these linguistic elements crucial for comprehension [4].

Linguistic antipatterns are particularly problematic because they violate developers' expectations about naming conventions and semantic consistency. When identifiers fail to accurately reflect functionality, developers must rely more on low-level code analysis rather than high-level reasoning, increasing cognitive effort. Studies on naming quality and identifier consistency further support the idea that well-chosen names improve comprehension speed and accuracy, while poor naming introduces ambiguity and misunderstandings [1]. This body of work establishes the importance of lexicon quality as a key factor influencing program comprehension, motivating its deeper investigation in relation to cognitive load.

2.1.1.2 Cognitive Load and Program Comprehension

The second foundation of the paper lies in understanding cognitive load during program comprehension. Cognitive load refers to the mental effort required to process information, and in the context of software development, it reflects how difficult it is for a developer to understand and reason about code. Prior research in software engineering has explored the relationship between code complexity, readability, and comprehension, showing that factors such as structural complexity, nesting, and formatting significantly impact how easily developers can interpret code [5]. However, these studies often rely on indirect measures such as performance or subjective assessments, leaving a gap in directly quantifying cognitive effort.

To address this, recent work has incorporated physiological measurement techniques to study developers' mental processes. Approaches such as functional magnetic resonance imaging (fMRI) and eye tracking have been used to analyze how developers read and understand code, revealing insights into attention patterns and brain activity during comprehension tasks [2, 6]. More recently, functional Near Infrared

Spectroscopy (fNIRS) has emerged as a practical alternative for measuring cognitive load in realistic settings, offering a balance between accuracy and usability. These advances provide the methodological foundation for studying how specific factors—such as poor lexicon and readability—affect developers' cognitive effort, enabling a more precise and empirical understanding of program comprehension.

2.2 Methodology

RQ 1. How do linguistic antipatterns affect developers' cognitive load during code comprehension tasks?

RQ 2. How does source code readability influence developers' cognitive load?

RQ 3. What is the combined effect of poor lexicon and low readability on cognitive load?

RQ 4. How do linguistic antipatterns and readability impact developers' task performance (e.g., comprehension and bug localization)?

2.2.1 Study Design

The study adopts a controlled experimental design to evaluate how linguistic antipatterns and readability influence cognitive load. Participants are asked to perform program comprehension and bug localization tasks on code snippets that are systematically manipulated to include or exclude linguistic antipatterns and readability issues. By controlling these variables, the study isolates their individual and combined effects on developer cognition.

The experiment follows a within-subject design, where each participant interacts with multiple code samples under different conditions. This allows for direct comparison of cognitive load across varying levels of lexicon quality and readability while minimizing variability due to individual differences.

2.2.2 Participants

The study involves participants with programming experience, typically including graduate students or developers with a background in software engineering. Participants are selected to ensure they possess sufficient familiarity with programming concepts required for comprehension tasks.

Before the experiment, participants are often screened or surveyed to assess their experience level, ensuring a relatively consistent baseline of programming proficiency. This helps reduce confounding factors related to expertise differences.

2.2.3 Experimental Tasks

Participants are assigned tasks that involve understanding code and identifying bugs. These tasks are designed to reflect realistic software engineering activities, particularly those requiring careful reasoning about program behavior.

The code snippets used in the tasks are modified to include linguistic antipatterns and varying levels of readability. This enables the study to evaluate how these factors influence both comprehension effort and task performance under controlled conditions.

2.2.4 Independent Variables

The primary independent variables in the study are the presence of linguistic antipatterns and the level of code readability. Linguistic antipatterns are introduced into code snippets to simulate poor naming and documentation practices. Readability is manipulated through structural characteristics such as formatting, complexity, and organization.

These variables are systematically combined to create different experimental conditions, allowing the study to analyze both individual and interaction effects on cognitive load.

2.2.5 Dependent Variables

The main dependent variable is cognitive load, which is measured using physiological signals obtained through fNIRS. This provides an objective measure of mental effort during task execution.

In addition to cognitive load, task performance metrics such as accuracy and completion time are recorded. These measures help assess how increased cognitive load translates into practical impacts on developer effectiveness.

2.2.6 Data Collection

Data is collected using a combination of physiological measurements and behavioral observations. fNIRS is used to capture brain activity associated with cognitive effort while participants perform tasks. This enables fine-grained tracking of mental workload in real time.

Alongside physiological data, the study records task outcomes such as correctness and time taken. This dual approach allows for a comprehensive analysis of both cognitive and performance-related effects.

2.2.7 Data Analysis

The collected data is analyzed to identify statistically significant differences across experimental conditions. Comparisons are made between code with and without linguistic antipatterns, as well as across different levels of readability.

The analysis focuses on understanding how these factors influence cognitive load and performance, both independently and in combination. This enables the study to draw conclusions about the role of lexicon quality and readability in program comprehension.

2.3 Key Findings

RQ1: Effect of Linguistic Antipatterns on Cognitive Load The results show that the presence of linguistic antipatterns significantly increases the cognitive load experienced by developers during program comprehension tasks. Participants exposed to code containing misleading or inconsistent identifiers exhibited higher mental effort, as measured through physiological signals. This indicates that poor naming forces developers to invest additional cognitive resources to interpret program intent. The study highlights that **linguistic inconsistencies disrupt semantic understanding**, leading to increased mental workload.

Furthermore, developers working with code containing linguistic antipatterns demonstrated reduced efficiency in reasoning about the code. Even when the structural complexity remained unchanged, the degraded lexicon quality alone contributed to higher cognitive strain. This suggests that **identifier quality independently impacts comprehension**, separate from structural code factors.

RQ2: Effect of Readability on Cognitive Load The findings indicate that reduced code readability also leads to a measurable increase in cognitive load. Code with poor formatting, higher complexity, or less clear structure required more effort for participants to process and understand. This reinforces the role of readability as a critical factor in supporting efficient program comprehension. The study emphasizes that **structural clarity directly reduces mental effort**, enabling developers to navigate and interpret code more effectively.

In addition, participants working with less readable code took longer to complete tasks and showed signs of increased cognitive strain. The lack of clear organization and formatting made it harder to follow control flow and logic. This demonstrates that **readability affects both cognitive load and task efficiency**, highlighting its importance in software quality.

RQ3: Combined Effect of Lexicon and Readability When linguistic antipatterns and poor readability were combined, the study observed the highest levels of cognitive load among participants. The interaction of semantic confusion (from poor naming) and structural complexity (from low readability)

created compounding difficulties for developers. The results show that **the combined effect is greater than individual factors**, indicating a cumulative burden on cognitive processing.

Participants in these conditions not only experienced higher mental effort but also struggled more with understanding program behavior. The simultaneous presence of multiple issues reduced their ability to form accurate mental models of the code. This suggests that **lexicon and readability jointly influence comprehension**, and improving only one may not be sufficient to reduce cognitive load effectively.

RQ4: Impact on Task Performance The study further demonstrates that increased cognitive load due to linguistic antipatterns and poor readability negatively affects task performance. Participants working with degraded code were more likely to make errors in comprehension and bug localization tasks. This highlights that cognitive strain translates directly into practical inefficiencies. The results indicate that **higher cognitive load leads to lower accuracy**, affecting developers' ability to correctly interpret code.

Additionally, task completion times were longer in conditions with poor lexicon and readability. Developers required more time to understand and reason about the code, reflecting increased effort and reduced productivity. This shows that **cognitive load impacts both correctness and speed**, reinforcing the importance of writing clear and well-structured code.

2.4 Threats to Validity

One potential threat to internal validity arises from the controlled experimental setting and the use of artificially constructed code snippets. While such control allows isolation of linguistic antipatterns and readability effects, it may not fully capture the complexity of real-world software systems. Participants may behave differently when working with larger, more familiar codebases compared to short experimental tasks. Additionally, the manipulation of code to inject antipatterns could introduce unintended artifacts that influence cognitive load beyond the intended variables. These factors suggest that **experimental control may limit realism**, potentially affecting the interpretation of causal relationships [2].

Threats to external validity stem from the participant pool and task design. The study primarily involves participants with academic or limited professional experience, which may not fully represent the diversity of expertise found in industry settings. Experienced developers might employ different strategies or be more resilient to poor naming and readability issues. Furthermore, the tasks focus on comprehension and bug localization, which, while common, do not cover the full spectrum of software engineering activities. This raises concerns that **results may not generalize across all developer populations and tasks** [6].

Construct validity is another concern, particularly in how cognitive load is measured and interpreted. Although fNIRS provides a promising physiological measure of mental effort, it captures brain activity indirectly and may be influenced by factors unrelated to the experimental variables, such as fatigue or individual differences in cognitive processing. Moreover, mapping physiological signals to cognitive load involves assumptions that may not fully reflect the nuanced mental processes involved in program comprehension. Therefore, **cognitive load measurements may not perfectly represent developers' true mental effort**, potentially affecting the accuracy of conclusions [2].

References

[1] Florian Deissenboeck and Markus Pizka. "Concise and Consistent Naming". In: *Proceedings of the 13th IEEE International Workshop on Program Comprehension*. 2006.

[2] Janet Siegmund et al. "Understanding Understanding Source Code with Functional Magnetic Resonance Imaging". In: *Proceedings of the 36th International Conference on Software Engineering*. 2014.

[3] Benjamin Floyd et al. "Decoding the Representation of Code in the Brain". In: *Proceedings of the 2017 IEEE/ACM International Conference on Software Engineering*. 2017.

[4] Venera Arnaoudova, Massimiliano Di Penta, and Giuliano Antoniol. "Linguistic Antipatterns: What They Are and How Developers Perceive Them". In: *Proceedings of the 2013 IEEE International Conference on Software Maintenance*. 2013.

[5] Raymond P. L. Buse and Westley Weimer. "Learning a Metric for Code Readability". In: *Proceedings of the 2010 IEEE International Conference on Software Maintenance*. 2010.

[6] Thomas Fritz et al. "Using Psycho-Physiological Measures to Assess Task Difficulty in Software Development". In: *Proceedings of the 36th International Conference on Software Engineering*. 2014.

Chapter 3

[Required] An fMRI study of code review and expertise

This paper studies how the brain processes computer code compared to natural language using fMRI. The researchers asked 29 participants to perform three tasks: code comprehension, code review, and prose (English text) review, while their brain activity was recorded. *They found that programming languages and natural language are represented differently in the brain.* Using machine learning on brain activity data, they could classify which task a participant was performing with high accuracy. Interestingly, the same brain regions were involved in distinguishing code from prose across tasks. *The study also found that expertise matters: more experienced programmers showed less difference in brain activity between code and prose tasks.* This suggests that as programmers gain skill, their brains may process code more similarly to natural language.

3.1 Motivation

The central motivation of this paper is that software engineering heavily depends on human judgment, yet we know very little about how the brain actually processes programming tasks. Activities like code comprehension and code review are fundamental to software development, but research in this area has mostly relied on behavioral studies, surveys, performance metrics, or self-reports. These methods tell us what developers do, but not how their brains represent and process code. The authors argue that this is a major gap. Developers spend a large portion of their time understanding code, and companies like Google and Facebook require formal code review for new contributions. Code understanding is often ranked as more important than even functional correctness in real-world software contexts. Despite this practical importance, there is almost no neuroscientific understanding of how programming is represented in the brain.

Another key motivation is the relationship between programming languages and natural languages. Prior research has shown that software has statistical properties similar to natural language. This raises an important question: does the brain process code like it processes English? Or does programming rely on completely different neural systems? Understanding this distinction could influence how we teach programming, how we design tools, and how we conceptualize programming as a cognitive activity.

The paper is also motivated by expertise. There are well-documented productivity differences between novice and expert programmers, sometimes by an order of magnitude. Other fields like physics, music, or taxi driving have shown that expertise can physically alter brain organization. However, no prior work had examined whether programming expertise changes neural representations. The authors aim to explore whether more experienced programmers process code differently at a neural level.

Beyond curiosity, the authors outline several broader reasons for using medical imaging in software engineering:

- Self-reports are often unreliable; brain data could provide more objective insight.

- Understanding neural processing could improve pedagogy by clarifying whether programming is cognitively closer to language, math, or something else.
- It may help explain how expertise develops and why experts are more efficient.
- It could guide tool design by revealing how humans mentally evaluate code.

In short, the paper is motivated by a fundamental question: what is the neural basis of programming? By using fMRI, the authors aim to move beyond behavioral performance and directly examine how code and prose are represented in the brain, and how that representation changes with expertise.

3.1.1 Background

Most of the background is quoted from the paper.

3.1.1.1 Code Review

Static program analysis methods aim to find defects in software and often focus on discovering those defects very early in the codes lifecyle. Code review is one of the most commonly deployed forms of static analysis in use today; well-known companies such as Microsoft, Facebook, and Google employ code review on a regular basis. At its core, code review is the process of developers reviewing and evaluating source code. Typically, the reviewers are someone other than author of the code under inspection. Code review is often employed before newly-written code can be committed to a larger code base. Reviewers may be tasked to check for style and maintainability deficiencies as well as defects.

3.1.1.2 Code comprehension

Much research, both recent and established, has argued that reading and comprehending code play a large role in software maintenance. A well-known example is Knuth, who viewed this as essential to his notion of Literate Programming. He argued that a program should be viewed as *"a piece of literature, addressed to human beings"* and that a readable program is *"more robust, more portable, [and] more easily maintained."* Knight and Myers argued that a source-level check for readability improves portability, maintainability and reusability and should thus be a first-class phase of software inspection.

3.1.1.3 Computer Science and Neuroscience

Siegmund et al. presented the first publication to use fMRI to study a computer science task: code comprehension. The focus of their work was more on the novelty of the approach and the evaluation of code comprehension and not on controlled experiments comparing code and prose reasoning.

Some researchers have begun to investigate the use of functional near-infrared spectroscopy (fNIRS) to measure programmer blood flow during code comprehension tasks. In smaller studies involving eleven or fewer participants, Nakagawa et al. found that programmers show higher cerebral blood flow when analyzing obfuscated code and Ikutani and Uwano found higher blood flow when analyzing code that required memorizing variables. fNIRS is a low-cost, non-invasive method to estimate regional brain activity, but it has relatively weak spatial resolution (only signals near the cortical surface can be measured); the information that can be gained from it is thus limited compared to fMRI. These studies provide additional evidence that medical imaging can be used to understand software engineering tasks, but neither of them consider code review or address expertise.

3.2 Methodology

3.2.1 fMRI

Functional magnetic resonance imaging (fMRI) is a non-invasive neuroimaging technique that measures brain activity by detecting changes in blood flow. When a specific brain region is more active, it consumes more oxygen, leading to increased blood flow to that area. fMRI detects these changes in blood oxygenation levels, allowing researchers to infer which regions of the brain are involved in specific

cognitive tasks. In this study, fMRI was used to compare brain activity during code comprehension, code review, and prose review tasks. [1]

3.2.2 Experiment Design

3.2.2.1 Participants

Thirty-five students at the University of Virginia were recruited for this study. Solicitations were made via fliers in two engineering buildings and brief presentations in a third-year "Advanced Software Development Techniques" class and a fourth-year "Language Design & Implementation" class. Out of these 35 participants, 29 were included in the final analysis after excluding those with excessive head motion during scanning. All of the participants were,

- Right-handed
- Native English speakers
- Screened for programming experience and MRI safety

The final sample comprised of 18 males and 11 females. Additional objective and subjective measures of programming experience were collected, including years of programming experience, number of programming languages known and GPA.

3.2.2.2 Tasks

Participants performed three tasks while undergoing fMRI scanning:

- **Code Comprehension**: Participants read a short code snippet and answered a true/false question about its behavior.
- **Code Review**: The participants were shown a Github style pull request, code differences and a comment from a reviewer. They were asked to evaluate the comment and decide to accept or reject the pull request.
- **Prose Review**: Participants were shown a short English passage and suggested edits. They were asked to evaluate the edits and decide to accept or reject them.

3.2.3 Preprocessing

Before analysis, the raw brain data underwent standard preprocessing for motion correction (head movement adjustment), alignment to a standard brain template (MNI space), noise correction, and no spatial smoothing was applied. This ensured that the data was clean and comparable across participants, while preserving the fine-grained spatial information necessary for multivariate pattern analysis (MVPA).

3.2.4 Processing

The extremely large dimension of fMRI data is a major obstacle for machine learning — we commonly have tens of thousands of voxels[2] but only a few dozen training examples. This can be solved using a simple linear map (a kernel function) that reduces the dimensionality of the feature space. The training inputs were given as a set of vectors $\{x_n\}_{n=1}^N$, with corresponding labels $\{y_n\}_{n=1}^N$ where N is the number of beta images for each participant across both classes. To learn a mapping, the authors employed a cumulative Gaussian, and specified posterior conditional over f using the Bayes rule

$$p(f|\mathcal{D},\theta) = \frac{\mathcal{N}(f \mid 0, K)}{p(\mathcal{D} \mid \theta)} \prod_{n=1}^{N} \phi(y_n f_n)$$

where $\mathcal{N}(f \mid 0, K)$ is a zero-mean prior, ϕ is a factorization of the likelihood of training samples.

[1] The authors do point out that the use of fMRI in Software Engineering is still exploratory and that there are limitations to the technique

[2] A voxel is the smallest unit of an fMRI image, analogous to a pixel in a regular image.

3.3 Key Findings

3.3.1 Research Question 1

> RQ1: Can we classify which task a participant is performing based solely on patterns of their brain activity?

Code Comprehension vs Prose tasks were highly distinguishable, with a 79% balanced accuracy ($p < 0.001$) and Code Review vs Prose tasks were also highly distinguishable, with a 71% balanced accuracy ($p < 0.001$). This strongly suggests that the brain forms distinct neural representations for programming tasks versus natural language tasks. Code is not processed identically to English prose and even though Code Review and Code Comprehension share similarities, but still differ at the neural level.

3.3.2 Research Question 2

> RQ2: Can we relate these task differences to specific brain regions?

The same set of brain regions distinguished code review and comprehension from prose. The regional importance maps were almost perfectly correlated ($r = 0.99, p < 0.001$). Highly involved regions included the prefrontal cortex (executive control, reasoning, decision-making) and the language-related areas near Wernicke's area.

This means that code and prose differ consistently in how they activate higher-order cognitive regions. Programming is not processed purely as math or purely as language. It heavily recruits executive control regions (working memory, abstract reasoning). But language-related areas are also involved. This finding suggests programming is a hybrid cognitive activity, involving language systems and executive reasoning systems. And importantly, the same neural system distinguishes code from prose across different kinds of code tasks.

3.3.3 Research Question 3

> RQ3: Does expertise influence neural representation of code vs prose?

This is one of the most interesting parts of the paper. The authors measured expertise using CS GPA (corrected for number of credits), then correlated it with classifier accuracy. There was a negative correlation between expertise and classification accuracy ($r = -0.44, p = 0.016$) for code comprehension vs prose.

This means that as programmers become more skilled, their brain activity for code and prose becomes less distinguishable. Experts process code in a way that is more similar to natural language. With experience, code may become more "language-like" in neural representation. Expertise leads to more integrated cognitive processing. This aligns with findings from other domains (e.g., physics, music, taxi driving) where expertise reorganizes neural representations.

3.4 Implications

Programming Has a Distinct Neural Basis The finding that code and prose can be reliably classified from brain activity implies that programming is not simply "reading English with symbols." It recruits distinct neural representations. This challenges the common assumption that programming is purely a linguistic activity. Instead, it appears to involve a unique cognitive configuration that blends language and executive reasoning systems. This has foundational implications: programming should be understood as its own cognitive domain, rather than being reduced to either language or mathematics. It justifies studying programming cognition as a standalone scientific field.

Programming Is a Hybrid Cognitive Activity Although code and prose are distinct, they activate overlapping language-related regions along with executive control regions (e.g., prefrontal cortex). This suggests programming is cognitively hybrid — it involves structured language processing but also heavy working memory, abstraction, planning, and decision-making. The implication here is that programming education and research should not frame coding as purely syntactic or linguistic. Instead, it requires

integrated training in reasoning, control processes, and structured abstraction. This supports the idea that difficulty in programming may arise not from syntax learning, but from cognitive load and executive demand.

Expertise Changes Neural Representation One of the most profound implications is that increased programming expertise reduces the neural distinction between code and prose. This suggests that with experience, code becomes cognitively more "natural." Experts may internalize programming structures similarly to how fluent readers process language. This supports theories of neural plasticity — expertise reorganizes how information is represented in the brain. It implies that the dramatic productivity gap between novice and expert programmers may not just be skill-based, but neurologically grounded. Programming expertise appears to alter mental representation itself.

Rethinking Code as "Natural Language" Previous research showed software shares statistical properties with natural language. This paper adds neural evidence to the discussion. While code is statistically language-like, it is neurally distinct — especially for novices. However, expertise moves it closer to language representation. This nuanced finding suggests that programming may be "learned language," rather than inherently language-like. It strengthens interdisciplinary connections between cognitive science, linguistics, and software engineering.

Chapter 4

[Skim] Why Is It Difficult for Programmers to Learn a New Programming Language

4.1 Summary

This paper explores why experienced programmers still struggle when learning a new programming language, even though they already know how to program. The core idea is something called **cross-language interference**. When developers learn a new language, they naturally try to reuse knowledge from languages they already know. Sometimes this helps — but very often, it leads to incorrect assumptions. These wrong assumptions cause confusion and mistakes.

4.2 Methodology

4.2.1 Phase 1: Stack Overflow Study

In Phase I, the researchers conducted an empirical study using Stack Overflow posts to investigate whether cross-language interference actually occurs in practice. They collected 450 posts across 18 programming language pairs, focusing on situations where programmers were transitioning from one language to another. To identify relevant posts, they used filtering criteria that captured questions referencing both a source language and a target language. The goal was to detect cases where developers were implicitly or explicitly bringing assumptions from a previously known language into a new one.

The researchers manually inspected and categorized the posts into two types: correct assumptions (facilitation) and incorrect assumptions (interference). A post was labeled as interference if the programmer made an incorrect assumption about the new language based on prior experience. To ensure reliability, two researchers independently coded the data and achieved high inter-rater agreement (Cohen's $K = 0.89$), indicating strong consistency in classification. Their analysis revealed that 61% of the posts contained incorrect assumptions caused by cross-language interference, providing large-scale evidence that prior programming knowledge can negatively impact learning a new language.

4.2.2 Phase 2: Interviews with Professional Programmers

In Phase II, the researchers conducted semistructured interviews with 16 professional programmers to gain deeper insight into how experienced developers learn new languages and how interference manifests in real-world learning. The participants had between 5 and 31 years of programming experience and had recently transitioned to a new language. These interviews lasted approximately 60 minutes and explored learning strategies, challenges faced, surprising differences, and the role of prior knowledge during the transition process.

The researchers used inductive thematic analysis to analyze the interview transcripts. They coded recurring patterns and grouped them into broader themes that described how programmers approach language learning and where difficulties arise. This qualitative phase revealed that experienced developers often rely on opportunistic, self-directed learning strategies, frequently trying to relate new languages to ones they already know. However, when fundamental differences exist—such as paradigm shifts, syntax differences, or tooling ecosystem changes—these prior mental models can become sources of confusion. This phase helped explain the cognitive and practical mechanisms behind the interference observed in Phase I, complementing the large-scale data with rich experiential insights.

4.3 Key Findings

Cross-language interference is common and measurable The study found strong evidence that prior programming knowledge often interferes with learning a new language. In their Stack Overflow analysis, 61% of the 450 posts examined contained incorrect assumptions caused by transferring knowledge from a previously known language. This shows that interference is not rare or anecdotal — it is widespread across many language pairs. Importantly, interference appeared across diverse transitions, not just between very different languages.

Prior knowledge is a double-edged sword Previous programming experience does not uniformly help. The authors found both:

- **Facilitation**: When knowledge transfers correctly and helps learning.
- **Interference**: When prior assumptions lead to misunderstanding.

Whether knowledge helps or harms depends heavily on how similar the two languages are — syntactically, conceptually, and paradigmatically. When languages share surface similarities but differ in deeper semantics, programmers are especially prone to errors.

Experienced programmers rely on opportunistic learning From interviews, the researchers found that professionals rarely learn new languages systematically (like reading a full book front to back). Instead, they use just-in-time learning strategies — Googling errors, using cheat sheets, browsing documentation, and asking colleagues. They often try to quickly relate the new language to one they already know. While efficient, this strategy increases the risk of shallow understanding and incorrect mental mappings.

Paradigm shifts create major difficulty Transitions involving paradigm changes (e.g., object-oriented → functional, or manual memory → ownership models) were particularly difficult. Developers had to actively suppress old habits and rethink how problems are structured. This mental restructuring — not syntax — was often the biggest challenge.

Tooling and ecosystem differences add friction Learning a new language is not just about syntax and semantics — it also involves adapting to new IDEs, package managers, debugging tools, and workflows. The interviews revealed that ecosystem transitions can be just as cognitively demanding as language-level differences.

Lack of explicit transition support The authors observed that documentation and learning resources are usually designed for beginners, not experienced programmers switching languages. There is little structured support for cross-language transitions, even though interference is common.

4.4 Implications

Documentation should explicitly support language transitions The findings suggest that documentation and learning resources should not assume learners are complete beginners. Many programmers already know other languages and try to map new concepts onto prior knowledge. Instead of generic tutorials, resources could include transition-focused guides such as "If you're coming from Java…" or "For

Python developers...". Making similarities and differences explicit can reduce incorrect assumptions and help programmers build accurate mental models faster.

Tools can reduce interference through smarter feedback The study highlights opportunities for IDEs and compilers to provide context-aware error messages that anticipate common misconceptions. For example, if a developer coming from Python makes a Java-style mistake, the system could detect that pattern and explain the difference explicitly. This kind of adaptive tooling could help programmers correct faulty assumptions before they solidify into confusion.

Language designers should anticipate prior mental models When designing new languages, designers should consider how programmers from popular ecosystems will interpret features. Some features may look familiar but behave differently, increasing the risk of interference. Being mindful of how existing mental models transfer — or clash — can reduce friction during adoption. In other words, usability across language boundaries matters.

Learning a language means entering an ecosystem The study shows that difficulty is not limited to syntax or semantics. Tooling, IDEs, debugging workflows, package managers, and build systems also create friction. Therefore, improving onboarding into an ecosystem (not just the language itself) is crucial. Streamlined setup tools, unified build environments, and consistent tooling conventions can ease transitions.

Programming knowledge does not fully generalize A broader implication is theoretical: programming expertise does not automatically transfer smoothly across languages. Mental models are often language-specific. This challenges the assumption that "once you know one language, the rest are easy." For educators and researchers, this means we need better frameworks for understanding knowledge transfer and interference in programming.

Chapter 5

[Required] What Predicts Software Developers' Productivity?

Organizations have a variety of options to help their software developers become their most productive selves, from modifying office layouts, to investing in better tools, to cleaning up the source code. But which options will have the biggest impact? Drawing from the literature in software engineering and industrial/organizational psychology to identify factors that correlate with productivity, this paper designed a survey that asked 622 developers across 3 companies about these productivity factors and about self-rated productivity.

5.1 Motivation

Improving productivity of software developers is important.By definition developers who have completed their tasks can spend their freed-up time on other useful tasks, such as implementing new features or on new verification and validation activities. Organizations such as ours demand empirical guidance on which factors to try to manipulate in order to best improve productivity. For example, should an individual developer (1a) spend time seeking out the best tools and practices, or (1b) shut down email notifications during the day? Should a manager (2a) invest in refactoring to reduce code complexity, or (2b) give developers more autonomy over their work? Should executives (3a) invest in better software development tools, or (3b) should they invest in less distracting office space? In an ideal world, we would invest in a variety of productivity-improving factors, but time and money are of productivity-improving factors, but time and money are limited, so we must make selective investments.

The paper focuses on answering three questions:

1. What factors most strongly predict developers' self-rated productivity
2. How do these factors differ across companies?
3. What predicts developer self-rated productivity, in particular, compared to other knowledge workers?

5.2 Methodology

The authors designed a large-scale survey study to investigate what factors predict software developers' self-rated productivity. Their methodological approach was primarily **quantitative and correlational**, relying on regression analysis across multiple companies. The central goal was breadth: instead of deeply examining one or two factors, they aimed to evaluate a wide spectrum of potential productivity predictors drawn from both software engineering and industrial/organizational psychology literature.

Measuring Productivity. But How? A foundational methodological decision was how to measure productivity. Rather than using objective metrics such as lines of code or number of commits alone, the authors chose a subjective measure: *self-rated productivity*[1]. Participants were asked to rate their agreement with the statement, *"I regularly reach a high level of productivity."* The researchers justified this choice by arguing that objective measures can be gamed, incomplete, or misleading (for example, a small but critical bug fix might reflect high productivity despite minimal code written). Self-ratings, although imperfect, allow respondents to integrate multiple dimensions of their work experience, including output quality, efficiency, and workflow effectiveness. To validate this decision, they conducted a contextual analysis correlating self-rated productivity with objective measures (lines changed and changelists merged) within Google. These objective measures showed statistically significant but very small explanatory power ($R^2 < 3\%$)[2], reinforcing that subjective productivity captures broader aspects than raw output metrics.

What are the factors that decide productivity? The next major methodological component involved identifying candidate productivity factors. The authors began with 127 potential factors derived from four primary literature sources: prior structured reviews of knowledge worker productivity, validated survey instruments from organizational psychology (such as the Work Design Questionnaire), a systematic review of software engineering productivity factors, and prior empirical work on developers' perceived productivity. They also added three additional factors of particular organizational interest. To reduce survey fatigue and ensure practicality, they applied three filtering criteria: eliminating duplicates, condensing similar constructs, and favoring factors with practical utility. This iterative, collaborative refinement process ultimately reduced the pool to 48 survey items. Each factor was presented as a Likert-scale agreement statement (Strongly disagree to Strongly agree).

5.2.1 The Experiment

The study was conducted across three companies: Google, ABB, and National Instruments. This multi-company design strengthened external validity by testing whether relationships generalized across different organizational contexts (e.g., open vs. mixed office layouts, monolithic vs. distributed repositories, centralized vs. flexible tooling). Each company administered the same core survey, though small contextual adjustments were made when necessary. In total, the final valid sample included 407 Google developers, 137 ABB developers, and 78 National Instruments developers.

To address their third research question, *how developers differ from other knowledge workers*, the authors also deployed a modified version of the survey to Google analysts. Software-specific questions were removed or adapted. This comparison group allowed the researchers to determine whether certain productivity predictors were uniquely strong for developers or more broadly applicable to knowledge workers.

The survey also collected demographic variables — gender, tenure, and seniority — which were treated as control variables in the regression models. These were included to reduce confounding effects. For example, earlier analysis showed that more senior developers tended to rate themselves slightly less productive, making seniority an important covariate. Missing demographic data were imputed conservatively (e.g., using mean or most common values), since demographics were not primary variables of interest but controls.

To ensure data quality, the researchers included an attention check item instructing respondents to select a specific answer. Surveys failing this check were discarded. This step removed 6–22% of responses depending on the company, increasing confidence that retained responses reflected careful consideration.

The core analytical strategy involved running separate multiple linear regression models for each factor at each company. In each model, self-rated productivity served as the dependent variable, and one productivity factor served as the independent variable, while demographic variables were included as controls. Importantly, the authors ran 48 separate regressions per company rather than building a single multivariate model including all factors simultaneously. This allowed them to isolate the strength of

[1] Productivity here does not mean *how productive one is?* but rather *how productive they are consistently?*

[2] In regression analysis, R^2 (R-squared) tells us how much of the variation in the outcome variable can be explained by the predictor(s). It ranges from 0 to 1

association for each factor individually while preserving company data privacy. Because running many statistical tests increases the risk of false positives, they corrected p-values using the Benjamini–Hochberg method to control the false discovery rate.

In interpreting results, the authors emphasized effect size (regression coefficient magnitude) more than statistical significance alone. They noted that statistical significance is sensitive to sample size (Google's larger sample produced more significant results), whereas effect size better captures practical impact. They also analyzed variance across companies to assess generalizability and compared coefficient differences between developers and analysts to detect role-specific predictors.

5.2.2 Causality Limitations

The authors are very clear that their methodology identifies correlations, not causal relationships. That distinction is crucial. The survey measures how strongly each factor (like job enthusiasm or feedback quality) is statistically associated with self-rated productivity. But association does not automatically mean that the factor causes productivity changes.

1. Correlation Does Not Establish Direction The biggest limitation is directionality. For example, the strongest predictor in the study is job enthusiasm. The data show that higher enthusiasm is associated with higher self-rated productivity. But which direction is the arrow?

- Does enthusiasm cause productivity? enthusiasm $\rightarrow$ productivity
- Or does being productive make you feel enthusiastic? productivity $\rightarrow$ enthusiasm
- Or is there a bidirectional relationship, where enthusiasm and productivity reinforce each other? enthusiasm $\leftrightarrow$ productivity

The survey design cannot distinguish between these possibilities because all variables were measured at the same time (cross-sectional design). There is no time-based sequencing that would allow us to infer "A happened before B."

2. Potential Third Variables (Confounding Factors) Another causality threat is the presence of unmeasured third variables. A third factor might influence both the predictor and productivity simultaneously. For example

- Strong leadership culture could increase both useful feedback and productivity.
- Organizational stability might increase both enthusiasm and performance.
- Personal traits (e.g., conscientiousness) might make someone both more productive and more likely to report positive perceptions.

If such confounders are not measured or controlled for, the regression coefficient may reflect indirect relationships rather than direct causal effects. The authors controlled for demographics (gender, tenure, seniority), but many psychological or organizational traits were not controlled.

3. Causality Evidence Depends on Prior Literature The authors acknowledge that their confidence in causality partly depends on prior research.

- There is strong meta-analytic evidence from organizational psychology that feedback improves performance.
- There is experimental evidence in other contexts that autonomy increases motivation and output.

So for some factors, there is theoretical and empirical support for causal direction. However, not all 48 factors have equally strong causal backing. The paper does not conduct a systematic meta-analysis to establish causal strength for each factor. Therefore, the strength of causal inference varies across factors.

5.3 Key Findings

RQ 1: What factors most strongly predict developers' self-rated productivity? The central finding of the study is that the strongest predictors of self-rated productivity were non-technical, human-centered factors, not technical or platform-related ones. After running regression models across three companies, the authors identified three factors that were statistically significant predictors across all organizations - (1) Job enthusiasm, (2) Peer support for new ideas, (3) Receiving useful feedback about job performance.

Among these, job enthusiasm had the strongest overall effect size. This means that developers who reported being more enthusiastic about their jobs were substantially more likely to rate themselves as highly productive.

A particularly striking result is that the top-ranked predictors were overwhelmingly social and psychological, rather than technical. Factors such as architecture complexity, processing power requirements, or platform volatility — many of which are emphasized in traditional productivity models like COCOMO II[3] — ranked much lower in predictive strength.

This directly answers RQ1 by showing that individual-level productivity is more strongly associated with - motivation and emotional engagement, team climate and support and feedback and recognition mechanisms rather than purely technical characteristics of the software or development environment.

RQ 2: How do these productivity factors differ across companies? The study compared results across Google, ABB, and National Instruments to examine consistency and variation. Some factors showed remarkable stability across organizations, particularly, peer support for new ideas, useful feedback and the use of remote work for concentration. These factors had low variance across companies, suggesting they may generalize well beyond the specific organizations studied.

However, some factors varied substantially across companies, including, the use of best tools and practices, code reuse and the accuracy of incoming information. For example, the importance of using the best tools was much stronger at Google than at National Instruments. The authors suggest organizational structure may explain this — such as Google's large monolithic codebase making tool effectiveness more critical.

Thus, RQ2 is answered by showing that

- Human and social factors tend to generalize across organizations
- Technical and tooling-related factors are more context-dependent

This highlights that productivity drivers are not universally identical; organizational environment matters.

RQ 3: What predicts developer productivity in particular, compared to other knowledge workers? To address this, the authors compared Google developers with Google analysts (a non-developer knowledge worker group). They found both similarities and differences.

Similarities The authors found that job enthusiasm predicted productivity strongly for both groups. Some social and autonomy-related factors were important for both developers and analysts.

Differences ' productivity was more strongly related to task variety and their ability to work effectively away from their desks. In contrast, analysts' productivity was more strongly associated with positive feelings toward teammates and their time management autonomy.

The authors interpret this as evidence that:

- Developers may be especially sensitive to interruption and context switching.
- The ability to work away from distractions may be particularly important for programming work.

[3]COCOMO II model, Constructive Cost Model II, is a widely used, updated, and flexible algorithmic model designed to estimate software project cost, effort, and schedule - https://softwarecost.org/tools/COCOMO/

- Task variety may matter more for developers due to shared toolsets and reduced switching costs.

Thus, RQ3 is answered by showing that while developers share many productivity drivers with other knowledge workers, certain factors — especially around focus and task structure — are uniquely important in software development.

5.4 Implications

1. Productivity investments should prioritize human factors The most immediate implication is that organizations may be over-investing in technical optimizations while under-investing in social and psychological factors.Since the strongest predictors of productivity were job enthusiasm, peer support for new ideas and receiving useful feedback, companies aiming to improve developer productivity should prioritize (1) increasing engagement and motivation, (2) creating psychologically safe environments, and (3) improving feedback systems

This suggests that productivity initiatives might be more effective if they focus on culture and management practices rather than solely on tooling or architecture improvements.

2. Developer productivity is not just about tools Although tools and practices matter, they were not consistently the strongest predictors across companies. In some organizations (like Google), tooling had stronger relationships; in others, it did not. Tooling investments may yield diminishing returns if cultural factors are weak. Organizations should evaluate whether morale, autonomy, or feedback systems need attention before investing heavily in technical infrastructure.

In short, improving productivity is not just a tooling problem — it's a leadership and organizational design problem.

3. Feedback systems are strategically important Useful feedback about job performance was one of the few factors consistently significant across all companies. Companies should design structured, constructive feedback processes and ensure feedback is timely and behavior-focused, while also avoiding vague or purely evaluative performance reviews. Feedback is not just an HR mechanism — it is a productivity driver.

4. Software engineering research may be overly technical The finding that non-technical factors dominate productivity prediction suggests that software engineering research may need to shift focus. Traditionally, productivity studies emphasize - (1) code complexity, (2) architecture, and (3) platform volatility.

But this study suggests that organizational psychology and human factors may yield greater practical impact and thus, cross-disciplinary research between SE and I/O psychology is needed.

5. Rethinking traditional productivity models Many traditional models (like COCOMO II) emphasize project-level technical factors. This study suggests that those models may not translate well to individual-level productivity. Individual productivity is more socially and psychologically influenced than those models account for. Future productivity models may need to integrate human and cultural dimensions more explicitly.

Chapter 6

[Required] What Improves Developer Productivity at Google? Code Quality

This paper investigates what causes improvements in individual developer productivity at Google. Prior research mostly identified correlations (e.g., good tools correlate with productivity) or used tightly controlled experiments. The authors aim to go further by using panel data analysis, which allows stronger causal inference in a real-world setting. They analyze data from over 2,000 Google engineers using a longitudinal internal survey measuring self-reported productivity and workplace factors and detailed engineering logs (e.g., coding time, build times, code reviews).

Among these, satisfaction with project code quality emerges as the strongest factor. A follow-up lagged panel analysis shows that improvements in perceived code quality are followed by measurable improvements in objective productivity metrics (like reduced coding and deployment time), but not the other way around. This strengthens the claim that better code quality leads to higher developer productivity, rather than productivity simply enabling better quality.

6.1 Motivation

The motivation behind this paper comes from a very practical and high-stakes question: organizations invest heavily in tools, processes, and structural changes to improve developer productivity — but which of these actually cause productivity improvements in real-world settings? Prior research leaves leaders in an uncomfortable middle ground. On one side, controlled experiments (like testing a new debugging tool) can establish strong causality, but they often use simplified tasks, small samples, or student participants, raising doubts about real-world applicability. On the other side, large field studies in companies observe real developers doing real work, but they typically rely on cross-sectional correlations, which cannot rule out confounding factors. This gap creates uncertainty for decision-makers: should they invest in tooling, reduce meetings, reorganize teams, focus on code quality, or something else? The authors are motivated to bridge this methodological gap by applying panel data techniques in a real industrial context (Google), allowing them to make stronger causal claims without sacrificing ecological validity.

Specifically, the paper is motivated by:

- The need to move beyond simple correlations and identify what causes productivity improvements in practice.
- The limitations of prior research methods (controlled experiments vs. field studies) in addressing this question.
- The weaknesses of cross-sectional surveys, which cannot eliminate confounding factors (e.g., education, seasonal effects, personality traits).

- The desire to provide actionable, evidence-based guidance for engineering organizations.
- The broader research challenge of combining real-world data with stronger causal inference methods.

At its core, the paper aims to answer a single driving question: What truly improves developer productivity in practice, in a real organizational setting?

6.1.1 Previous (or Related) Work

A wide spectrum of prior research provides some answers to this question, but with significant caveats.

1. Ko and Myers' controlled experiment showed that a novel debugging tool called Whyline[1] helped Java developers fix bugs twice as fast as those using traditional debugging techniques.
2. At the other end of the spectrum, Murphy-Hill[2] and colleagues surveyed developers across three companies, finding that job enthusiasm consistently correlated with high self-rated productivity

While this evidence is compelling, organizational leaders are faced with many open questions about applying these findings in practice, such as whether the debugging tasks performed in that study are representative of the debugging tasks performed by developers in their organizations.

On one hand, software engineering research that uses controlled experiments can help show with a high degree of certainty that some practices and tools increase productivity, yet such experiments are by definition highly controlled, leaving organizations to wonder whether the results obtained in the controlled environment will also apply in their more realistic, messy environment. On the other hand, research that uses field studies ś often with cross sectional data from surveys or telemetry ś can produce contextually valid observations about productivity, but drawing causal inferences from field studies that rival those drawn from experiments is challenging.

6.2 Methodology

The study adopts a quasi-experimental longitudinal field design to investigate causal factors influencing developer productivity at Google. Rather than conducting a controlled laboratory experiment, the researchers analyze real-world data collected over time from professional software engineers. The central methodological choice is the use of panel data analysis, which allows the authors to make stronger causal inferences than traditional cross-sectional studies while preserving ecological validity. By observing the same engineers at multiple points in time, the study is able to isolate changes within individuals and control for stable personal characteristics that might otherwise confound results.

6.2.1 Data Sources

Engineering Satisfaction (EngSat) Survey The research integrates two primary data sources: a longitudinal internal survey and engineering logs data. The survey, known as the Engineering Satisfaction (EngSat) survey, is administered company-wide every three months. Engineers report their experiences over the previous quarter, including perceptions of productivity, code quality, technical debt, tooling, communication, and organizational factors. Engineers with at least six months of tenure are eligible, and each engineer appears at most twice in the final panel dataset. After joining survey responses with behavioral data, the authors obtained complete panel data for 2,139 engineers.

Engineering Logs In addition to survey data, the study incorporates detailed logs from internal engineering systems. These logs include data from version control systems, code review platforms, build and test systems, and calendar systems. From these sources, the researchers extracted objective measures such as active coding time, wall-clock coding duration, number of changelists merged, build latency, review rounds, review wait times, and time spent in meetings. This integration of subjective and objective data strengthens the study's validity by allowing perceived productivity to be compared against measurable work behavior.

[1] Whyline for Java - https://github.com/amyjko/whyline
[2] The previous required reading

6.2.2 Variables

Dependent Variable The primary dependent variable is self-rated productivity. Engineers were asked to rate their overall productivity over the previous three months on a five-point Likert scale ranging from "Not at all productive" to "Extremely productive." Although subjective measures can raise concerns about bias, the authors validated this measure by training a random forest classifier using objective productivity metrics. The model achieved high precision and recall, demonstrating a strong relationship between self-reported productivity and actual work patterns. This validation step supports the use of subjective productivity as a meaningful outcome variable.

Independent Variables The study initially considered 42 potential productivity factors derived from both survey responses and logs data. After conducting multicollinearity checks using Variance Inflation Factor (VIF) analysis, three highly correlated variables were removed, leaving 39 independent variables in the final model. These variables were grouped into six conceptual categories: (1) code quality and technical debt; (2) infrastructure tools and support; (3) team communication; (4) goals and priorities; (5) interruptions; and (6) organizational and process factors. Some variables were perceptual (e.g., satisfaction with code quality, perceived hindrance from technical debt), while others were behavioral (e.g., build times, code review rounds, meeting duration). This broad set of predictors allowed the researchers to evaluate multiple dimensions of the software development environment simultaneously.

6.2.3 Panel Data Model

To analyze the relationship between productivity and these factors, the researchers employed a fixed-effects panel regression model. The general form of the model includes individual-specific effects and time-specific effects. Individual fixed effects capture all stable characteristics of a developer—such as inherent ability, education, personality, or long-term role assignment—while time effects account for broader company-wide changes or seasonal influences.

$$y_{it} = \alpha_i + \lambda_t + \beta D_{it} + \epsilon_{it} \tag{6.1}$$

where y_{it} is the self-rated productivity of engineer i at time t, α_i represents the individual fixed effect, λ_t captures time fixed effects, D_{it} is a vector of the 39 independent variables for engineer i at time t, β is the vector of coefficients to be estimated, and ϵ_{it} is the error term.

The model was then differenced between time periods. By analyzing changes within each engineer rather than differences between engineers, the fixed-effects approach eliminates time-invariant confounding factors automatically. In other words, any characteristic that does not change over time (such as education or long-standing skill level) is removed from the equation. This methodological choice substantially strengthens causal inference compared to cross-sectional correlation analysis.

$$\Delta y_{it} = \Delta\lambda_t + \beta\Delta D_{it} + \Delta\epsilon_{it} \tag{6.2}$$

To estimate the model parameters, the authors used Feasible Generalized Least Squares (FGLS)[3]. This method addresses issues of heteroskedasticity and serial correlation in panel data, improving statistical efficiency and robustness. Most continuous predictors were log-transformed so that estimated coefficients could be interpreted as elasticities, meaning that percentage changes in predictors correspond to percentage changes in productivity.

6.2.4 Lagged Panel Analysis

While the standard panel model identifies factors that are causally linked to productivity, it does not establish the direction of causality. To address this limitation, the authors conducted a lagged panel analysis focusing specifically on the relationship between code quality and productivity.

[3]Feasible Generalized Least Squares (FGLS) is a two-step estimation technique used in regression analysis to produce efficient, consistent estimators when the error terms are heteroscedastic or serially correlated (autocorrelated) with an unknown covariance structure

In this analysis, they tested two competing hypotheses: first, that changes in code quality at time $T-1$ lead to changes in productivity at time T; and second, that changes in productivity at time $T-1$ lead to changes in code quality at time T. To avoid reliance on subjective measures in this phase, the researchers used logs-based productivity metrics such as median active coding time and wall-clock coding duration. The lagged dataset included 3,389 engineers.

By examining whether earlier changes in one variable predict later changes in the other, the researchers were able to infer directional causality. This additional modeling step significantly strengthens the study's conclusions regarding the impact of code quality.

6.3 Key Findings

6.3.1 Code Quality and Technical Debt

Project Code Quality Satisfaction with project code quality had the largest effect size among all predictors. Developers who reported improvements in the quality of the codebase they directly worked in also reported meaningful increases in productivity. Interestingly, satisfaction with dependency code quality was not significant. This suggests that developers are most impacted by the quality of the code they directly modify rather than upstream dependencies.

Technical Debt Technical debt within projects was also significantly associated with productivity changes. Developers who perceived lower technical debt reported higher productivity. Technical debt in dependencies was significant but had a smaller effect. This reinforces the practical intuition that maintainability and structural quality of codebases affect development velocity.

6.3.2 Infrastructure Tools and Support

Tool and Infrastructure Satisfaction Engineers who reported higher satisfaction with infrastructure and tools experienced higher productivity. Perceived innovation in tooling was also positively associated with productivity. Interestingly, both extremes of tool choice (too many or too few options) were associated with lower productivity. Similarly, rapid or slow changes in the developer tool stack were linked to reduced productivity.

Build and Test Latency Long build times were negatively associated with productivity. Engineers who experienced slower build cycles or reported being hindered by build and test processes showed lower productivity. This highlights that system latency and tooling friction create measurable productivity drag.

6.3.3 Team Communication

Among communication variables, code review dynamics were significant. Developers who experienced more review rounds or reported slow review processes tended to report lower productivity. This suggests that review inefficiencies introduce coordination overhead and delay feature delivery. However, physical distance from managers or reviewers, as well as organizational distance, were not significant. This challenges assumptions from distributed work literature that geographic distance inherently reduces productivity.

6.3.4 Goals and Priorities

Shifting project priorities showed one of the strongest negative associations with productivity. Developers who reported frequent priority changes experienced lower productivity. This finding emphasizes the importance of stability in project direction. Interruptions caused by goal shifts appear to impose cognitive switching costs that reduce development efficiency.

6.3.5 Organizational and Process Factors

Several structural variables were significant. Developers who experienced team or organizational changes, complicated processes, or direct manager changes reported lower productivity. Although the magnitudes were smaller than code quality effects, these findings indicate that structural instability creates measurable productivity impact.

6.3.6 Non-Significant Findings

Some expected productivity drivers were not statistically significant:

- Documentation support and documentation hindrance
- Meeting time (both median and total time spent in meetings)
- Physical and organizational distance

These results contradict common developer perceptions that meetings and documentation are primary productivity inhibitors. The panel analysis suggests these factors may correlate with productivity perceptions but are not causally linked when controlling for within-person changes.

6.3.7 Statistical Analysis Summary

The core model estimated was a fixed-effects panel regression examining within-developer changes over time:

$$\Delta y_{it} = \Delta\lambda_t + \beta\Delta D_{it} + \Delta\epsilon_{it} \tag{6.3}$$

where Δy_{it} is the change in self-rated productivity for engineer i at time t, $\Delta\lambda_t$ captures time fixed effects, ΔD_{it} is the vector of changes in the 39 independent variables, β is the vector of coefficients, and $\Delta\epsilon_{it}$ is the error term. The model was estimated using Feasible Generalized Least Squares (FGLS) to address heteroskedasticity and serial correlation in the panel data. The adjusted R^2 of the model came out to be $R^2 = 0.1019$. This indicates that approximately 10% of within-person productivity variation was explained by the included predictors.

6.3.8 Lagged Model Analysis

The two lagged models tested the directional relationship between code quality and productivity.

$$\Delta\text{Productivity}_T = \alpha + \beta_1\Delta\text{Code Quality}_{T-1} + \epsilon \tag{6.4}$$

$$\Delta\text{Code Quality}_T = \alpha + \beta_2\Delta\text{Productivity}_{T-1} + \epsilon \tag{6.5}$$

Results supported the first model but not the second. A 100% increase in project code quality at time $T-1$ led to (1) a 10% reduction in median active coding time; (2) a 12% reduction in wall-clock coding time; and (3) a 22% reduction in deployment latency at time T. In contrast, changes in productivity at time $T-1$ did not predict changes in code quality at time T. This asymmetry provides strong evidence that improvements in code quality lead to productivity gains, rather than productivity improvements enabling better code quality.

$$\text{Code Quality}_{T-1} \rightarrow \text{Productivity}_T \tag{6.6}$$

6.4 Implications

1. Strategic Implications for Engineering Leadership The most important implication of this study is that code quality is not merely a long-term maintainability concern — it is a direct driver of short-term developer productivity. Organizations often treat quality work (refactoring, reducing technical debt, improving design) as secondary to feature delivery. However, this research provides causal evidence that improving code quality leads to measurable improvements in productivity.

For engineering leadership, this reframes quality initiatives from "maintenance overhead" to "productivity investment." Allocating time for refactoring, improving code readability, reducing architectural

complexity, and systematically managing technical debt should be considered productivity-enhancing strategies rather than trade-offs against velocity. In practical terms, this suggests:

- Embedding code health goals into quarterly planning.
- Protecting refactoring time in sprint cycles.
- Evaluating teams not only on feature output but also on structural quality improvements.
- Treating technical debt reduction as a productivity accelerator.

2. Tooling and Infrastructure Investment The findings also show that satisfaction with tools and infrastructure has a strong causal relationship with productivity. This has important implications for internal platform teams and developer experience (DevEx) organizations. Organizations should (1) prioritize reducing build and test latency; (2) invest in tooling innovation that directly addresses developer pain points; and (3) carefully manage the balance between tool choice and standardization.

Importantly, this study suggests that infrastructure investments yield measurable productivity returns. Investments in faster builds, smoother deployment pipelines, and stable tooling ecosystems are not merely developer satisfaction initiatives — they directly influence performance.

3. Stability of Goals and Organizational Structure Another major implication concerns project stability. Shifting priorities and frequent reorganizations were associated with reduced productivity. This suggests that cognitive switching costs and structural instability impose real productivity penalties. Organizations should (1) minimize abrupt priority changes; (2) provide clearer long-term direction; (3) avoid unnecessary reorganizations; and (4) stabilize reporting structures where possible. This finding challenges the assumption that agility requires constant change. While adaptability is necessary, excessive strategic churn may undermine productivity at the individual level.

4. Reconsidering Common Productivity Assumptions One of the most striking implications comes from the non-significant findings. Documentation quality and meeting time were not found to be causally linked to productivity in this panel analysis. This does not mean these factors are unimportant, but it suggests:

- Meetings may not inherently reduce productivity when controlling for other variables.
- Documentation may have benefits (e.g., onboarding, inclusion, knowledge sharing) that do not directly manifest as short-term productivity gains.
- Some developer complaints may reflect perceived friction rather than measurable productivity loss.

For managers, this implies that decisions should rely on longitudinal evidence rather than intuition or one-time survey correlations.

Chapter 7

[Required] Using Logs Data to Identify When Sofware Engineers Experience Flow or Focused Work

This paper proposes a non-intrusive, logs-based method to identify when software engineers are engaged in focused work, which often includes instances of flow. Since traditional flow research relies heavily on self-reports (which are disruptive and retrospective), the authors aim to create an "always-on" behavioral metric using tool-generated logs (e.g., IDE usage, documentation views, code edits).

They first conducted a diary study to understand how engineers experience flow. They found that flow is highly subjective, task-independent (not limited to coding), and often overlaps strongly with focused work. However, because flow depends on personal feelings about the task (e.g., satisfaction, progress), it cannot be reliably detected from logs alone. As a result, the authors shifted their goal from detecting pure flow to detecting focused work, treating flow as a subset of focus.

To operationalize this, they developed a "focus time" metric based on task similarity. Using Word2Vec embeddings trained on millions of developer log sessions, they computed how semantically related an engineer's actions were within sliding time windows. Periods with highly related actions (low behavioral "distance") were labeled as focus sessions.

The metric was validated in two ways:

1. Against a large-scale diary study, where engineers marked when they felt "in flow or focused." The model showed strong agreement (median PABAK ≈ 0.71).
2. Against quarterly survey data from over 13,000 engineers, where frequency-based focus metrics significantly predicted self-reported flow/focus.

The authors conclude that while their metric does not distinguish pure flow from focused work, it provides a scalable, passive, and validated behavioral measure that can support future research on flow, productivity, and workplace design.

7.1 Motivation

Flow is widely regarded as an "optimal experience" and has long been associated with satisfaction and productivity, especially in knowledge work like software engineering. However, most existing research measures flow through self-reports (e.g., surveys, experience sampling), which are inherently retrospective and disruptive. These methods can only capture flow after it happens and may even interfere with the experience itself.

Attempts to measure flow non-intrusively (e.g., via physiological sensors, keystrokes, or partial log data) have had mixed success and often still require added hardware or interruptions. As a result, there is no

reliable, scalable, and passive way to detect flow as it occurs in real-world work environments.

The authors argue that instead of directly detecting flow—which is subjective and emotionally driven—it may be more feasible to detect focused work, since focus is a necessary precursor to flow. By leveraging rich logs data from engineering tools, they aim to build an "always-on," non-intrusive behavioral metric that identifies periods of focused work (which often include flow). This would enable large-scale, real-time measurement of focus and support future research on improving productivity and designing better work environments.

Focus Time Focus time is defined as a behavioral, logs-based metric that represents periods during which an engineer is engaged in focused work, which may include—but does not explicitly distinguish - instances of flow. Rather than attempting to detect the subjective experience of flow directly, the authors conceptualize focus as a prerequisite to flow and operationalize it through patterns of related actions over time. Focus time is calculated by analyzing sequences of tool-generated log events (e.g., IDE edits, documentation views, code builds) within sliding time windows and measuring the semantic similarity of these actions using learned embeddings. Periods in which actions are highly related—indicating sustained engagement with a coherent task—are classified as focus sessions. The metric is task-agnostic, robust to minor interruptions, and independent of subjective value judgments about the work, making it a scalable and non-intrusive proxy for focused work and, by extension, potential flow states.

Conceptually, the authors treat Flow $\subset$ Focus, meaning flow is a subset of focused work.

7.1.1 Related Work

Flow and focused work have a long history of research, primarily in the field of psychology. With respect to unpacking and measuring f;ow and focused work in software engineers and other knowledge workers, Human Computer Interaction (HCI) research has been at the core of understanding how people achieve fow in computer based work.

Flow and Focused Work The concept of fow was first articulated by Csikszentmihalyi in the 1970s and since then has been studied by researchers across a variety of domains. To summarize, fow has been defined as an enjoyable experience engaging in a task that is appropriately challenging and motivating

Flow and Focused Work in Engineering There is an extensive literature on productivity in software engineering, which considers time spent in fow or focused work to be an important factor. Sanchez et al. leveraged Integrated Development Environment (IDE) logs to observe that engineers engage in a high level of activity switching and fragmentation over the course of their day, which had negative impacts on productivity.

More recently, Chen et al. developed a measure of focus that leverages logs from a number of work tools (e.g., IDE, chat, internal wiki) by grouping together interactions that do not have interruptions (e.g., a chat message) and present descriptive statistics about how much time engineers spend in focus, as well as what factors impact this time.

7.2 Methodology

The study employed a multi-phase, mixed-methods research design to (1) understand how software engineers experience flow, (2) develop a logs-based behavioral metric of focused work (*"focus time"* Paragraph 7.1), and (3) validate this metric against self-reported data collected at different time scales. The methodology integrates qualitative research, large-scale behavioral log analysis, machine learning, and statistical validation.

7.2.1 The Experiment

7.2.1.1 Phase 1: Preliminary Diary Study (Understanding Flow in Context)

The authors recruited 18 software engineers across six countries at a large technology company. Participants had at least six months of tenure and had opted into research participation.

Over five working days, participants completed end-of-day diary surveys. They reported:

1. Whether they experienced flow,
2. When it occurred,
3. How long it lasted,
4. What activities they were performing,
5. Or why they did not experience flow.

A total of 75 diary responses were collected. Logs data corresponding to reported time periods were analyzed alongside diary responses. Additionally, six participants took part in semi-structured follow-up interviews to verify alignment between their reported flow experiences and their observed logs data.

Analysis A thematic analysis (inductive coding) was conducted on diary responses and interview transcripts. This resulted in:

- 3 themes describing how engineers experience flow,
- 6 categories of tasks performed during flow,
- 7 types of disruptions.

The qualitative phase revealed that flow is highly subjective. Similar behavioral patterns could be labeled as "flow" or "not flow" depending on retrospective emotional evaluation. Flow often occurs during focused work but cannot be distinguished behaviorally without subjective input. This led to a methodological shift: instead of detecting pure flow, the authors focused on detecting focused work, which is behaviorally observable and a prerequisite to flow.

7.2.1.2 Phase 2: Logs-Based Metric Development (Focus Time)

In Phase 2, the authors developed a logs-based behavioral metric called focus time (7.1) using large-scale internal activity data from software engineers. The data were drawn from a diverse suite of engineering tools used in day-to-day development work, including Integrated Development Environments (IDEs), code repositories, documentation systems, and communication tools. Each log entry represented a discrete event and contained structured metadata such as the tool name, the action type (e.g., edit, view, build), a timestamp marking when the action occurred, and an anonymized engineer identifier. Importantly, the dataset consisted only of behavioral events; user-generated content (e.g., message text or code content) was excluded to preserve privacy. The training corpus included approximately 12 million activity sessions generated by about 59,000 engineers over a 30-day period, providing a large-scale behavioral foundation for modeling focus.

To translate raw logs into a meaningful representation of focused work, the authors introduced the concept of *task similarity*. They hypothesized that focused work is characterized by engagement in semantically related actions within a bounded time window. As a first step, they constructed sessions by grouping events by engineer and ordering them chronologically. Sessions were segmented based on a 10-minute inactivity threshold, meaning that if no events occurred for 10 minutes, a new session was started. Each session therefore represented a contiguous block of activity. These sessions were treated analogously to sentences in natural language processing, where each event was replaced by a representative label (e.g., "IDE edit," "View documentation"). In this way, sequences of developer actions were converted into structured sequences suitable for embedding-based modeling.

Next, the authors trained a Word2Vec skip-gram model on these session sequences to learn distributed representations of developer activities. The model used 20-dimensional embeddings with a context window size of five, meaning that each event was trained to predict surrounding events within five positions. Events that occurred fewer than 200 times were excluded to ensure statistical robustness. Through this training process, events that frequently occurred in similar contexts were positioned close to one another in embedding space. For example, actions such as documentation viewing and internal code search clustered together because they often co-occur in related workflows. This learned embedding space

allowed the researchers to quantify the semantic similarity between actions, forming the computational foundation for identifying periods of focused work.

Focus Time Computation Focus time was calculated in sliding windows across event sequences. For each window:

1. Compute pairwise distances between all events in embedding space.
2. Weight distances by event duration.
3. Normalize distances between 0 and 1.

The focus value $F(W)$ for a window W is

$$F(W) = \frac{\sum_{e,f \in W} (|e| + B)(|f| + B) \operatorname{dist}(e, f)}{\sum_{e,f \in W} (|e| + B)(|f| + B)} \tag{7.1}$$

where, $|e|$ is the duration of event e, $\operatorname{dist}(e, f)$ is the distance between events e and f in embedding space (normalized between 0 and 1), and B is a smoothing constant to prevent zero-duration events from dominating the metric. *Lower focus values indicate more semantically related actions, which are indicative of focused work.*

7.2.1.3 Phase 3: Large-Scale Diary Validation

To evaluate whether the logs-derived focus time metric accurately reflected engineers' lived experiences, the authors conducted a large-scale diary study. Fifty-one software engineers across seven countries participated, and forty-six of them recorded two full workdays, resulting in 97 diary days in total. Participants were asked to log every activity they performed during the day using a structured digital form. For each activity, they recorded what they did, when they did it, and critically, whether they felt "in flow or focused" during that activity. This design enabled fine-grained alignment between subjective experiences and behavioral logs.

The validation was conducted using an agreement-based approach. The researchers compared time intervals marked as focused or in flow in the diary data against focus sessions identified by the logs-based metric. Agreement was quantified using Prevalence and Bias Adjusted Kappa (PABAK), which accounts for imbalance in classification. To optimize the metric, a grid search was performed across combinations of hyperparameters, including sliding window length and similarity threshold values. The optimal configuration achieved a median PABAK of 0.71, indicating strong agreement between the behavioral metric and self-reported focus. Importantly, the authors also tested naive behavioral proxies—such as long uninterrupted sessions or sessions with many events—and found that these showed substantially lower agreement, thereby justifying the embedding-based similarity approach.

7.2.1.4 Phase 4: Quarterly Survey Validation

In addition to short-term diary validation, the authors examined whether focus time aligned with longer-term perceptions of focus and flow using longitudinal survey data. The company administers a quarterly survey to software engineers, and the researchers analyzed responses from three consecutive quarters, encompassing 13,383 engineers. The relevant survey item asked participants how often they were able to reach a high level of focus or achieve flow during development tasks, using a five-point Likert scale.

To compare behavioral and survey data, focus time metrics were aggregated at the quarterly level in several forms: total hours spent in focus, percentage of active time spent in focus, total number of focus sessions, and percentage of days containing at least one focus session. Linear regression models were then used to estimate the relationship between these aggregated focus measures and self-reported focus/flow. The models controlled for job-related variables such as role, tenure, level, and job code, as well as general activity levels (e.g., total logged sessions).

The results showed that frequency-based measures—specifically the total number of focus sessions and the percentage of days with at least one focus session—were significant positive predictors of self-reported focus and flow. These relationships remained significant even after controlling for general activity levels,

and inclusion of the focus metrics significantly improved model fit. In contrast, duration-based measures (total hours or percentage of time in focus) were not significant predictors. This multi-level validation demonstrated that the focus time metric not only aligns with moment-to-moment subjective reports but also corresponds with broader perceptions of focus over extended periods.

7.2.2 Limitations

1. Observational Design Limits Causal Inference The study relies entirely on observational data, including logs-based behavioral records, diary entries, and longitudinal survey responses. Although regression models demonstrate statistically significant associations between focus time metrics and self-reported focus/flow, the authors do not claim that focus sessions cause flow experiences. The analyses establish correlation and predictive relationships but do not involve experimental manipulation. As a result, the findings cannot determine whether increased focus time leads to greater flow, whether flow leads to more coherent behavioral patterns, or whether both are driven by unmeasured third variables.

2. Logs Data Do Not Capture Internal Psychological States A central limitation acknowledged in the paper is that logs data reflect only observable behaviors and do not provide access to internal emotional or cognitive states. The preliminary diary study revealed that nearly identical behavioral patterns could be labeled as "flow" or "not flow" depending on how engineers felt about the outcome. This indicates that behavioral coherence alone does not determine flow. Consequently, even when focus time aligns with self-reported flow, the metric cannot causally explain why flow occurred, nor can it isolate the psychological mechanisms underlying the experience.

3. Missing Contextual Variables May Confound Relationships The authors note that the logs lack contextual information such as project identity and other environmental factors. For example, switching between unrelated projects may still appear behaviorally similar in the embedding space. Because such contextual details are not incorporated, the model may misclassify behavior, and unobserved contextual factors may influence both focus time and self-reported flow. This absence of contextual controls limits the ability to draw causal conclusions about how specific types of work or task structures influence focus or flow.

4. Diary and Survey Self-Reports Are Subject to Bias The validation of the focus time metric depends on self-reported diary and survey data, which are inherently retrospective and subjective. Engineers' reports of flow may be influenced by recall bias, emotional evaluation of outcomes, or response tendencies. Since the behavioral metric is validated against these subjective measures, any biases in self-report could affect observed associations. Therefore, while agreement is demonstrated, the study cannot establish causal validity of the metric independent of self-report biases.

7.3 Key Findings

Flow Is Highly Subjective and Closely Linked to Focused Work The preliminary diary study revealed that engineers do not experience flow consistently every day, and when they do, it typically occurs once per day and often in the morning. Flow episodes varied in duration, commonly ranging from 5 minutes to 3 hours. However, the most important finding was that flow is strongly subjective and tied to retrospective emotional evaluation. Engineers often described nearly identical behavioral patterns as either "flow" or "not flow" depending on whether they felt positive about the outcome. In other words, the behavioral signature alone was insufficient to determine whether flow occurred.

This finding led to a conceptual shift: while flow cannot be reliably detected from logs alone due to its emotional and experiential components, focused work can be behaviorally identified. The study therefore positioned focus as a measurable precursor to flow, with flow conceptualized as a subset of focused work.

Flow Occurs Across Diverse Tasks and Is Not Limited to Coding Another key insight from the qualitative phase was that engineers experience flow across a broad range of tasks, not only during coding. While coding was the most commonly reported flow activity, engineers also reported flow during documentation review, email management, planning, debugging, and other non-coding activities.

Additionally, flow was often maintained across multiple tools when those tools supported a unified task. Engineers described moving between IDEs, documentation systems, chat, and version control without feeling fragmented, as long as the actions were related to the same goal. This supports the "working sphere" concept: related actions across different tools can constitute continuous focused engagement.

Flow Can Withstand Small Interruptions Contrary to traditional assumptions that interruptions necessarily break flow, the study found that engineers could maintain flow despite minor disruptions, such as brief chat messages. Flow was more likely to break when interruptions were unrelated or required significant context switching, such as meetings or major task changes.

This finding directly informed the design of the focus time metric, which allows small interruptions within focus sessions rather than requiring uninterrupted blocks of activity.

Logs-Based Focus Time Accurately Aligns with Diary Reports In the large-scale diary validation phase, the focus time metric demonstrated strong agreement with self-reported "flow or focused" activities. The optimal model configuration achieved a median PABAK of 0.71, indicating substantial agreement. This level of agreement was comparable to previously validated observable behaviors such as time spent in meetings.

Importantly, naive behavioral assumptions—such as defining focus as simply long sessions or sessions with many events—performed poorly. These naive benchmarks yielded significantly lower agreement values. This demonstrates that focus cannot be reduced to duration or activity volume alone; instead, semantic relatedness of actions is a critical signal.

Frequency of Focus Sessions Predicts Self-Reported Flow Over Time In the longitudinal quarterly survey validation, aggregated focus metrics were tested against self-reported frequency of achieving focus or flow. The key finding was that frequency-based focus metrics—specifically the number of focus sessions and the percentage of days with at least one focus session—were significant positive predictors of self-reported flow.

These effects remained significant even after controlling for overall activity levels, indicating that simply working more does not explain perceived focus or flow. Moreover, including focus session metrics significantly improved model fit, strengthening the argument that the metric captures meaningful behavioral signals.

Interestingly, duration-based metrics (total hours spent in focus or percentage of time in focus) were not significant predictors of survey responses. This aligns with the wording of the survey item ("How often...") and with psychological literature suggesting that flow is associated with distorted time perception.

Focus Time Represents a Valid, Scalable, and Non-Intrusive Behavioral Metric Across validation phases, the study demonstrates that focus time is:

- Behaviorally grounded,
- Robust to minor interruptions,
- Task-agnostic,
- Strongly aligned with subjective reports of focus and flow,
- More predictive than simple duration-based heuristics.

While it does not differentiate pure flow from focused work, it provides the first scalable, passive, logs-based behavioral metric that meaningfully aligns with self-reported experiences at both short and long timescales.

7.4 Implications and Future Work

Advancing Non-Intrusive Measurement of Flow and Focus One of the central implications of this work is methodological. The study demonstrates that focused work can be measured passively and at scale using behavioral logs data, without requiring self-reports or physiological sensors. Traditional approaches to measuring flow—such as experience sampling, surveys, or wearable devices—are disruptive and retrospective. By contrast, the focus time metric operates continuously and unobtrusively.

This shifts the paradigm from relying on subjective interruptions to using real-world behavioral traces. *While the metric does not isolate "pure flow," it establishes a validated behavioral foundation for detecting focused work, which is a necessary precursor to flow.*

Reframing Flow as Behaviorally Overlapping with Focus The research introduces a conceptual implication: rather than treating flow as an entirely distinct state, it suggests that flow behaviorally overlaps with focused work. The study proposes a nested relationship:

$$\text{Flow} \subset \text{Focus}$$

This reframing implies that behavioral signatures of flow may be detectable indirectly through patterns of task-related engagement. It also suggests that flow research may benefit from first identifying sustained focus and then layering additional signals to detect emotional or experiential components.

Enabling Intervention Design and Workplace Optimization Because focus time is measurable at scale, organizations can now empirically investigate factors that increase or decrease focused work. The study's qualitative findings already highlight potential facilitators of flow and focus, such as

- Schedule management (e.g., minimizing fragmented meetings)
- Goal setting
- Dedicated uninterrupted time blocks

With a behavioral metric in place, organizations can experimentally test interventions like - (1) blocking "focus time" on calendars; (2) reducing meeting fragmentation; (3) encouraging structured goal-setting practices; and (4) modifying tool interfaces to reduce cognitive switching. The metric allows for quantitative evaluation of whether such interventions increase focus sessions over time.

Distinguishing Frequency from Duration An important implication of the statistical findings is that frequency of focus sessions predicts perceived flow, while duration does not. This suggests that the experience of flow may depend more on how often individuals reach a focused state rather than how long they remain in it. This has implications for productivity research. *Instead of encouraging long uninterrupted work blocks alone, organizations may benefit from increasing the number of opportunities for focused engagement throughout the day.*

Implications for Productivity Research Existing literature links self-reported flow to productivity. With a logs-based focus metric, researchers can now investigate whether behavioral focus sessions predict - (1) self-reported productivity; (2) objective productivity measures such as code commits, issue resolution rates, or peer evaluations; and (3) other outcomes like job satisfaction or burnout. The metric provides a new tool for empirically testing the relationship between focus, flow, and productivity in real-world settings.

7.4.1 Future Work

Identifying Additional Signals to Disambiguate Flow from Focus One important direction for future research is the development of additional signals to distinguish pure flow from focused work. The current focus time metric intentionally captures sustained engagement in related activities without attempting to differentiate between focus and the more nuanced psychological experience of flow. However, flow is traditionally characterized by additional components such as intrinsic enjoyment, challenge-skill

balance, clear goals, and altered time perception. Future research could integrate complementary behavioral, contextual, or even lightweight physiological signals to capture these dimensions. For example, incorporating task difficulty indicators, goal-setting markers, or measures of challenge could help refine the metric and move closer to detecting flow itself rather than focus as a proxy.

Investigating Drivers of Focus and Flow A second promising avenue is investigating the factors that influence focus time and experimentally testing interventions aimed at increasing it. The qualitative findings in the study suggest that schedule management, goal clarity, and uninterrupted time blocks may facilitate flow and focused work. With a scalable behavioral metric now available, researchers can conduct controlled experiments to evaluate whether interventions—such as reducing meeting fragmentation, blocking dedicated focus time on calendars, or encouraging structured goal-setting—lead to measurable increases in focus sessions. This would allow organizations to move from intuition-based productivity strategies to evidence-based workplace design.

Examining the Relationship Between Focus and Productivity A third direction involves examining the relationship between focus time and productivity outcomes. Although prior literature links self-reported flow with productivity, it remains an open question whether logs-derived focus sessions predict objective performance measures. Future research could explore associations between focus time and indicators such as task completion rates, code quality, review efficiency, or reduced context switching. Establishing such links would not only validate the practical significance of the metric but also deepen our understanding of how sustained behavioral coherence translates into meaningful work outcomes.

Chapter 8

[Required] Creativity, Generative AI, and Software Development: A Research Agenda

This paper presents a research agenda examining how Generative AI (GenAI) may reshape creativity in software development . Using the 4P framework of creativity (Person, Product, Process, and Press/environment) alongside McLuhan's tetrad, the authors systematically explore how GenAI could enhance, obsolete, retrieve, or reverse aspects of creative work. They outline optimistic and pessimistic future scenarios, ranging from GenAI augmenting human creativity by freeing developers from mundane tasks to potentially diminishing human creative skills through overreliance. Based on 76 identified potential impacts, the paper proposes six interconnected research themes—individual capabilities, team capabilities, product outcomes, unintended consequences, societal impact, and human aspects—calling for longitudinal and interdisciplinary research to ensure GenAI amplifies positive creative effects while mitigating risks to developers, products, and society.

8.1 Motivation and Related Work

Creativity has long been recognized as a foundational element of software development, shaping both routine problem-solving and breakthrough innovation. As Glass argues, creativity differentiates ordinary software work from exceptional contributions [1]. Everyday "Little-c" (Definition 8.5.1) creativity supports tasks such as debugging, refactoring, and feature implementation, while "Big-C" (Definition 8.5.2) creativity enables major architectural shifts and novel product visions. Creativity in organizational settings depends on the interaction of individual traits, domain expertise, and supportive environments, as articulated in established models of creativity [2]. Despite its importance, creativity in software engineering remains comparatively underexplored in empirical research [3]. The rapid integration of Generative AI (GenAI) into development workflows introduces a potentially transformative force, with early evidence suggesting that AI-powered tools can assist developers in idea generation and creative exploration [4].

Much of the existing discourse surrounding GenAI in software engineering has centered on productivity gains rather than creative transformation. Empirical studies on AI-powered pair programming and coding assistants have primarily measured efficiency and task completion improvements [5, 6]. At the same time, public discourse has emphasized the possibility that AI systems may automate a substantial proportion of coding tasks in the near future [7]. If GenAI increasingly "takes the rote out" of software development, creativity may emerge as the primary remaining human differentiator. In such a context, understanding how GenAI influences creative processes, outcomes, and professional roles becomes not only an academic concern but a strategic imperative for the discipline.

Finally, the motivation for this work arises from a clear research gap: there is currently no systematic

body of research directly examining the impact of GenAI on creativity in software development. While related studies address productivity, code generation, and quality, the creative dimension has not been analyzed in a structured manner. Given the disruptive trajectory of GenAI and its expanding integration into software engineering practice, it is both timely and necessary to articulate a forward-looking research agenda that rigorously investigates its short-, medium-, and long-term implications for human creativity.

8.1.1 Related Work

Creativity in software development has historically received limited direct empirical attention. A systematic review by Amin et al. highlights that creativity remains an underexplored area within software engineering research [3]. Existing studies have examined creativity at the individual level, including the influence of personality traits and knowledge behaviors on programmer creativity [8], as well as the inhibitory effects of overly templated requirements on creative design processes [9]. At the product level, work on open-source ecosystems has explored innovation through novel reuse and community dynamics [10]. Process-oriented research has investigated techniques that foster creative thinking, such as structured ideation in requirements engineering [11] and collaborative practices like whiteboarding and hybrid teamwork [12, 13].

Parallel to this, research on Large Language Models (LLMs) in software engineering has grown rapidly, primarily focusing on code generation, testing, and productivity outcomes rather than creativity itself [14, 15]. While these surveys acknowledge the transformative potential of GenAI tools, none explicitly center creativity as a primary construct. Consequently, there remains a clear gap in systematically examining how Generative AI may reshape creative processes, outcomes, and environments in software development.

8.2 Methodology

The paper adopts a structured, exploratory methodology to investigate the potential impact of Generative AI (GenAI) on creativity in software development. Rather than conducting empirical experiments, the authors employ a conceptual and analytical approach grounded in established theoretical frameworks. Specifically, they combine the 4P framework of creativity [16] with McLuhan's tetrad [17] and elements of the Disruptive Research Playbook [18] to systematically identify and organize potential impacts.

Phase 1: Scenario Development The first phase involved constructing a spectrum of plausible future scenarios describing how GenAI may evolve within software development. The authors explicitly consider both "optimistic" and "pessimistic" futures. In the optimistic scenario, GenAI offloads "mundane work," allowing developers more time for creative thinking. In contrast, the pessimistic scenario envisions GenAI performing "almost all the work," potentially diminishing the human role in creative processes.

These scenarios were not intended as predictions but as structured thought experiments to explore the breadth of possible outcomes. The authors emphasize that "any research agenda should include the possible spectrum of impact that may transpire," ensuring coverage of both short-term and long-term consequences.

Phase 2: Application of McLuhan's Tetrad In the second phase, the authors apply McLuhan's tetrad [17] as an analytical lens. The tetrad poses four guiding questions about any disruptive technology:

- What does the technology enhance?
- What does it make obsolete?
- What does it retrieve from obsolescence?
- What does it reverse into when pushed to extremes?

GenAI is selected as the disruptive technology under investigation, and creativity in software development is defined as the focal phenomenon. To structure the analysis, the tetrad is applied separately to each component of the 4P framework of creativity [16]:

1. Person (individual developer),
2. Product (creative outcomes),
3. Process (creative methods),
4. Press (environment).

This resulted in four distinct tetrads, each populated through structured brainstorming among the researchers. The authors note that the tetrad "merely provides a framework for identifying future possibilities," requiring imagination and collective reasoning rather than empirical measurement.

Brainstorming and Impact Identification Drawing on the developed scenarios and the tetrad structure, the research team collectively generated potential impacts for each 4P dimension. The process incorporated both everyday "Little-c" (Definition 8.5.1) creativity and "Big-C" (Definition 8.5.2) innovation, ensuring coverage of routine problem-solving and breakthrough advances. This structured brainstorming produced 76 potential impacts across the four tetrads. These impacts are treated as hypotheses or research opportunities rather than empirical findings. The methodology thus serves to systematically surface possibilities rather than validate them.

Derivation of the Research Agenda Finally, instead of converting each identified impact into an isolated research question, the authors synthesize them into six interconnected research themes:

1. Individual capabilities
2. Team capabilities
3. The product
4. Unintended consequences
5. Societal impact
6. Human aspects

This thematic abstraction aligns with Steps 3 and 4 of the Disruptive Research Playbook [18], which recommend transforming disruptive insights into structured research programs. The methodology therefore transitions from exploratory scenario analysis to a coherent research agenda intended to guide future empirical and longitudinal studies.

Overall, the methodology is conceptual, theory-driven, and forward-looking. It leverages established creativity and media theory frameworks to anticipate and structure inquiry into the evolving relationship between GenAI and creativity in software development.

8.3 Key Findings

8.3.1 Analysis of the 4P Framework

8.3.1.1 Person (Individual Creativity)

At the individual level, the paper suggests that Generative AI may significantly reshape how developers engage in creative work. GenAI can act as an ideation partner, expanding cognitive reach by generating alternatives, offering cross-domain insights, and reducing design fixation. This may lower the creative barrier to entry and democratize creative contribution across developers with varying levels of experience. However, the same mechanisms that amplify creativity may also weaken foundational cognitive skills such as deep reflection, independent reasoning, and complex problem-solving. The long-term concern is not whether individuals become more productive, but whether sustained reliance may gradually erode the very capacities that enable independent creative thinking.

- **Enhances:**
  - Idea generation and avoidance of design fixation

  - Cross-domain inspiration
  - Creative prompts and checks/balances
  - Assessment of alternative designs
- **Obsolesces:**
  - Dependence on personality traits traditionally linked to creativity
  - Deep reflective thinking
  - Traditional expertise accumulation
  - Mentorship as primary creative guidance
- **Retrieves:**
  - Manual sketching and doodling
  - Verification and evaluation skills
  - Greater emphasis on purely creative work
- **Reverses Into:**
  - Loss of second-guessing behavior
  - Reduced exploration of alternatives
  - Erosion of problem-solving skills

8.3.1.2 Product (Creative Outcomes)

At the product level, GenAI may increase both the speed and scale of innovation. By synthesizing design patterns, analyzing user feedback, and generating novel feature combinations, it can accelerate continuous enhancement and enable highly personalized user experiences. This may expand the diversity of technical possibilities while reducing development friction. However, the paper highlights a paradox: while GenAI can generate novel outputs, widespread reliance may also lead to convergence, where products increasingly resemble one another due to shared model biases and training data. Furthermore, the risk of biased, overly complex, or even harmful features becoming embedded in products introduces new ethical and quality concerns.

- **Enhances:**
  - Continuous delivery of innovative features
  - Customized and adaptive user experiences
  - Identification of new product opportunities
  - Radical innovation possibilities
- **Obsolesces:**
  - Routine or generic solutions
  - Dependence on fixed frameworks
  - Traditional intermediate design artifacts
  - Specialty artifacts produced manually
- **Retrieves:**
  - Forgotten design patterns and styles
  - Legacy system design ideas
  - Analog design principles

- **Reverses Into:**
  - Homogeneous products
  - Echo chambers in design suggestions
  - Overly complex or impractical designs
  - Biased or harmful product features

8.3.1.3 Process (Creative Methods)

GenAI has the potential to fundamentally transform creative processes within software development teams. By automating preparatory and routine tasks, it may free cognitive and temporal resources for experimentation and ideation. It can also serve as a moderator or facilitator in collaborative settings, potentially improving coordination across diverse teams. However, the automation of ideation and evaluation processes raises concerns about diminished human intuition, reduced constructive disagreement, and weakening of traditional creativity techniques. Over time, teams may become increasingly dependent on AI-generated direction, risking a decline in collective creative competence.

- **Enhances:**
  - Automation of mundane creative inputs
  - Rapid prototyping and iteration
  - Cross-disciplinary collaboration
  - AI-moderated brainstorming
- **Obsolesces:**
  - In-person brainstorming sessions
  - Strictly linear development processes
  - Traditional design techniques (e.g., focus groups)
  - Rigid specialization boundaries
- **Retrieves:**
  - Pair and mob programming
  - Structured idea-generation techniques
  - Human-centered design emphasis
  - Cross-domain study for inspiration
- **Reverses Into:**
  - Loss of intuitive decision-making
  - Erosion of creativity technique proficiency
  - Reduced constructive disagreement
  - Increased roteness of developer roles

8.3.1.4 Press (Creative Environment)

The creative environment—encompassing organizational culture, physical space, and social interaction—may also be reshaped by GenAI integration. AI systems could enhance contextual awareness, optimize resource allocation, and improve distributed collaboration through intelligent environmental support. Creativity may become more explicitly recognized as a strategic differentiator within organizations. Nevertheless, excessive reliance may lead to diminished human interaction, isolation, weakened team identity, and reduced pride in creative contribution. In extreme cases, organizational trust may shift from human expertise to AI systems, altering professional agency and psychological safety.

- **Enhances:**
  - Contextual information support
  - Adaptive creative environments
  - Improved virtual collaboration
  - Greater organizational focus on creativity
- **Obsolesces:**
  - Traditional whiteboards and sticky notes
  - Fixed office layouts
  - Physical co-location requirements
  - Dependence on colleagues for creative support
- **Retrieves:**
  - Physical movement and embodied creativity
  - Supportive management recognition of creativity
- **Reverses Into:**
  - Social isolation
  - Reduced pride and agency
  - Management overreliance on AI
  - Devaluation of creativity as a distinct skill

8.3.2 Other findings

Reframing the Discourse: From Productivity to Creativity The paper establishes that the discourse surrounding GenAI in software engineering has been disproportionately centered on productivity, leaving creativity comparatively underexamined. While empirical and industry studies have emphasized efficiency gains, task completion speed, and code generation performance, the authors argue that creativity warrants equal—if not greater—attention. They position creativity not merely as a peripheral benefit of automation, but as a potential core differentiator in a future where routine technical tasks are increasingly delegated to AI systems. This reframing represents a significant conceptual contribution: it shifts the research lens from "how fast can we build?" to "what does human creative contribution become?"

Identifying a Critical Research Gap in GenAI and Creativity This paper highlights the existence of a substantial research gap. Despite growing literature on Large Language Models (LLMs) and AI-assisted development, no systematic body of research directly investigates the impact of GenAI on creativity in software development. The authors identify this absence as both surprising and concerning, especially given that creativity has historically been acknowledged as central to innovation in software practice. By explicitly identifying this gap, the paper justifies the need for a forward-looking research agenda and positions creativity as a primary research construct rather than a secondary outcome.

A Multi-Level Perspective on Creative Transformation The authors emphasize that the impact of GenAI must be examined across multiple units of analysis. Creativity is not confined to individuals; it emerges through interactions among individuals, teams, artifacts, and environments. The paper therefore argues for a multi-level perspective that includes individual capabilities, team dynamics, product outcomes, unintended consequences, societal implications, and human psychological aspects. This broader systems-oriented view expands the scope of inquiry beyond technical augmentation and toward socio-technical transformation.

The Need for Longitudinal and Interdisciplinary Inquiry Finally, the paper underscores the importance of longitudinal and interdisciplinary research. Many of the potential consequences—such as erosion of expertise, homogenization of products, shifts in professional identity, or societal revaluation of the software profession—may unfold gradually over time. The authors therefore advocate for proactive investigation rather than reactive correction. They stress that understanding both positive amplification and potential harms is essential to shaping responsible GenAI integration. In this sense, one of the paper's key findings is normative: researchers and practitioners must intentionally guide the evolution of GenAI in software development, rather than allowing its trajectory to be defined solely by technological momentum.

Collectively, these findings extend beyond the tetrad tables and establish the paper's central message: GenAI is not merely a productivity tool but a transformative force with deep implications for creativity, professional identity, organizational practice, and society at large.

8.4 Implications

1. Creativity Must Become a Core Research Focus in Software Engineering The paper explicitly argues that creativity deserves equal—if not greater—attention than productivity in the era of GenAI. A key implication is that future software engineering research should formally operationalize and measure creativity, rather than treating it as an indirect or secondary outcome. This reorientation shifts research priorities from efficiency metrics toward creative capability, innovation diversity, and long-term cognitive impact.

2. Proactive Monitoring of Unintended Consequences The authors emphasize the need to anticipate and measure unintended consequences, including erosion of expertise, over-reliance on AI, homogenized products, and bias amplification. The implication is that researchers should develop early assessment metrics and longitudinal benchmarks to detect negative shifts before they become systemic. Waiting for harm to manifest at scale would be insufficient.

3. Multi-Level, Systems-Oriented Research Design The work explicitly calls for research across six themes: individual capabilities, team capabilities, product outcomes, unintended consequences, societal impact, and human aspects. The implication is that studying GenAI purely at the tool level is inadequate. Instead, future research must adopt socio-technical and systems-level approaches that integrate psychological, organizational, and societal dimensions.

4. Interdisciplinary Collaboration is Necessary The paper clearly notes that understanding societal impact and human aspects requires expertise beyond software engineering. Psychologists, sociologists, economists, and policy scholars must be involved. The implication is that GenAI-creativity research cannot remain discipline-bound.

8.4.1 Extended Implications (Self Research)

5. Redefinition of Software Engineering Education If GenAI automates large portions of implementation and routine ideation, curricula may need to shift toward - (1) Creative problem framing; (2) Critical evaluation of AI outputs; (3) Cross-domain synthesis; and (4) Ethical design thinking. The paper hints at skill shifts, but the educational restructuring implication is an extrapolation that follows logically from the identified changes in creative processes and outcomes.

6. Emergence of "Creative Verification" as a Core Skill While the paper discusses verification skills, a broader implication is that developers may transition from creators to curators and evaluators of AI-generated creativity. This suggests a new professional identity centered around oversight, refinement, and contextual judgment.

7. Ethical Governance Frameworks for Creative AI Use Given risks of bias, homogenization, and harmful product features, there may be a need for - (1) Governance standards for AI-assisted creative

workflows; (2) Auditing frameworks for AI-generated design decisions; (3) Transparency requirements in AI-influenced products. This builds upon the paper's societal and unintended consequence themes.

8.5 Appendix

Definitions

Definition 8.5.1 (Little-c Creativity)**.** *Everyday, domain-level creativity involving novel and useful problem solving within routine tasks (e.g., debugging, refactoring, feature design).*

Definition 8.5.2 (Big-C Creativity)**.** *Eminent, transformative creativity that produces groundbreaking innovations or paradigm shifts with broad and lasting impact.*

References

[1] Robert L. Glass. *Software Creativity*. Prentice-Hall, Inc., 1994.

[2] Teresa M. Amabile et al. "A Model of Creativity and Innovation in Organizations". In: *Research in Organizational Behavior* 10.1 (1988), pp. 123–167.

[3] Aamir Amin et al. "A Snapshot of 26 Years of Research on Creativity in Software Engineering – A Systematic Literature Review". In: *International Conference on Mobile and Wireless Technologies (ICMWT)*. 2018, pp. 430–438.

[4] Birgitta Bockeler and Ryan Murray. *Generative AI and the Software Development Lifecycle: Much More Than Coding Assistance*. https://www.thoughtworks.com/en-us/insights/articles/generative-ai-software-development-lifecycle-. Accessed: 2026-02-25. 2023.

[5] Christian Bird et al. "Taking Flight with Copilot: Early Insights and Opportunities of AI-Powered Pair-Programming Tools". In: *Queue* 20.6 (2023), pp. 35–57. DOI: 10.1145/3582083.

[6] Albert Ziegler et al. "Measuring GitHub Copilot's Impact on Productivity". In: *Communications of the ACM* 67.3 (2024), pp. 54–63. DOI: 10.1145/3633453.

[7] Ian Scheffler. *GitHub CEO Says Copilot Will Write 80% of Code "Sooner Than Later"*. https://www.freethink.com/robots-ai/github-copilot. Accessed: 2026-02-25. 2023.

[8] Aamir Amin et al. "The Impact of Personality Traits and Knowledge Collection Behavior on Programmer Creativity". In: *Information and Software Technology* 128 (2020), p. 106405.

[9] Rahul Mohanani et al. "How Templated Requirements Specifications Inhibit Creativity in Software Engineering". In: *IEEE Transactions on Software Engineering* 48.10 (2021), pp. 4074–4086.

[10] Hongbo Fang, James Herbsleb, and Bogdan Vasilescu. "Novelty Begets Popularity, but Curbs Participation – A Macroscopic View of the Python Open-Source Ecosystem". In: *Proceedings of the International Conference on Software Engineering (ICSE)*. 2024.

[11] Neil Maiden, Antonis Gizikis, and Suzanne Robertson. "Provoking Creativity: Imagine What Your Requirements Could Be Like". In: *IEEE Software* 21.5 (2004), pp. 68–75.

[12] Wouter Groeneveld et al. "Exploring the Role of Creativity in Software Engineering". In: *ICSE-SEIS*. 2021, pp. 1–9.

[13] Victoria Jackson et al. "Team Creativity in a Hybrid Software Development World: Eight Approaches". In: *IEEE Software* 40.2 (2022), pp. 60–69.

[14] Angela Fan et al. "Large Language Models for Software Engineering: Survey and Open Problems". In: *arXiv preprint arXiv:2310.03533* (2023).

[15] Xinyi Hou et al. "Large Language Models for Software Engineering: A Systematic Literature Review". In: *arXiv preprint arXiv:2308.10620* (2024).

[16] Mel Rhodes. "An Analysis of Creativity". In: *The Phi Delta Kappan* 42.7 (1961), pp. 305–310.

[17] Marshall McLuhan. "Laws of the Media". In: *ETC: A Review of General Semantics* (1977), pp. 173–179.

[18] Margaret-Anne Storey et al. "A Disruptive Research Playbook for Studying Disruptive Innovations". In: *arXiv preprint arXiv:2402.13329* (2024).

Chapter 9

[Required] Beliefs, Practices, and Personalities of Software Engineers: A Survey in a Large Software Company

This paper reports on an exploratory survey of 797 engineers at Microsoft examining the relationship between personality traits, work practices, and professional beliefs. Using the Five-Factor Model (Openness, Conscientiousness, Extraversion, Agreeableness, and Neuroticism) and statistical analyses including Kruskal–Wallis tests with multiple-hypothesis correction, the authors analyzed differences across roles, practices, and viewpoints.

The results reveal relatively few strong personality differences. No significant differences were found between developers and testers, while managers were more conscientious and extraverted. Engineers who built their own tools were more open, conscientious, and extraverted, and less neurotic. Agreement with statements such as "Agile development is awesome" was also associated with higher extraversion and lower neuroticism. Overall, the findings suggest that personality differences among software engineers exist but are limited in scope.

9.1 Motivation

There has been sustained interest in understanding what makes effective and successful software engineers, particularly as software development has become increasingly collaborative and complex. Prior research has explored the characteristics of high-performing engineers and the broader question of what differentiates great software professionals from their peers [1]. At the same time, a substantial body of work has investigated the role of personality in software engineering, examining how individual traits relate to job satisfaction, software quality, team effectiveness, and project outcomes [2, 3]. These studies suggest that personality traits may influence not only individual performance but also team dynamics and the overall success of software projects [4].

Despite decades of research, empirical evidence remains fragmented, and findings are sometimes inconsistent. For example, prior studies have reported personality differences across software roles such as developers and testers [5], yet these findings have not been broadly validated in large industrial contexts. Moreover, much of the existing work has focused on personality in relation to performance or team composition, rather than examining how personality relates to engineers' everyday work practices and beliefs about controversial or widely debated development practices. Understanding these relationships is important because beliefs about methodologies, tooling, and development practices shape engineering decisions, team culture, and adoption of processes. Therefore, the paper is motivated by the need to systematically examine, within a large industrial setting, how personality traits relate to engineers' work

practices and beliefs, and whether personality meaningfully differentiates subgroups within the software engineering population.

9.2 Methodology

The study employed a survey-based empirical methodology to investigate the relationship between software engineers' personality traits, work practices, and professional beliefs. The research was conducted within a large industrial organization and targeted engineers in the United States. A total of 3,000 engineers were invited to participate in an anonymous online survey, and 797 responses were received, corresponding to a 26% response rate. The survey was advertised as a "Developer Personality Survey," and participation was voluntary. To encourage participation, the invitation was personalized, anonymity was preserved, and respondents were offered entry into a gift-card drawing via a separate communication channel to maintain confidentiality .

The survey instrument consisted of two main components: a personality assessment and a set of non-personality questions addressing demographics, work practices, and beliefs. Personality was measured using the International Personality Item Pool (IPIP), a widely used public-domain instrument aligned with the Five-Factor Model[1] (FFM) of personality [6]. The Five-Factor Model—often referred to as the OCEAN model—comprises five dimensions:

- Openness to experience,
- Conscientiousness,
- Extraversion,
- Agreeableness,
- Neuroticism.

The personality assessment included 50 IPIP items. Only engineers based in the United States were surveyed to minimize cultural and linguistic ambiguity in interpreting survey items, particularly those containing idiomatic expressions .

Following the personality inventory, participants answered 23 additional multiple-choice questions categorized as follows:

- 3 demographic questions (e.g., role, management status, educational background),
- 12 work-practice questions (e.g., tool usage, remote work, coding habits),
- 8 belief-related questions (e.g., opinions on Agile, code reviews, static vs. dynamic typing).

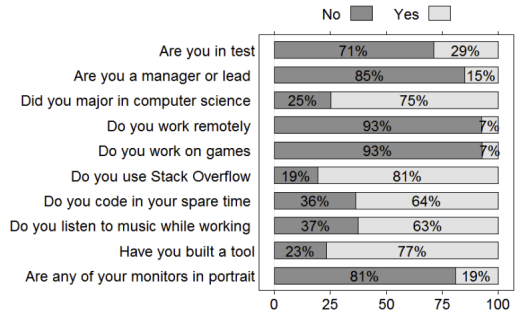
Belief questions were derived in part from controversial programming discussions, with the intention of capturing variation in strongly held professional viewpoints. All non-personality items were structured as categorical responses (e.g., Yes/No or Disagree/Neutral/Agree).

The data analysis proceeded in two stages. First, descriptive statistics were computed to examine the distributions of responses to all non-personality questions. This step provided an overview of the demographic composition, work-practice patterns, and diversity of beliefs within the sample .

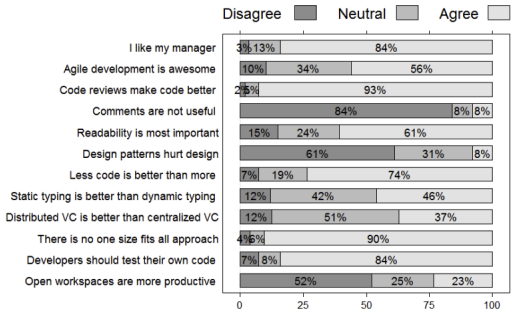
Second, inferential statistical analysis was conducted to identify relationships between personality traits and responses to non-personality questions. Because several survey questions contained more than two response categories, the authors employed the Kruskal–Wallis rank-sum test to evaluate differences in personality scores across groups. For each of the 23 non-personality questions, five separate tests were conducted—one for each personality dimension—resulting in 115 hypothesis tests in total.

To enable comparison across groups, personality scores were normalized such that the mean was set to 25 and the standard deviation to 5, following established normalization practices in cross-population personality research [9]. Higher scores indicate stronger expression of the respective personality trait.

[1]The authors selected the Five-Factor Model rather than the Myers–Briggs Type Indicator (MBTI) due to its stronger theoretical grounding, higher empirical validity, and superior test–retest reliability [7, 8].



(a) Distribution of demographic and work-practice responses.



(b) Distribution of beliefs about development practices.

Figure 9.1: Survey response distributions

Given the large number of statistical tests performed, the authors applied the Benjamini–Hochberg procedure to control the false discovery rate and mitigate the risk of Type I errors due to multiple hypothesis testing [10]. Initially, 20 differences were statistically significant at the $\alpha = 0.05$ level. After adjustment for multiple comparisons, 6 differences remained statistically significant at the same threshold. The authors reported both pre- and post-correction results to balance caution against potential loss of informative findings.

Overall, the methodology combines psychometrically validated personality measurement, structured survey design, and non-parametric statistical analysis with multiple-testing correction to rigorously examine the relationship between personality, beliefs, and work practices in a large industrial software engineering context.

9.3 Key Findings

Distribution of Beliefs and Work Practices The survey results indicate substantial agreement among engineers on several core development practices, alongside clear disagreement on others Figure 9.1a. A majority of respondents agreed that developers should test their own code, that code reviews and comments enhance code quality, and that readability is a central attribute of well-written software. Similarly, more than half endorsed the view that less code is preferable to more code.

In contrast, certain topics generated limited consensus (Figure 9.1b). Opinions regarding static versus dynamic typing and distributed versus centralized version control systems were widely dispersed, with a large proportion of neutral responses. The question of open workspaces proved particularly controversial, with a majority rejecting the claim that they improve productivity, while a notable minority supported it.

Limited Personality Differences Across Roles One of the most notable findings is the absence of significant personality differences between developers (SDEs) and testers (SDETs). This suggests that role assignment within the organization does not strongly correspond to differences in core personality traits as measured by the Five-Factor Model.

However, differences were observed for leadership roles. Managers and leads exhibited higher levels of conscientiousness and extraversion compared to non-managers. This pattern indicates that leadership positions may be associated with greater organizational discipline, reliability, and sociability.

Personality and Work Practices The analysis identified statistically significant relationships between personality traits and specific work practices. Engineers who reported listening to music while working were generally more open and extraverted and slightly less conscientious.

More substantial differences were observed among engineers who had built tools on their own initiative. These individuals scored higher in openness, conscientiousness, and extraversion, and lower in neuroticism

compared to those who had not built such tools. This pattern suggests that proactive technical initiative may be associated with cognitive flexibility, organizational reliability, social engagement, and emotional stability.

Personality and Professional Beliefs Personality traits were also associated with specific professional beliefs. The clearest example concerns attitudes toward Agile development. Engineers who agreed with the statement "Agile development is awesome" scored higher on extraversion and lower on neuroticism compared to those who disagreed.

Additionally, openness was positively associated with agreement that distributed version control systems are preferable to centralized systems. These results suggest that personality traits may shape how engineers evaluate development methodologies, collaboration structures, and tooling philosophies.

Robustness After Multiple Hypothesis Testing Initial statistical testing identified 20 significant personality differences at the $p < 0.05$ level. However, after applying the Benjamini–Hochberg correction to control the false discovery rate, only 6 differences remained statistically significant. This adjustment underscores the importance of accounting for multiple hypothesis testing and indicates that, although certain personality associations exist, the overall magnitude of differences is limited.

The reduction in statistically significant findings reinforces the broader conclusion that personality differences within the engineering population are present but not pervasive.

9.3.1 Threats to Validity

The primary threat to validity arises from the study's confinement to a single large industrial organization. Although the organization exhibits substantial internal diversity in terms of engineering practices and employee backgrounds, the findings may not generalize to smaller companies, startups, or open-source communities. Additionally, the survey was explicitly framed as a "Developer Personality Survey," which introduces the possibility of self-selection bias; individuals with an interest in personality assessment may have been more inclined to participate. This could systematically influence the distribution of personality traits within the respondent sample. Furthermore, as with any self-reported survey, responses may be affected by social desirability bias or subjective interpretation of survey items, potentially influencing both reported beliefs and personality measures.

9.4 Implications

Limited Role of Personality Testing in Hiring The findings suggest that personality differences among software engineers are relatively limited in scope. Given the small number of statistically significant differences observed, the results imply that the role of personality testing in hiring decisions may be less influential than sometimes assumed. However, the study does not draw definitive conclusions on this matter and emphasizes that additional research—particularly regarding the social dynamics of engineering teams—is necessary before making strong claims about the utility of personality assessments in recruitment or selection processes.

Absence of Expected Differences An important implication arises from the absence of personality differences in areas where such distinctions might have been anticipated. No significant differences were observed between developers and testers, nor for engineers working remotely versus locally, working on games, or holding particular views about open workspaces or typing paradigms. This suggests that common assumptions about personality-based distinctions across roles or preferences may not be strongly supported in this industrial context.

Opportunities for Future Research The study highlights broader opportunities for continued investigation into personality within software engineering teams. In particular, it underscores the need for further research on how personality interacts with collaborative environments, team structures, and organizational contexts. Additionally, the authors emphasize the value of systematically polling engineers

about their workplace practices and beliefs, indicating that large-scale empirical surveys can provide foundational statistics and open new avenues for understanding human factors in software engineering.

References

[1] P. L. Li, A. J. Ko, and J. Zhu. "What Makes a Great Software Engineer?" In: *Proceedings of the International Conference on Software Engineering (ICSE)*. 2015.

[2] S. S. J. O. Cruz, F. Q. B. da Silva, and L. F. Capretz. "Forty years of research on personality in software engineering: A mapping study". In: *Computers in Human Behavior* 46 (2015), pp. 94–113.

[3] L. F. Capretz. "Personality types in software engineering". In: *International Journal of Human-Computer Studies* 58.2 (2003), pp. 207–214.

[4] L. F. Capretz and F. Ahmed. "Making sense of software development and personality types". In: *IT Professional* 12.1 (2010), pp. 6–13.

[5] T. Kanij, R. Merkel, and J. Grundy. "An Empirical Investigation of Personality Traits of Software Testers". In: *Proceedings of the International Workshop on Cooperative and Human Aspects of Software Engineering (CHASE)*. 2015.

[6] Lewis R. Goldberg et al. "The International Personality Item Pool and the future of public-domain personality measures". In: *Journal of Research in Personality* 40 (2006), pp. 84–96.

[7] Paul T. Costa and Robert R. McCrae. *Revised NEO Personality Inventory (NEO PI-R) and NEO Five-Factor Inventory (NEO FFI): Professional Manual*. Psychological Assessment Resources, 1992.

[8] David J. Pittenger. "Measuring the MBTI... and coming up short". In: *Journal of Career Planning and Employment* 54 (1993), pp. 48–52.

[9] David P. Schmitt et al. "The Geographic Distribution of Big Five Personality Traits". In: *Journal of Cross-Cultural Psychology* 38.2 (2007), pp. 173–212.

[10] Yoav Benjamini and Yosef Hochberg. "Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing". In: *Journal of the Royal Statistical Society: Series B* 57.1 (1995), pp. 289–300.

Part II

Reading Notes - March

Chapter 10

[Required] Requirements Elicitation: A Survey of Techniques, Approaches and Tools

The paper presents a comprehensive review of the state of the art and practice in requirements elicitation within software engineering . It conceptualizes elicitation not as a passive gathering activity but as a dynamic process of discovery, emergence, and invention of requirements. The authors emphasize that elicitation is inherently complex, communication-intensive, and multidisciplinary, drawing heavily from social sciences, organizational theory, and practical experience. They outline the fundamental activities involved in elicitation—such as understanding the domain, identifying sources, analyzing stakeholders, selecting appropriate techniques, and conducting the elicitation itself—and discuss the diverse roles played by the requirements engineer, including facilitator, mediator, and documenter. The chapter also highlights how contextual factors such as project type, organizational maturity, stakeholder distribution, and constraints significantly influence how elicitation is conducted.

The survey systematically reviews a wide range of elicitation techniques and approaches, including interviews, questionnaires, task and domain analysis, ethnography, group work, prototyping, goal-based methods, scenarios, and viewpoints. It compares these techniques in terms of their suitability for different elicitation activities and their complementary or alternative use. The chapter further discusses methodology-based approaches (e.g., structured analysis, object-oriented modeling, Agile methods), available tool support, and persistent issues such as communication barriers, stakeholder conflicts, volatility, and gaps between research and practice. Concluding with trends, challenges, and future research directions, the authors argue that despite advances in tools and formal methods, requirements elicitation remains as much an art as a science, requiring skill, adaptability, and contextual judgment.

10.1 Introduction

10.1.1 The Importance of Requirements Elicitation

Requirements engineering (RE) has long been recognized as a critical component of successful software development. Among its various phases, requirements elicitation represents an early yet continuous and foundational activity. It is during this phase that the needs, expectations, and constraints of stakeholders are uncovered and articulated. Importantly, requirements are not merely “gathered” or “captured”; rather, they are elicited—a term that reflects the discovery, emergence, and even invention of requirements through interaction and analysis. This distinction emphasizes that requirements do not always pre-exist in explicit form but must often be surfaced through structured engagement with stakeholders and examination of the problem domain.

The importance of getting the requirements right cannot be overstated. Poorly elicited requirements

are a major contributor to project failure, cost overruns, and system inadequacies. Field studies have identified effective requirements practices—particularly strong elicitation practices—as key success factors in software projects [1]. Despite this recognition, elicitation remains one of the most difficult and error-prone activities in software engineering.

10.1.2 The Complexity and Multidisciplinary Nature of Elicitation

Requirements elicitation is inherently complex due to its communication-intensive and multidisciplinary nature. Requirements are typically distributed across multiple sources: stakeholders, problem owners, existing documentation, legacy systems, organizational procedures, and domain knowledge. Because of this diversity, elicitation extends beyond traditional software engineering and draws heavily from disciplines such as social sciences, organizational theory, knowledge engineering, and group dynamics [2].

Communication challenges further compound this complexity. Stakeholders and analysts often belong to different "communities of practice," resulting in semantic gaps and misunderstandings. Concepts that are clear within one group may be opaque to another, and these differences frequently go unnoticed unless deliberately addressed. Consequently, elicitation is not simply a technical exercise but a socially embedded process that requires strong interpersonal, negotiation, and facilitation skills.

Moreover, the type of system under development significantly influences how elicitation is conducted. A custom embedded control system demands a different elicitation strategy compared to a commercial off-the-shelf inventory management system. Factors such as project scale, organizational maturity, stakeholder distribution, regulatory constraints, time and budget limitations, and safety criticality all shape the selection of techniques and the structure of the elicitation process. Thus, elicitation is deeply contextual and cannot be reduced to a one-size-fits-all procedure.

10.1.3 Communication Barriers and Organizational Context

One of the central challenges in elicitation is the existence of communication barriers between stakeholders and development teams. Differences in terminology, expectations, assumptions, and priorities often lead to incomplete or conflicting requirements. Organizational and political contexts also influence the elicitation process, affecting stakeholder participation, commitment, and openness. Because elicitation is iterative and negotiation-driven, it demands sustained cooperation among diverse stakeholder groups.

Additionally, elicitation is rarely performed in isolation. It interacts with other RE activities such as prioritization and negotiation, particularly in market-driven environments where requirements influx can be substantial and continuous [3]. The dynamic and evolving nature of stakeholder understanding means that requirements often change as elicitation progresses, reinforcing the iterative character of the process.

10.1.4 Objectives and Structure of the Chapter

Given the interdisciplinary breadth and practical importance of requirements elicitation, the chapter aims to present a comprehensive survey of the techniques, approaches, and tools used in this domain. The authors seek not only to catalogue available methods but also to analyze their strengths and weaknesses, examine contextual applicability, and highlight current issues, trends, and challenges faced by both researchers and practitioners.

The chapter is structured to first define and contextualize requirements elicitation, then survey widely used techniques and approaches, compare their applicability, explore methodology - based and tool - supported elicitation, and finally discuss persistent issues, emerging trends, and future research directions. In doing so, it positions requirements elicitation as both a critical engineering activity and a socially embedded, skill-intensive practice that continues to evolve.

10.2 What is Requirements Elicitation

Requirements Engineering (RE) is the discipline concerned with identifying, analyzing, documenting, validating, and managing the requirements of a software system. It seeks to ensure that a system fulfills stakeholder needs and achieves its intended objectives within given constraints. RE encompasses

multiple interrelated activities such as elicitation, analysis, specification, validation, negotiation, and management.

At its core, requirements elicitation within RE is about learning and understanding stakeholder needs rather than merely recording stated wishes. Robertson and Robertson describe this process metaphorically as "trawling for requirements," emphasizing that stakeholders often reveal more requirements than initially expected [3]. This highlights the exploratory nature of elicitation: it is not simply a passive collection process but an active discovery process. In some contexts, elicitation even involves creativity—requirements may be invented or refined as stakeholders reflect on new possibilities and constraints.

Moreover, elicitation is deeply influenced by communication dynamics. Differences in terminology, background knowledge, and expectations between analysts and stakeholders can create semantic gaps. Taylor-Cummings identifies the persistent "user-IS gap" that often complicates meaningful dialogue between problem owners and solution developers [4]. Thus, requirements elicitation must be understood not only as a technical activity but also as a socio-organizational process grounded in communication, negotiation, and mutual understanding.

10.2.1 The Process of Requirements Elicitation

The requirements elicitation process consists of multiple interrelated activities. Although the sequence and emphasis vary by project context, the following core activities are generally involved:

1. **Understanding the application domain**: Investigating the real-world environment in which the system will operate, including organizational, social, political, and technical constraints. Analysts must understand existing processes, goals, and problems before proposing system solutions [5].
2. **Identifying sources of requirements**: Determining where requirements reside, such as stakeholders (users, customers, sponsors), existing systems, business processes, documentation, regulations, and legacy artifacts [6].
3. **Analyzing stakeholders**: Identifying all individuals and groups affected by the system and understanding their interests, priorities, and influence. This includes selecting key representatives and recognizing potential conflicts among stakeholder groups [7].
4. **Selecting techniques and approaches**: Choosing appropriate elicitation techniques (e.g., interviews, workshops, observation, prototyping) based on project characteristics. Research shows that technique selection is often influenced by analyst familiarity, methodology constraints, or contextual judgment [8].
5. **Eliciting and refining requirements**: Engaging stakeholders using selected techniques to uncover functional and non-functional requirements, clarify scope, identify constraints, and iteratively refine understanding.

This process is inherently iterative and incremental. It typically begins with an informal mission statement and evolves through successive refinement. Time, cost, organizational maturity, and volatility significantly influence how thoroughly each activity is performed. As noted in practice-oriented guidance literature, elicitation rarely concludes with perfect completeness; instead, it is often bounded by project constraints [9].

10.2.2 Roles of the Requirements Engineer During Elicitation

The requirements engineer (also referred to as systems analyst or business analyst) plays multiple dynamic roles during elicitation. These roles extend beyond technical modeling and require strong interpersonal and organizational skills.

Facilitator The analyst acts as a facilitator. During interviews and workshops, they guide discussions, ask probing questions, clarify ambiguities, and ensure that all participants contribute effectively. Effective facilitation requires structured questioning techniques and domain exploration skills [2].

Mediator The analyst often serves as a mediator and negotiator. Conflicts between stakeholders - particularly regarding priorities, constraints, and competing interests—are common. Negotiation and prioritization therefore become central aspects of the elicitation phase [10]. The analyst must balance technical feasibility with business value while managing political and organizational sensitivities.

Documenter The analyst acts as a documenter and translator. Requirements elicited from stakeholders must be represented clearly and consistently for development teams. This documentation frequently forms the basis for contractual agreements and downstream design decisions. The analyst translates informal, sometimes ambiguous stakeholder statements into structured and verifiable specifications.

Finally, the requirements engineer must often assume the perspectives of other future stakeholders—such as architects, designers, testers, and maintainers—who may not yet be formally involved. This anticipatory role ensures that elicited requirements are realistic, consistent, and aligned with broader system objectives.

Taken together, these roles illustrate that requirements elicitation is as much a human-centered discipline as it is a technical one. The effectiveness of the process depends heavily on the analyst's ability to integrate communication, negotiation, documentation, and contextual awareness into a coherent and adaptive practice.

10.3 Techniques and Approaches for RE

The paper provides a comprehensive survey of techniques and approaches for requirements elicitation. The following section summarizes the key parts of those techniques and approaches.

Interviews Interviews are one of the most traditional elicitation techniques and may be structured, semi-structured, or unstructured. They allow analysts to gather detailed information directly from stakeholders, with effectiveness largely depending on interviewer skill and preparation. Structured interviews rely on predefined questions, while unstructured interviews support exploratory dialogue [11, 12].

Questionnaires Questionnaires are typically used in early elicitation stages to collect information from a large number of stakeholders efficiently. They may contain open or closed questions but are limited in depth and lack opportunities for clarification. They are most effective when the domain is already well understood [13].

Task Analysis Task analysis decomposes high-level user tasks into subtasks to understand workflows, user-system interaction, and required knowledge. It is especially useful for usability-oriented requirements and process modeling but can be effort-intensive depending on required granularity [14, 15].

Domain Analysis Domain analysis involves studying existing systems, documentation, and related applications to understand reusable concepts and domain constraints. It is particularly important when replacing or enhancing legacy systems and supports the reuse of specifications and domain knowledge [16, 17].

Introspection Introspection involves the analyst deriving requirements based on their own domain knowledge and assumptions. It is generally used as a starting point or when stakeholders have limited ability to articulate needs, though it risks bias and incompleteness [2].

Repertory Grids Repertory grids model domain entities and their attributes in matrix form to identify similarities and distinctions. This structured abstraction technique is typically used with domain experts and supports conceptual modeling but is limited in representing detailed system behaviors [18].

Card Sorting Card sorting asks stakeholders to group domain concepts into categories based on their understanding. It helps reveal conceptual structures and relationships but operates at a relatively high level of abstraction [19].

Laddering Laddering uses iterative "why" and "how" probing questions to organize stakeholder knowledge hierarchically. It supports structured reasoning and requirement clarification but assumes that knowledge can be expressed in hierarchical form [20].

Group Work Group sessions facilitate collaborative requirement discovery among multiple stakeholders. They promote shared understanding and negotiation but require strong facilitation to manage conflicts and dominant personalities [21].

Brainstorming Brainstorming encourages rapid generation of ideas without immediate critique. It is effective for exploring innovative requirements and developing initial project scope but is not intended for detailed specification or decision-making [22].

Joint Application Development (JAD) JAD involves structured workshops with stakeholders and facilitators to rapidly define requirements and resolve issues. It differs from brainstorming by following defined steps and focusing on business objectives [23].

Requirements Workshops Requirements workshops are structured collaborative sessions aimed at refining and discovering requirements. Variants include cross-functional workshops, CRC-based sessions, and creativity-driven meetings [21, 24].

Ethnography Ethnography studies stakeholders in their natural work environments to understand contextual, social, and collaborative aspects of system use. It is particularly effective for uncovering implicit requirements and usability concerns [25, 26].

Observation Observation involves directly watching users perform tasks without interference. It provides insight into actual work practices but may alter user behavior due to awareness of being observed [26].

Protocol Analysis Protocol analysis requires stakeholders to verbalize their thought processes while performing tasks. It reveals cognitive reasoning behind actions but may omit habitual steps that users consider trivial [27].

Apprenticing Apprenticing requires the analyst to learn and perform stakeholder tasks under supervision. This immersive approach improves domain understanding and uncovers tacit knowledge [28].

Prototyping Prototyping presents early system models to stakeholders for feedback and refinement. It is particularly useful for interface design and innovative systems but may create premature attachment to incomplete designs [26].

Goal-Based Approaches Goal-oriented approaches decompose high-level system objectives into refined sub-goals that eventually translate into requirements. Frameworks such as KAOS and i* formalize this reasoning, though incorrect early goals can propagate errors [29, 30].

Scenarios Scenarios describe concrete narratives of system use, including interactions and exceptions. They support validation and incremental refinement and are often used in structured scenario-based methods [31, 32].

Viewpoints Viewpoint approaches model requirements from multiple stakeholder or system perspectives to improve completeness and consistency. They are useful in complex systems but can be resource-intensive [33, 34].

	Interviews	Domain	Groupwork	Ethnography	Prototyping	Goals	Scenarios	Viewpoints
Understanding the domain	X	X	X	X		X	X	X
Identifying sources of requirements	X	X	X			X	X	X
Analyzing the stakeholders	X	X	X	X	X	X	X	X
Selecting techniques	X	X	X					
Eliciting the requirements	X	X	X	X	X	X	X	X

Table 10.1: Key Features of Techniques

10.4 Comparison of Techniques and Approaches

10.4.1 Selecting Techniques for Specific Elicitation Activities

A fundamental concern in requirements elicitation is determining which techniques are most appropriate for particular elicitation activities. There is no universally optimal method; technique selection is inherently context-dependent and shaped by project characteristics, stakeholder dynamics, and system complexity. Research in requirements engineering emphasizes that elicitation approaches must be adapted to the nature of the problem and environment rather than applied uniformly across projects [35]. Consequently, analysts must align techniques with specific goals such as domain understanding, stakeholder analysis, or detailed requirement discovery.

Generic and flexible techniques such as interviews, domain analysis, and group work are suitable across multiple elicitation activities (Table 10.1). Interviews facilitate direct knowledge extraction from stakeholders, domain analysis grounds requirements in contextual understanding, and group sessions promote negotiation and shared interpretation [21, 16]. These approaches are versatile because they address both informational and communicative aspects of elicitation.

More structured approaches - such as goal-based, scenario-based, and viewpoint-oriented techniques - offer systematic support for modeling and refinement. Goal-oriented methods connect detailed requirements to higher-level system objectives and help ensure alignment with business intent [29]. Scenario-based techniques support incremental refinement and validation by describing realistic interactions [31]. Viewpoint approaches strengthen completeness by capturing diverse stakeholder perspectives and organizing requirements accordingly [33]. Thus, technique suitability varies according to the type of knowledge being elicited and the stage of the process.

10.4.1.1 Complementary Use of Techniques

The comparison further emphasizes that elicitation is most effective when techniques are used in combination. Complementary techniques compensate for one another's weaknesses and provide richer insights than isolated application. For instance, combining ethnographic observation with prototyping allows analysts to observe real-world behavior while gathering structured feedback on proposed solutions [26]. Similarly, goal modeling can be strengthened by integrating scenario analysis to validate goal decomposition through concrete use cases [32].

Interviews often complement domain studies by clarifying findings derived from documentation analysis. Likewise, individual interviews may precede collaborative workshops to reconcile divergent stakeholder priorities [21]. The paper stresses that such combinations increase coverage across contextual, social, and technical dimensions of requirements. Rather than relying on a dominant technique, effective elicitation involves strategic orchestration of multiple methods.

10.4.1.2 Alternative Techniques and Flexibility

In addition to complementarity, some techniques may serve as alternatives when constraints prevent the use of preferred methods. Environmental limitations, political sensitivity, resource constraints, or

stakeholder availability may necessitate substituting one approach for another. For example, when ethnographic observation is impractical, scenario-based methods may approximate contextual understanding [31]. If large-scale workshops are infeasible, structured interviews may serve as substitutes.

However, alternative techniques rarely yield identical outcomes. Each method introduces different forms of abstraction, bias, and depth of analysis. Substitution may therefore affect the type and granularity of requirements elicited. Empirical studies indicate that analysts often select techniques based on familiarity or intuition rather than systematic reasoning [8]. This reinforces the importance of informed and reflective technique selection.

10.4.1.3 Implications for Practice

The comparative discussion ultimately demonstrates that requirements elicitation is not the execution of a predefined recipe but the management of trade-offs between techniques. Analysts must evaluate project context, stakeholder diversity, system complexity, and resource constraints when selecting methods. The literature consistently highlights that structured guidance for technique selection remains limited, and greater empirical grounding is needed to support systematic decision-making [35].

In summary, the comparison underscores three central insights:

1. Technique suitability depends on elicitation activity and context.
2. Complementary combinations enhance requirement quality.
3. Alternative techniques provide flexibility but may alter outcomes.

10.4.2 Methodology-Based Requirements Elicitation

Structured Analysis and Design (SAD) Structured Analysis and Design (SAD) is a function-oriented methodology that supports elicitation through formal modeling techniques such as Data Flow Diagrams (DFDs), Entity-Relationship Diagrams (ERDs), Data Dictionaries, and Event Lists [36, 37]. These artifacts help structure stakeholder discussions and clarify system boundaries, data flows, and functional decomposition during early requirements analysis.

Object-Oriented Approaches and UML Object-oriented approaches, particularly UML, provide standardized modeling techniques such as Use Case diagrams and Class diagrams to support requirements elicitation [38]. Use Cases are especially influential, offering structured representations of system functionality from an actor's perspective. These models facilitate stakeholder validation while maintaining analytical structure.

Integrated and Exploratory Methodologies Some methodologies advocate combining techniques within a structured roadmap. For example, ethnographic study may precede structured interviews to ground elicitation in real-world context [2]. Soft Systems Methodology (SSM) extends this by focusing on organizational understanding and stakeholder worldviews before formalizing requirements [39]. These approaches emphasize contextual and socio-technical complexity.

Customer-Oriented and Quality-Driven Methods Quality Function Deployment (QFD) translates customer needs into technical specifications using structured matrices, ensuring traceability between stakeholder expectations and design decisions [40]. Similarly, Gause and Weinberg emphasize exploratory questioning techniques to uncover implicit assumptions and hidden constraints [41]. Both approaches strengthen analytical rigor in elicitation.

Agile Approaches Agile methods redefine elicitation as an ongoing, iterative process integrated with development [42]. Lightweight artifacts such as User Stories and continuously refined Product Backlogs support incremental discovery rather than heavy upfront specification.

10.5 Tool Support for Requirements Elicitation

Tool support for requirements elicitation ranges from lightweight templates to sophisticated modeling and collaboration systems. Basic tools such as standardized specification templates (e.g., IEEE Std 830 and Volere) provide structural guidance for documenting elicited requirements, while requirements management systems such as DOORS support organization and traceability [43, 3]. More specialized tools have been developed to assist particular elicitation approaches, including goal-modeling environments and scenario-based systems, though adoption in industry has been limited [44, 45]. Additionally, groupware and collaborative platforms facilitate distributed stakeholder engagement, supporting activities such as brainstorming, negotiation, and shared modeling [46]. Despite these advances, tool support remains uneven, with many elicitation activities still heavily dependent on analyst expertise and manual facilitation rather than automation.

10.6 Issues and Pitfalls

Process and Project-Related Issues Requirements elicitation is highly context-sensitive. Each project differs in scope, domain complexity, stakeholder distribution, and organizational maturity, making standardized processes difficult to apply uniformly. A common problem is poorly defined project scope at the outset, which leads to assumptions, misunderstandings, and requirement creep. External influences such as organizational change, economic pressures, or regulatory constraints further complicate the process. Because elicitation is iterative and socially driven, maintaining clarity of scope and alignment with business goals is an ongoing challenge.

Communication and Understanding Problems Communication breakdowns are among the most significant obstacles in elicitation. Stakeholders often struggle to articulate their needs clearly, especially when they lack technical vocabulary or awareness of possible system capabilities. Analysts, in turn, may misunderstand domain-specific language or implicit assumptions. Stakeholders may describe solutions rather than underlying problems, limiting exploration of alternatives. Additionally, routine practices are frequently overlooked because they are considered obvious, even though they may not be apparent to others. These communication gaps contribute directly to incomplete, ambiguous, or inconsistent requirements.

Quality of Requirements Elicited requirements frequently suffer from vagueness, inconsistency, infeasibility, or uneven levels of detail. The informal nature of many elicitation activities increases the risk of ambiguity and omission. Requirements are also prone to volatility: as stakeholders gain better understanding of the system and its implications, their expectations evolve. This continuous change can destabilize the specification and complicate validation. Moreover, the lack of clear metrics for assessing elicitation effectiveness makes it difficult to determine whether requirements are sufficiently complete and correct.

Stakeholder-Related Challenges Conflicts between stakeholders are common, particularly when priorities and trade-offs must be negotiated. Some stakeholders may be unwilling to compromise, unclear about their own needs, or resistant to changes introduced by a new system. Differences in authority, influence, and organizational interests can distort elicitation outcomes. Shifting stakeholder opinions and competing agendas further increase the difficulty of achieving stable and agreed-upon requirements.

Analyst-Related Limitations The success of elicitation depends heavily on analyst expertise. Analysts may lack sufficient domain knowledge, methodological training, or interpersonal skills such as facilitation and negotiation. Many rely on familiar techniques rather than systematically selecting methods appropriate to the context. A premature focus on solutions instead of problem exploration can also bias the elicitation process. Without structured guidance and experience, analysts may repeat common mistakes.

Research Gaps From a research perspective, the paper notes a significant gap between theory and practice. Although numerous elicitation techniques have been proposed, many lack strong empirical

validation or practical usability. There is limited availability of comprehensive process guidelines and integrated tool support, particularly for technique selection and contextual adaptation. The literature also lacks standardized metrics to measure elicitation performance and effectiveness. Without empirical case studies and industry reports, it remains difficult to evaluate the true impact of proposed methods.

Practice-Oriented Issues Despite extensive research on elicitation techniques, gaps remain between theory and practice. Many proposed methods lack strong empirical validation or practical adoption. Tool support and structured guidance for technique selection are often limited. In practice, organizations may not allocate adequate time and resources to elicitation, and customers may underestimate its importance. Differences between expert and novice analysts further contribute to variability in outcomes. As a result, recurring elicitation problems persist across projects.

10.7 Trends and Challenges in RE Research and Practice

10.7.1 Trends in Research

Research in requirements elicitation has evolved from adapting informal knowledge-acquisition techniques to developing structured, model-driven approaches such as goal-based, scenario-based, and viewpoint-oriented methods. More recently, attention has shifted toward providing tool support for these approaches and adapting elicitation practices to emerging paradigms such as agile development, distributed collaboration, web systems, and agent-oriented architectures. The overall trend reflects a move toward formalization, contextual adaptability, and integration of elicitation within broader software engineering frameworks.

10.7.2 Trends in Practice

In practice, requirements engineering is increasingly recognized as critical to project success, particularly in mature organizations. While traditional techniques such as interviews and group meetings remain dominant, structured approaches like Use Cases and UML modeling have gained widespread acceptance. Agile methodologies have further transformed practice by treating elicitation as an iterative, continuous activity supported by lightweight artifacts such as user stories and evolving backlogs. However, adoption of advanced or research-driven techniques remains uneven across organizations.

10.7.3 Challenges in Research

A key challenge in research is bridging the gap between theory and industrial practice. Many proposed elicitation techniques are complex and lack strong empirical validation, limiting adoption. The absence of standardized metrics for evaluating elicitation effectiveness further complicates comparative assessment. Researchers must focus on simplifying methodologies, providing stronger empirical evidence, and developing practical guidelines and educational frameworks that reduce reliance on expert intuition.

10.7.4 Challenges in Practice

In industry, elicitation is often constrained by limited time, resources, and stakeholder engagement. Organizations may underestimate its importance, leading to rushed processes and recurring quality issues. Stakeholder conflicts, organizational resistance to change, and variability in analyst expertise further complicate implementation. Strengthening analyst training, improving collaboration between research and practice, and fostering greater organizational commitment to structured elicitation remain persistent practical challenges.

References

[1] H. F. Hofmann and F. Lehner. “Requirements Engineering as a Success Factor in Software Projects”. In: *IEEE Software* 18.4 (2001), pp. 58–66.

[2] Joseph A. Goguen and Charlotte Linde. “Techniques for Requirements Elicitation”. In: *Proceedings of the IEEE International Symposium on Requirements Engineering*. 1993, pp. 152–164.

[3] Suzanne Robertson and James Robertson. *Mastering the Requirements Process*. Addison Wesley, 1999.
[4] A. Taylor-Cummings. "Bridging the user-IS gap: a study of major systems projects". In: *Journal of Information Technology* 13.1 (1998), pp. 29–54.
[5] Michael Jackson. "The World and the Machine". In: *Proceedings of the 17th International Conference on Software Engineering*. 1995, pp. 283–292.
[6] Pericles Loucopoulos and Vassilios Karakostas. *System Requirements Engineering*. McGraw-Hill, 1995.
[7] Ian F. Alexander and Richard Stevens. *Writing Better Requirements*. Addison Wesley, 2002.
[8] Ann M. Hickey and Alan M. Davis. "Elicitation Technique Selection: How Do Experts Do It?" In: *Proceedings of the 11th IEEE International Requirements Engineering Conference*. 2003, pp. 169–178.
[9] Ian Sommerville and Pete Sawyer. *Requirements Engineering: A Good Practice Guide*. John Wiley & Sons, 1997.
[10] Alan M. Davis. *Software Requirements: Analysis and Specification*. Prentice Hall, 1994.
[11] R. Agarwal and M. R. Tanniru. "Knowledge Acquisition Using Structured Interviewing". In: *Journal of Management Information Systems* 7.1 (1990), pp. 123–140.
[12] Karen Holtzblatt and Hugh R. Beyer. "Requirements Gathering: The Human Factor". In: *Communications of the ACM* 38.5 (1995), pp. 30–32.
[13] William Foddy. *Constructing Questions for Interviews and Questionnaires*. Cambridge University Press, 1994.
[14] P. Carlshamre and J. Karlsson. "A usability-oriented approach to requirements engineering". In: (1996).
[15] J. Richardson, T. Ormerod, and A. Shepherd. "The Role of Task Analysis in Capturing Requirements". In: *Interacting with Computers* (1998).
[16] A. Sutcliffe and N. Maiden. "The Domain Theory for Requirements Engineering". In: *IEEE TSE* (1998).
[17] Michael Jackson. *Problem Frames*. Addison Wesley, 2000.
[18] George Kelly. *The Psychology of Personal Constructs*. Norton, 1955.
[19] Kent Beck and Ward Cunningham. "A Laboratory for Teaching Object-Oriented Thinking". In: (1989).
[20] David Hinkle. "The change of personal constructs". In: (1965).
[21] Ellen Gottesdiener. *Requirements by Collaboration*. Addison Wesley, 2002.
[22] Alex Osborn. *Applied Imagination*. Scribner, 1979.
[23] J. Wood and D. Silver. *Joint Application Development*. Wiley, 1995.
[24] Linda Macaulay. "Requirements as a Cooperative Activity". In: (1993).
[25] L. Ball and T. Ormerod. "Putting ethnography to work". In: *IJHCS* (2000).
[26] Ian Sommerville. *Software Engineering*. Addison Wesley, 2001.
[27] J. Nielsen, T. Clemmensen, and C. Yssing. "Think-aloud technique". In: (2002).
[28] Hugh Beyer and Karen Holtzblatt. "Apprenticing with the Customer". In: *Communications of the ACM* (1995).
[29] A. Dardenne, A. van Lamsweerde, and S. Fickas. "Goal-Directed Requirements Acquisition". In: *Science of Computer Programming* (1993).
[30] Eric Yu. "Modeling and Reasoning Support for Early-Phase RE". In: (1997).
[31] Colin Potts, K. Takahashi, and A. Anton. "Inquiry-Based Requirements Analysis". In: *IEEE Software* (1994).
[32] C. Rolland, C. Souveyet, and C. Ben Achour. "Guiding Goal Modeling Using Scenarios". In: *IEEE TSE* (1998).
[33] Ian Sommerville, Pete Sawyer, and Stephen Viller. "Viewpoints for Requirements Elicitation". In: (1998).
[34] Bashar Nuseibeh, Anthony Finkelstein, and Jeff Kramer. "Method Engineering for Multi-Perspective Development". In: *Information and Software Technology* (1996).
[35] Bashar Nuseibeh and Steve Easterbrook. "Requirements Engineering: A Roadmap". In: *Proceedings of the Conference on The Future of Software Engineering*. ACM, 2000, pp. 35–46.
[36] Tom DeMarco and P. J. Plauger. *Structured Analysis and System Specification*. Prentice Hall, 1979.
[37] Edward Yourdon. *Modern Structured Analysis*. Prentice Hall, 1989.

[38] Alistair Cockburn. *Writing Effective Use Cases*. Addison Wesley, 2001.
[39] Peter Checkland and Jim Scholes. *Soft Systems Methodology in Action*. John Wiley & Sons, 1990.
[40] Yoji Akao. *Quality Function Deployment: Integrating Customer Requirements into Product Design*. Productivity Press, 1995.
[41] Donald C. Gause and Gerald M. Weinberg. *Exploring Requirements: Quality Before Design*. Dorset House, 1989.
[42] Robert C. Martin. *Agile Software Development: Principles, Patterns, and Practices*. Prentice Hall, 2003.
[43] IEEE. *IEEE Std 830-1998: Recommended Practice for Software Requirements Specifications*. 1998.
[44] Howard Reubenstein and Richard Waters. "The Requirements Apprentice: Automated Assistance for Requirements Acquisition". In: *IEEE Transactions on Software Engineering* 17.3 (1991), pp. 226–240.
[45] L. Goldin and D. M. Berry. "AbstFinder: A Prototype Natural Language Text Abstraction Finder for Use in Requirements Elicitation". In: *Automated Software Engineering* 4.4 (1994), pp. 375–412.
[46] Daniela Herlea and Saul Greenberg. "Using a Groupware Space for Distributed Requirements Engineering". In: *Proceedings of the Seventh Workshop on Enabling Technologies: Infrastructure for Collaborative Enterprises*. 1998, pp. 57–62.

Chapter 11

[Skim] Hey, ChatGPT, Look at My Work: Using Conversational AI in Requirements Engineering Education

This paper presents an empirical study on integrating conversational AI (specifically ChatGPT) into requirements engineering (RE) education within an undergraduate software engineering course. The authors designed an experiment involving 20 student groups who developed software projects and used ChatGPT during the requirements specification phase under two different processes: one where students first created requirements manually before refining them with ChatGPT (Process A), and another where students directly used ChatGPT from the start (Process B). The study evaluates the quality of requirements artifacts, prompting strategies, and student effort. Results show that both approaches yield comparable outcomes, with average artifact quality around 80%, but achieving higher-quality results requires significant effort in prompt design, critical evaluation, and refinement. The study also highlights a strong correlation between prompt quality and output quality, emphasizing the importance of structured, detailed, and context-rich prompts.

Beyond performance, the paper explores how students interact with conversational AI and their perceptions of its usefulness. Students primarily used ChatGPT for idea generation, refinement, and rewriting, with "concretization" (refining existing ideas) being the most effective use case. However, the study identifies several challenges: students often lacked prompting proficiency, were sometimes overly influenced by ChatGPT's confident responses, and frequently accepted generic or incorrect outputs without sufficient critical analysis. The findings suggest that while conversational AI can enhance productivity and support learning, its effective use requires training in prompt engineering and critical thinking. Overall, the paper argues for a shift in software engineering education toward teaching students how to collaborate with AI tools effectively rather than treating them as simple automation aids.

11.1 Motivation

The introduction motivates the study by highlighting the rapid and disruptive emergence of conversational AI tools such as ChatGPT and GitHub Copilot, which have significantly reshaped computing education and software engineering practices. These tools introduce both opportunities and challenges for instructors and students, requiring a rethinking of traditional pedagogical approaches [1, 2, 3, 4, 5, 6]. The authors emphasize that resisting this shift is impractical, echoing prior arguments that adaptation to AI-assisted workflows is inevitable [6]. Consequently, there is a pressing need to understand how such tools can be systematically integrated into education while preserving learning objectives and fostering critical thinking.

Another key motivation stems from the gap between the availability of powerful AI tools and the lack of structured guidance on how to use them effectively in software engineering tasks, particularly in requirements engineering. While prior work has explored AI-assisted coding and educational applications, fewer studies examine how conversational AI can support early-stage development activities such as requirements elicitation and specification. This is especially important because requirements engineering is a cognitively demanding and collaborative process that benefits from creativity, precision, and iterative refinement. The authors thus position their work as an effort to explore how AI can augment, rather than replace, human reasoning in these tasks, aligning with broader calls to adapt teaching practices to the AI era [3, 4].

Finally, the study is motivated by the need to evaluate not only the effectiveness of AI tools but also their impact on student behavior, learning, and decision-making. Prior research suggests that AI tools can influence how students approach problem-solving, sometimes leading to over-reliance or reduced critical engagement [5, 6]. Therefore, the authors aim to investigate how students interact with conversational AI in a controlled educational setting, what strategies they adopt, and how these interactions affect the quality of their outputs. This motivation reflects a broader educational goal: preparing students to collaborate effectively with AI systems by developing skills in prompt engineering, evaluation, and critical thinking.

11.2 Methodology

11.2.1 Course Setup

The study was conducted in the context of an upper-level undergraduate software engineering course where students worked in 20 groups of four to develop large-scale projects involving an Android client and a Node.js backend. The course followed an iterative development process, requiring multiple deliverables such as requirements, design, implementation, testing, and documentation. A key pedagogical objective was to systematically integrate conversational AI while fostering critical thinking, allowing students to use ChatGPT in a controlled manner rather than as an unrestricted tool.

To evaluate AI-assisted requirements engineering, the instructors designed two structured workflows: Process A, where students first created requirements manually and then refined them using ChatGPT, and Process B, where students used ChatGPT from the beginning. Students were required to submit final artifacts, intermediate drafts, and complete ChatGPT interaction logs, enabling detailed analysis of both outcomes and processes. This setup ensured that the study could compare human-first vs. AI-first workflows while capturing how students iteratively engaged with the tool.

11.2.2 RQ1: Effectiveness of ChatGPT Interaction Processes

This research question investigates how different interaction modes (Process A vs. Process B) affect the quality of requirements artifacts. The methodology involved evaluating student submissions using predefined grading criteria to ensure consistency across groups. The study compares outputs generated through manual-first refinement versus direct AI-driven generation, enabling a structured assessment of whether early human reasoning leads to better outcomes than immediate AI reliance.

The evaluation focuses on determining whether either process leads to significantly better results or whether both approaches converge to similar quality levels. A key aspect of this analysis is understanding the trade-off between human effort and AI assistance, particularly whether AI-first approaches reduce effort or simply shift it toward prompt engineering and refinement. The study thus examines not only final outputs but also the process efficiency and effectiveness of each interaction mode.

11.2.3 RQ2: Prompting Strategies and Their Impact

This research question explores how students construct prompts and how these strategies influence the quality of ChatGPT-generated outputs. The methodology includes analyzing the full conversation logs submitted by students to identify patterns in prompting behavior, such as level of detail, structure, ambiguity, and use of examples. The authors particularly examine how different prompting styles correlate with improvements or deficiencies in the resulting requirements.

A central finding investigated here is the relationship between prompt quality and output quality, with emphasis on how vague prompts (e.g., "improve this") lead to misaligned or generic responses, while structured prompts produce more relevant results. The study also evaluates students' prompting proficiency, identifying common shortcomings such as lack of context, insufficient constraints, and unclear expectations. This analysis highlights the importance of prompt engineering as a learned skill in AI-assisted development workflows.

11.2.4 RQ3: Student Effort and Interaction Behavior

This research question examines how students interact with ChatGPT and the effort required to achieve high-quality results. The methodology involves analyzing both qualitative and quantitative data from interaction logs to understand how students allocate time across activities such as brainstorming, prompting, and validating outputs. The study pays particular attention to how effort differs across interaction modes and how it influences final artifact quality.

A key aspect of this analysis is identifying behavioral patterns, including students being overly influenced by ChatGPT's confident responses and sometimes accepting incorrect suggestions without sufficient critique. The study also evaluates how much effort is needed to move beyond mid-range quality outputs (80%), emphasizing that higher-quality results require significant investment in refinement and critical assessment. This highlights the role of human oversight and critical thinking in effective human-AI collaboration.

11.3 Key Findings

11.3.1 RQ1: Process A vs. B

The results show that students achieved comparable quality across all processes, with average scores of 82% (A^+), 75% (A^-), and 77% (B), and no statistically significant difference between them . Interestingly, half of the top-performing projects came from each process, indicating that neither human-first nor AI-first workflows dominate in terms of final outcomes. However, a key behavioral difference emerges: students in Process A (especially A^+) were more likely to edit ChatGPT outputs, suggesting stronger engagement and critical refinement compared to Process B.

Another important observation is that many students in Process A (7/20 groups) did not actually provide their initial human-written specifications to ChatGPT, instead using it only for idea generation. This effectively made their workflow closer to Process B, highlighting inconsistencies in how students applied the intended methodology. Overall, while both processes can yield similar results, Process A encourages more active editing and critical interaction, whereas Process B tends to involve more direct reliance on AI-generated content.

11.3.2 RQ2: Effort

The study finds that achieving high-quality outputs with ChatGPT requires substantial effort beyond initial generation, particularly in prompt design, iterative refinement, and validation of outputs . While students can relatively easily reach a mid-level quality (80%), improving beyond this threshold demands significant investment in brainstorming, crafting detailed prompts, and critically assessing AI responses. This highlights that AI does not eliminate effort but rather shifts it toward more cognitive and interaction-based tasks.

Additionally, the analysis shows that prompt quality is strongly tied to the effort invested, with better prompts requiring more time, more detailed instructions, and clearer structure. The expert investigator experiment reinforces this finding, demonstrating that high-quality outputs can be consistently achieved with well-structured prompts, but only when sufficient effort is spent on designing them. Thus, the key takeaway is that effective use of ChatGPT is effort-intensive and skill-dependent, rather than a shortcut to high-quality results.

11.3.3 RQ3: Usage Patterns

Students primarily used ChatGPT for ideation, refinement, and formalization of requirements, with refinement and rewriting being the most commonly adopted uses . This suggests that ChatGPT is most effective when students already have partially developed ideas, allowing the tool to enhance clarity and structure rather than generate content from scratch. Different students exhibited different preferences, with some favoring initial human brainstorming and others preferring direct AI interaction, indicating that usage patterns vary based on individual working styles.

At the same time, the study reveals notable challenges in usage behavior. Students often showed low prompting proficiency, failing to provide sufficient context, structure, or clarity in their queries. Moreover, many students were overly influenced by ChatGPT's confident responses, sometimes accepting incorrect or suboptimal suggestions without sufficient critique. This highlights a critical concern: while ChatGPT is perceived as useful, effective usage requires strong critical thinking and evaluation skills, as well as awareness of the risks of over-reliance.

11.4 Implications

The implications of this work suggest that conversational AI can be effectively integrated into software engineering education, but only when accompanied by appropriate guidance and pedagogical adjustments. The findings indicate that AI tools should not be treated as simple automation aids but rather as collaborative partners that require active human involvement. Teaching students how to construct effective prompts, critically evaluate AI outputs, and iteratively refine results becomes essential. This implies a shift in curriculum design toward emphasizing prompt engineering, critical thinking, and human–AI collaboration skills.

Another key implication is that different interaction modes with AI may suit different learners, and educational environments should accommodate this diversity rather than enforcing a single workflow. While AI can support productivity and idea generation, over-reliance on it without critical assessment can negatively impact learning outcomes. Therefore, educators should focus on developing students' ability to question and validate AI-generated content. More broadly, the study highlights the need for a balanced approach where AI augments human reasoning without diminishing core problem-solving and analytical skills.

11.5 Threats to Validity

Threats to validity arise primarily from the study's educational context and experimental design. The participants were undergraduate students in a single course at one institution, which may limit the generalizability of the findings to other populations such as professional developers or students with different backgrounds. Additionally, the study focuses specifically on requirements engineering tasks within a project-based course, and the results may not directly transfer to other software engineering activities. The controlled use of ChatGPT, including restricting students to a specific version and requiring submission of interaction logs, may also influence how students naturally interact with AI tools outside an academic setting.

Another important threat concerns the evaluation and measurement process. Although grading rubrics were carefully designed and cross-validated, assessing requirements quality and prompt effectiveness inherently involves some level of subjectivity. Variability in ChatGPT's responses across different runs could also affect reproducibility, even though efforts were made to mitigate this through repeated experiments. Furthermore, students' self-reported reflections and effort estimates may not always accurately capture their actual behavior, introducing potential bias in understanding interaction patterns and effort distribution.

References

[1] OpenAI. *ChatGPT: Optimizing Language Models for Dialogue*. https://openai.com/blog/chatgpt. 2023.

[2] GitHub. *GitHub Copilot: Your AI Pair Programmer*. https://github.com/features/copilot. 2021.
[3] Paul Denny et al. "Teaching Prompt Engineering in Computing Education". In: *Proceedings of the ACM Conference on International Computing Education Research*. 2023.
[4] Brett A. Becker et al. "AI in Computing Education: Opportunities and Challenges". In: *Computer Science Education* (2023).
[5] James Prather et al. "Student Use of AI-Based Tools in Programming Education". In: *ACM Transactions on Computing Education* (2023).
[6] Enkelejda Kasneci et al. "ChatGPT for Good? On Opportunities and Challenges of Large Language Models for Education". In: *Learning and Individual Differences* (2023).

Chapter 12

[Required] Advantages and Disadvantages of a Monolithic Repository

This paper investigates the advantages and disadvantages of monolithic source code repositories compared to multi-repository systems through a case study conducted at Google. Using a mixed-methods approach that combines a large-scale survey of software engineers with analysis of developer tool logs, the study explores how repository structure influences developer experience, productivity, and code management. The results show that engineers generally prefer the monolithic repository model due to its high visibility across the entire codebase, which enables easier discovery of reusable APIs, access to example implementations, and streamlined dependency management. The ability to update dependencies and propagate API changes across the codebase further supports development efficiency.

However, the study also identifies important trade-offs between monolithic and multi-repository systems. While monolithic repositories provide centralized tooling, consistent development practices, and easier code reuse, multi-repository environments offer greater flexibility in selecting toolchains and managing stable, versioned dependencies. Engineers also associate multi-repo systems with improved build performance and reduced complexity due to smaller codebases. Overall, the findings highlight that the choice between repository models involves balancing visibility, standardization, and dependency management against flexibility, stability, and development autonomy.

12.1 Motivation

Modern software organizations produce extremely large and complex codebases, often composed of numerous projects that depend on shared libraries and frameworks. As the number of interdependencies between components grows, managing these relationships becomes increasingly important for maintaining development velocity and software quality. Organizations must therefore carefully choose how to structure and manage their source code repositories. A key architectural decision is whether to maintain a monolithic repository, where all projects reside in a single shared codebase, or a multi-repository system, where each project is stored in its own repository. Large technology companies such as Facebook, Google, and Microsoft have adopted monolithic repository models at scale, yet there has been relatively little empirical research examining the trade-offs between these repository structures [1, 2, 3]. Understanding the implications of these different repository strategies is essential for organizations seeking to support large-scale collaborative software development.

Another important motivation for the study arises from the uncertainty surrounding how developers actually benefit from the characteristics of monolithic repositories. Proponents of monolithic repositories argue that broad visibility into the entire codebase allows developers to understand APIs more effectively, discover reusable components, and coordinate changes across dependent systems. At the same time,

critics suggest that such large codebases may overwhelm developers with irrelevant information and reduce flexibility in choosing tools and development practices. Prior research on developer workflows has shown that developers frequently rely on searching and exploring code to understand APIs and system behavior [4]. However, it remains unclear whether developers working in monolithic repositories genuinely exploit these capabilities in practice or whether the benefits are primarily perceived rather than realized. Investigating developer perceptions alongside actual usage behavior provides an opportunity to better understand the real impact of repository structure on development practices.

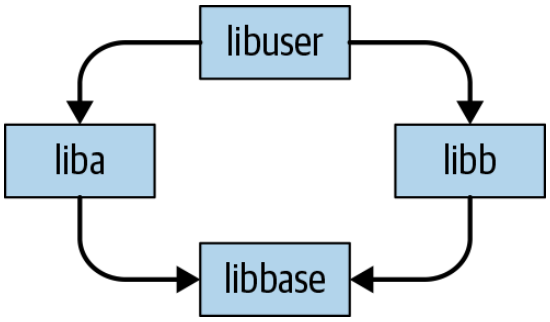


Figure 12.1: The diamond dependency problem in multi-repository systems, where two projects depend on different versions of the same library, leading to potential conflicts and coordination challenges.

The study is also motivated by long-standing challenges related to dependency management and collaboration in large software systems. In multi-repository environments, dependencies between projects are typically versioned and maintained independently, which can lead to problems such as version skew and the well-known "diamond dependency" (Figure 12.1) issue in dependency graphs [5]. Managing these dependencies often requires significant coordination between teams and careful communication about changes to shared components. Prior research has shown that developers frequently rely on communication channels to coordinate dependency updates and manage changes across components [6]. Monolithic repositories offer an alternative approach in which dependencies are centralized and updated simultaneously across projects, potentially eliminating certain coordination challenges. Exploring whether this centralized model improves development practices provides an important motivation for empirically studying the trade-offs between repository models.

12.1.1 Monolithic Repositories

A monolithic repository (often referred to as a monorepo) is a source code management model in which the code for many projects within an organization is stored in a single, centralized repository. In this model, engineers typically have broad visibility into the entire codebase and interact with a shared set of tools for building, testing, browsing, and reviewing code. Large technology organizations have adopted this model to support large-scale development across multiple teams and products, demonstrating that a single repository can successfully manage extremely large codebases and thousands of developers [3].

Monolithic repositories are characterized by several key features:

1. **Centralization** All source code for multiple projects is stored within a single shared repository rather than being distributed across multiple repositories.
2. **Visibility** Engineers across the organization can search, browse, and read code from any project in the repository, enabling easier discovery of APIs and reusable components.
3. **Synchronization** Development typically follows a trunk-based workflow, where developers commit changes directly to the main branch of the repository.
4. **Completeness** Projects rely on dependencies that are also stored within the same repository, meaning builds can be performed using code contained entirely within the repository without external versioned packages.

5. **Standardization** Developers use a shared and consistent set of tools for activities such as building, testing, code browsing, and code review, ensuring uniform development practices across the organization.

In contrast to monolithic repositories, multi-repository systems store projects in separate repositories, often leading to versioned dependencies and independent development workflows. While such systems provide greater flexibility in choosing tools and development practices, they can introduce challenges such as version skew and complex dependency relationships between projects. Monolithic repositories attempt to address these challenges by consolidating dependencies and development workflows within a single environment, enabling consistent tooling and easier coordination across teams.

12.2 Methodology

To investigate the advantages and disadvantages of monolithic repositories compared with multi repository systems, the study employs a mixed-methods case study within a large organization using a monolithic repository. The methodology combines two primary approaches. First, the researchers conducted a large-scale survey of software engineers to understand their perceptions, preferences, and experiences working with both monolithic and multi-repo environments. Second, they performed a quantitative analysis of developer tool logs to observe how engineers actually interact with the codebase, such as how frequently they view or modify code outside their immediate project. By combining perception-based survey data with behavioral log analysis, the study aims to validate whether the reported benefits of monolithic repositories are reflected in real developer activity.

12.2.1 Assumptions

The methodology is built upon several assumptions derived from prior internal observations about developer workflows and tooling. These assumptions guided both the design of the survey and the subsequent log analysis.

One key assumption is that developer tools strongly influence engineers' preferences for a particular repository model. Internal surveys had consistently shown that the organization's development tools - such as code browsing, building, testing, and review systems - receive very high satisfaction ratings from engineers. Because of this, there was a concern that developers might rate the monolithic repository highly not because of the repository structure itself, but because of the quality of the associated tools. To address this potential bias, the survey design included questions that attempted to isolate the influence of tooling by asking participants to compare repository models under the assumption that tooling would remain constant. This approach was intended to distinguish satisfaction with the repository model from satisfaction with the development tools.

A second assumption concerns the importance of code visibility within a monolithic repository. *Developers often claim that the ability to search and explore the entire codebase is critical for productivity*, enabling them to understand APIs, locate examples of usage, and identify reusable components. However, the researchers were uncertain whether engineers truly make frequent use of this capability or whether its perceived importance stems mainly from the theoretical benefit of having access to all code. To investigate this, the study incorporated log analysis to measure actual developer interactions with the codebase. The logs allowed the researchers to determine how often engineers view or edit code outside their own teams or projects, thereby validating whether the visibility advantage is actively utilized in practice.

A third assumption relates to the potential complexity of very large codebases. Prior internal surveys suggested that the massive scale of the organization's monolithic repository could overwhelm engineers due to the size and complexity of the codebase. *Developers had previously reported concerns that navigating such a large repository might introduce cognitive overhead or make it difficult to identify relevant components.* The researchers therefore expected complexity to emerge as a significant theme when comparing monolithic and multi-repository systems. At the same time, they sought to understand whether these challenges outweigh the potential benefits of visibility, centralized dependency management, and standardized tooling.

12.2.2 Survey Methodology

To understand developers' perceptions of monolithic repositories and compare them with experiences in multi-repository environments, the study conducted a large-scale survey of software engineers within the organization. **The researchers randomly selected 1,902 engineers from a population of approximately 23,000 software engineers. To ensure participants had sufficient familiarity with the repository and development tools, the sample was restricted to engineers who had worked at the company for at least three months, had committed code to the repository in the previous six months, and had spent an average of at least five hours per week using developer tools**. The survey invitation followed established best practices from prior software engineering survey research to maximize participation, and reminder emails were sent to participants who had not completed the survey within the first 24 hours [7]. Ultimately, 869 engineers completed the survey, resulting in a response rate of approximately 46%.

The survey was structured into three blocks of questions, with certain sections shown only to participants who had relevant prior experience.

1. The first block was presented to all respondents and focused on overall satisfaction with the organization's monolithic repository, as well as perceptions of how the ability to search or edit code across the repository affects development velocity and code quality. It also collected background information about participants' prior experiences with different types of development environments, such as multi-repository corporate systems or open-source projects.
2. The second block was shown only to engineers who had previously worked with multi-repository codebases. These questions asked participants to compare their prior experiences with the organization's monolithic repository, identify benefits of each approach, and indicate which model they preferred.
3. The third block targeted engineers with open-source development experience and asked similar comparative questions regarding open-source repositories and the monolithic repository.

In addition to quantitative rating questions, **the survey included free-response questions to capture developers' reasoning and motivations behind their preferences. The researchers analyzed these responses using an open coding methodology, in which responses were categorized into thematic tags**. Initially, common responses were identified and assigned labels representing recurring themes. The remaining responses were then collaboratively coded by multiple researchers to ensure consistent interpretation. Some responses were assigned multiple tags when they addressed several themes simultaneously. Disagreements between coders were resolved through discussion and re-examination of the responses, and the tagging process was refined as new themes emerged. *The resulting coded data allowed the researchers to identify patterns in developer perceptions and quantify the most frequently cited advantages and disadvantages of different repository models.*

12.2.3 Log Based Methodology

In addition to the survey, the study conducted a quantitative analysis of developer tool logs to examine how engineers actually interact with the monolithic repository. **While the survey captured developers' perceptions and preferences, the log analysis provided behavioral evidence regarding how the repository is used in practice. The goal was to determine whether engineers truly take advantage of the visibility and cross-project access enabled by a monolithic repository**, particularly by measuring how frequently developers view or modify code outside their immediate areas of responsibility.

The analysis focused on two types of logs generated by internal developer tools.

1. **Code Browsing Logs** which recorded every instance in which an engineer viewed a source file using the organization's web-based code browsing tool. This tool is commonly used by developers to search and explore the codebase, particularly because traditional local development environments have difficulty indexing and navigating extremely large repositories.
2. **Code Commit Logs** which captured the changes engineers submitted to the repository. By examining both file views and code commits, the researchers were able to study both passive

interactions (such as reading code) and active contributions (such as modifying code).

To determine whether interactions occurred outside a developer's primary project area, the researchers analyzed activity at the level of Product Areas (PAs) within the organization. The company's engineering structure contained twelve such areas, and engineers were assigned to one of them during each week of the study period. Any interaction with code belonging to a different product area was therefore considered likely to represent cross-project activity. To enable this analysis, the researchers created mappings between engineers and product areas, as well as between source files and product areas. File ownership information was derived from project metadata that identified which directories belonged to particular projects and product areas. When metadata was incomplete or ambiguous, the researchers inferred ownership by examining code review records, since reviewers were typically the owners responsible for the corresponding directories.

After establishing these mappings, **the researchers calculated the percentage of each engineer's file views and commits that occurred outside their assigned product area. These percentages were computed weekly for each engineer over a one-year period and then averaged across time**. The resulting distribution allowed the researchers to quantify how frequently developers interacted with code outside their primary domain. This analysis provided empirical evidence about the extent to which engineers utilize the visibility and cross-project collaboration capabilities that monolithic repositories are designed to support.

12.2.4 Threats to Validity

The authors discuss several threats to validity that may affect the interpretation of the study's results. One potential issue is selection bias in the survey population. **Developers may choose to work at organizations whose repository structures align with their personal preferences**. For example, engineers who prefer multi-repository environments may choose companies that use such systems, while those comfortable with monolithic repositories may prefer organizations structured around that model. **As a result, the survey responses may reflect the preferences of developers who are already accustomed to a monolithic repository environment**. Additionally, although the survey achieved a relatively high response rate of 46%, there is still a possibility of nonresponse bias, where the opinions of engineers who did not participate could differ from those who responded.

Another potential threat arises from the influence of internal developer tools. **Engineers might have rated the monolithic repository positively not because of the repository structure itself, but because of the quality of the tools used to interact with it. Since the organization has invested heavily in specialized tools for code browsing, building, testing, and reviewing code, these tools may strongly shape developer satisfaction**. To mitigate this confounding factor, the survey included questions designed to separate preferences for repository structure from preferences for tooling by asking participants to compare repository models under the assumption that the same tools would be available in both environments.

The survey design may also be affected by priming effects. Certain questions asked participants to consider how the ability to search and edit the entire codebase influences development velocity and code quality. These questions might have influenced how participants later evaluated the advantages of monolithic repositories, potentially encouraging them to think more about visibility and productivity benefits when responding to later survey questions. Such priming could bias responses toward emphasizing certain perceived benefits.

Another limitation relates to the subjectivity of the qualitative analysis process. The researchers used an open-coding methodology to categorize free-response survey answers into thematic groups. While multiple researchers collaborated during the coding process to reduce individual bias and disagreements were resolved through discussion, qualitative coding inherently involves interpretation. Therefore, the resulting themes and categorizations may not represent the only possible interpretation of the responses.

Finally, there are potential validity threats in the log analysis methodology, particularly in the heuristic used to map source files and developers to product areas. Although most directories were mapped using project metadata or code review records, some directories lacked clear ownership information or had multiple possible owners. In such cases, the researchers relied on majority ownership assumptions or

excluded certain directories from the analysis. These approximations may introduce small inaccuracies in determining whether interactions occurred inside or outside a developer's primary product area, though the authors report that the overall impact of such inaccuracies is minimal.

12.3 Results and Key Findings

12.3.1 Results

The results of the study combine insights from the survey responses and the developer tool log analysis to understand how engineers perceive and use a monolithic repository. The findings reveal both strong preferences for the monolithic model and clear trade-offs compared with multi-repository environments. **Overall, the results suggest that the advantages of visibility, code reuse, and centralized dependency management are major drivers behind developer satisfaction with monolithic repositories**.

From the survey responses, the study first analyzed the background of the 869 engineers who participated. A large portion of respondents had diverse development experiences, including starting new codebases from scratch, working with corporate multi-repository environments, and contributing to open-source projects. This diversity allowed the researchers to compare perceptions of the monolithic repository against alternative repository models based on real developer experience. The survey also examined how developers perceived the impact of repository features on development velocity and code quality. **The majority of participants reported that the ability to search across the entire codebase is highly important for both productivity and code quality. In contrast, opinions were more mixed regarding the importance of the ability to edit code across projects, suggesting that visibility of code may be more valuable than cross-project modification capabilities in everyday development tasks**.

The log analysis results provided behavioral evidence supporting the importance of code visibility in a monolithic repository. The analysis measured how often engineers interacted with code belonging to different product areas within the organization. **The results showed that developers frequently view code outside their own teams or projects. For instance, the median value indicated that more than a quarter of code views by engineers involved files from outside their assigned product area. Similarly, a significant portion of engineers committed changes to code belonging to other product areas**. These findings demonstrate that developers actively explore and interact with code across the organization, confirming that the visibility provided by a monolithic repository is widely utilized in practice.

To further understand the nature of cross-project interactions, the researchers examined whether developers primarily accessed widely used shared libraries or whether they explored other parts of the codebase as well. The analysis excluded highly common files such as widely used libraries, build configuration files, and service interface definitions. **Even after excluding these commonly accessed files, the patterns of cross-project interactions remained largely unchanged. This indicates that developers are not only accessing well-known shared libraries but are also exploring a wide variety of code throughout the repository**, often to understand implementations or find examples of API usage.

12.3.1.1 Participants with Corporate Multi-Repo Codebase Experience

A significant portion of the survey participants reported prior experience working in organizations that used multiple repositories for managing source code. *Out of the 869 engineers who completed the survey, 379 participants indicated that they had previously worked with a corporate multi-repository codebase.* This subgroup provided an important basis for comparing developer experiences between monolithic repositories and multi-repository environments, since these participants had direct exposure to both development models. Their responses allowed the researchers to examine satisfaction levels, preferences, and perceived advantages of each repository structure based on real-world experience rather than hypothetical comparisons.

The study first compared developer satisfaction with the current monolithic repository against satisfaction with their previous multi-repository environments. **The results showed a clear difference in overall satisfaction levels. While engineers reported generally high satisfaction with the monolithic repository used in the organization, their satisfaction with previous multi-repository systems was considerably more mixed**. This suggests that many developers found the monolithic repository to provide a more consistent or supportive development environment compared with the multi-repository systems they had previously used.

The survey also asked participants to directly compare the two repository models and indicate which they preferred. Among the respondents with multi-repository experience, a large majority expressed a preference for the monolithic repository, while only a small number preferred their previous multi-repository environments and some participants reported no strong preference. When participants explained the reasons behind their choices, several recurring themes emerged. **The most frequently cited motivations for preferring the monolithic repository included improved code reuse, the ability to search and view the entire codebase, and simplified dependency management**. These factors enable developers to easily discover APIs, examine implementation examples, and integrate existing components without needing to locate code across separate repositories.

Despite the strong preference for the monolithic repository, participants also identified several advantages of multi-repository systems. **One commonly cited benefit was the ability to maintain stable and versioned dependencies, which allows projects to control when updates are adopted rather than automatically receiving changes from other parts of the codebase. Developers also associated multi-repository systems with improved build performance, since smaller repositories often require fewer dependencies to be compiled**. Additionally, multi-repository environments allow teams greater flexibility in selecting development tools and configuring their own workflows, which some developers considered beneficial for project autonomy.

The survey further explored how engineers would prefer to structure the organization's codebase if given the option. Even when asked to assume that developer tools would remain the same in both scenarios, most participants still preferred maintaining the existing monolithic repository structure rather than splitting the codebase into multiple repositories. However, a minority of respondents expressed interest in a multi-repository model primarily due to concerns about build times and the stability of dependencies. These responses highlight the inherent trade-offs between centralized coordination and flexibility in repository design.

Overall, the responses from developers with corporate multi-repository experience indicate that while both repository models have advantages, engineers in this study generally favored the monolithic repository because of its ability to improve visibility, facilitate code reuse, and simplify dependency management across a large organization.

12.3.1.2 Participants with Open-Source Experience

The survey also included a subgroup of engineers who had prior experience contributing to open-source projects. *Out of the 869 survey respondents, 337 participants reported working with open-source repositories. These repositories are particularly relevant for comparison because they typically provide broad visibility into source code, similar to monolithic repositories, while still following a multi-repository structure.* By analyzing the responses of developers familiar with open-source development, the study aimed to understand how engineers perceive the differences between open-source repository environments and the organization's monolithic repository.

Participants in this group were asked to compare their experience with their most recent open-source codebase and the monolithic repository used within the organization. The results showed that most respondents preferred the monolithic repository, while a smaller number favored the open-source repository environment and some participants reported no strong preference. **These responses indicate that although open-source repositories offer many advantages, developers in this study generally viewed the monolithic repository as providing a more efficient environment for large-scale collaborative development within an organization**.

Respondents were also asked to identify the benefits of working within the monolithic repository when

compared with open-source codebases. **Many developers highlighted advantages such as the ability to easily search and explore the entire codebase, improved code reuse, and the availability of usage examples for APIs. Developers also cited the ease of updating dependencies and propagating changes across projects, as well as the presence of integrated development tools that support code browsing, testing, and collaboration**. These characteristics make it easier for engineers to understand how different parts of the system interact and to reuse existing components when implementing new features.

At the same time, respondents identified several advantages associated with open-source repositories. One commonly cited benefit was the strong sense of community and collaboration, where developers contribute to projects that are shared with a broader public audience. Participants appreciated the opportunity to share their work with the wider developer community and contribute to widely used software. Open-source environments were also associated with greater flexibility in tools and workflows, allowing developers to choose technologies and practices that best fit their projects. In addition, the typically smaller size of individual repositories in open-source development can reduce complexity and make projects easier to navigate.

Overall, the responses from developers with open-source experience highlight both similarities and differences between open-source and monolithic repository environments. While both models provide significant visibility into code, the monolithic repository offers stronger integration of tools, easier cross-project coordination, and improved opportunities for code reuse within a large organization. Conversely, open-source repositories emphasize community engagement, flexibility, and autonomy in development practices.

12.3.2 Key Findings

Visibility of the Codebase Improves Development Efficiency One of the most important findings of the study is that the visibility provided by a monolithic repository strongly benefits developers. Engineers reported that the ability to search and explore the entire codebase significantly improves development velocity and code quality. Developers frequently rely on this visibility to understand APIs, examine implementations, and locate reusable components. The log analysis confirmed that engineers regularly view code across different product areas, demonstrating that this visibility is actively used rather than simply perceived as beneficial.

Code Reuse and API Discovery Are Major Advantages Another key finding is that monolithic repositories facilitate code reuse and easier API discovery. Because all projects exist within a single repository, developers can quickly find examples of how APIs are used in other parts of the organization. This enables engineers to reuse existing functionality instead of reimplementing similar features. The visibility of implementation details and usage examples helps developers learn how to integrate APIs effectively and reduces the effort required to adopt shared libraries.

Centralized Dependency Management Simplifies Development The study found that monolithic repositories simplify dependency management across projects. In a monolithic system, dependencies are maintained within the same repository and typically use a single version at the repository head. This allows changes to libraries and APIs to propagate across the codebase more easily and reduces issues such as incompatible versions of the same dependency. Developers reported that this centralized approach eliminates many of the coordination challenges commonly found in multi-repository environments.

Trade-off Between Dependency Updates and Stability Despite the advantages of centralized dependency management, the study identifies a trade-off between automatic updates and dependency stability. While monolithic repositories allow developers to receive updates to dependencies automatically, this can sometimes cause disruptions if upstream changes introduce breaking modifications. In contrast, multi-repository systems allow projects to maintain versioned dependencies and update them only when they choose. This trade-off highlights the balance between ease of updates in monolithic repositories and stability in multi-repository environments.

Developer Tools Play a Critical Role The research also found that developer tools are a significant factor in developer satisfaction, sometimes even more influential than the repository structure itself. Engineers frequently cited internal tools—such as code browsing and review systems—as key reasons for preferring the monolithic repository environment. At the same time, developers working in open-source environments often valued industry-standard tools such as Git. This finding suggests that the effectiveness of repository models is closely tied to the quality of the tooling that supports them.

Tension Between Standardization and Flexibility The study highlights a tension between standardization and flexibility in repository design. Monolithic repositories enforce consistent development practices, standardized tooling, and shared coding styles across the organization. Many developers consider this consistency beneficial because it improves code quality and reduces confusion when working across projects. However, some developers prefer the flexibility offered by multi-repository environments, where teams can select their own tools, programming languages, and workflows based on the needs of their projects.

Managing Cognitive Load in Large Codebases Another key insight relates to developer cognitive load when working with large codebases. Some engineers believed that monolithic repositories reduce cognitive load because developers can easily search the codebase, find examples, and rely on consistent tools. Others argued that the large size of a monolithic repository can increase complexity and make it harder to navigate. In contrast, multi-repository systems reduce cognitive load by limiting the scope of the code developers must understand, as each repository is typically smaller and more focused.

Cross-Project Collaboration Is Common The log analysis revealed that cross-project interaction occurs frequently in a monolithic repository. Developers regularly view and modify code belonging to different product areas, indicating that engineers collaborate across team boundaries more often than expected. Importantly, this behavior persists even after excluding commonly accessed libraries, suggesting that developers actively explore diverse parts of the codebase rather than only interacting with widely used shared components.

12.3.3 Implications

Repository Visibility Supports Organizational Knowledge Sharing One implication highlighted by the study is that repository visibility can significantly improve knowledge sharing across an organization. When developers have access to the entire codebase, they can examine implementations, understand how APIs are used, and identify reusable components. This allows engineers to learn from existing solutions and reduces duplicated development effort. As a result, organizations adopting monolithic repositories may benefit from faster onboarding and improved collaboration across teams.

Centralized Dependency Management Reduces Coordination Overhead The findings suggest that centralized dependency management in monolithic repositories can simplify coordination between teams. Since all dependencies are maintained within the same repository and are typically updated together, developers can propagate changes to dependent systems more easily. This reduces the need for complex communication and version negotiation that often occurs in multi-repository environments when dependencies change. Consequently, organizations may experience smoother integration and maintenance of shared libraries.

Developer Tooling Is Critical to Repository Model Success Another implication from the study is that the effectiveness of any repository model depends heavily on the quality of the supporting development tools. Engineers in the study frequently cited developer tools—such as code browsing and review systems—as key factors influencing their satisfaction with the monolithic repository. This suggests that organizations considering large-scale repository models must invest in strong tooling infrastructure to ensure efficient navigation, building, and testing of large codebases.

Repository Design Should Align with Organizational Scale An additional implication is that repository structure should be chosen based on the scale and collaboration patterns of an organization.

Large organizations with many interconnected projects may benefit from monolithic repositories because they enable cross-team collaboration and code reuse. In contrast, smaller organizations or independent teams may find multi-repository systems more suitable, as they provide flexibility and reduced complexity. This suggests that repository architecture decisions should consider organizational size, dependency structure, and development workflows.

Improved Tooling Could Mitigate Monorepo Complexity The study also implies that many challenges associated with monolithic repositories—such as navigating extremely large codebases—can potentially be mitigated through improved tooling. Advanced code search, dependency visualization, and automated refactoring tools could help developers navigate large repositories more effectively. By investing in such capabilities, organizations could preserve the benefits of repository visibility while minimizing the cognitive load associated with large codebases.

Hybrid Repository Models May Provide Balanced Trade-offs Finally, the results suggest that hybrid repository strategies may be worth exploring. While monolithic repositories provide strong visibility and centralized dependency management, multi-repository systems offer stability and flexibility. A hybrid approach—such as grouping related systems into larger repositories while keeping loosely connected projects separate—could allow organizations to balance the benefits of both models. Such approaches may help organizations tailor repository structures to the specific needs of their development ecosystem.

12.4 Related Work

Research on monolithic repositories is relatively limited because the widespread adoption of such repository structures in industry is a comparatively recent development. However, some prior work has explored the scale and rationale behind these repository models. For example, earlier discussions of large-scale codebase management describe how major technology companies store massive codebases within a single repository to support collaboration across thousands of developers and enable consistent development practices [3]. These industry experiences demonstrate that monolithic repositories can scale to extremely large codebases while enabling shared tooling and centralized development workflows.

Related work has also examined how developers interact with large codebases, particularly through code search and exploration tools. Studies investigating developer search behavior have shown that developers frequently rely on searching across large codebases to understand APIs, examine implementations, and debug issues [4]. These findings highlight the importance of visibility and code discovery in software development, supporting the idea that large repositories with comprehensive search capabilities can improve developer productivity. The results of the present study reinforce these observations by showing that engineers actively use repository-wide search and browsing capabilities.

Another relevant line of research focuses on version control systems and repository management practices. Early version control systems such as the Source Code Control System and the Revision Control System established foundational mechanisms for tracking code changes and managing collaboration in software projects [8, 9]. Later work has explored the conceptual design and usability challenges of modern version control systems such as Git [10, 11]. While most traditional version control systems were designed around per-project repositories, these systems can also support monolithic repository structures with appropriate tooling and infrastructure.

The rise of distributed version control systems has also prompted research on the differences between centralized and distributed development models. Some studies examine how these systems influence collaboration, development workflows, and software changes in large projects [12, 13]. Although this research primarily focuses on centralized versus distributed version control, it provides insights into how repository organization and collaboration models affect development practices. These questions are related but orthogonal to the monolithic versus multi-repository debate explored in this paper.

Research on dependency management and coordination among developers is also closely related to repository structure. Prior empirical studies have shown that developers often rely on communication channels to coordinate changes to shared components and dependencies [6]. In multi-repository environments,

teams must notify downstream projects about changes to shared libraries, which can introduce coordination overhead. Monolithic repositories attempt to address this challenge by centralizing dependencies and allowing updates to propagate across the codebase simultaneously.

Finally, a large body of work has examined open-source repositories and collaborative development ecosystems. Studies of platforms such as GitHub have analyzed collaboration patterns, programming language usage, and community dynamics in open-source projects [14, 15, 16]. While open-source repositories share certain characteristics with monolithic repositories—such as broad visibility into source code—they typically lack the centralized tooling and unified dependency management found in monolithic repository environments. These differences highlight the distinct trade-offs between open-source repository ecosystems and large organizational monolithic repositories.

References

[1] Durham Goode and Siddarth P. Agarwal. *Scaling Mercurial at Facebook*. https://code.facebook.com/posts/218678814984400/scaling-mercurial-at-facebook/. 2014.

[2] Brian Harry. *Scaling Git at Microsoft*. https://blogs.msdn.microsoft.com/bharry/2017/02/03/scaling-git-and-some-back-story/. 2017.

[3] Rachel Potvin and Josh Levenberg. "Why Google Stores Billions of Lines of Code in a Single Repository". In: *Communications of the ACM* (2016).

[4] Caitlin Sadowski, Kathryn T. Stolee, and Sebastian Elbaum. "How Developers Search for Code: A Case Study". In: *Proceedings of the 2015 International Symposium on Foundations of Software Engineering*. 2015.

[5] Mark Florisson and Alan Mycroft. *Towards a Theory of Packages*. http://www.cl.cam.ac.uk/~mbf24/packages.pdf. 2017.

[6] Cleidson R. B. de Souza and David F. Redmiles. "An Empirical Study of Software Developers' Management of Dependencies and Changes". In: *Proceedings of the International Conference on Software Engineering (ICSE)*. 2008.

[7] Edward Smith et al. "Improving Developer Participation Rates in Surveys". In: *Proceedings of the International Workshop on Cooperative and Human Aspects of Software Engineering*. 2013.

[8] Marc J. Rochkind. "The Source Code Control System". In: *IEEE Transactions on Software Engineering* (1975).

[9] Walter F. Tichy. "RCS—A System for Version Control". In: *Software: Practice and Experience* (1985).

[10] Santiago Perez De Rosso and Daniel Jackson. "What's Wrong with Git?: A Conceptual Design Analysis". In: *Onward!* 2013.

[11] Santiago Perez De Rosso and Daniel Jackson. "Purposes, Concepts, Misfits, and a Redesign of Git". In: *OOPSLA*. 2016.

[12] Caius Brindescu et al. "How Do Centralized and Distributed Version Control Systems Impact Software Changes?" In: *ICSE*. 2014.

[13] Brian de Alwis and Jonathan Sillito. "Why Are Software Projects Moving from Centralized to Decentralized Version Control Systems?" In: *CHASE*. 2009.

[14] Eirini Kalliamvakou et al. "The Promises and Perils of Mining GitHub". In: *Mining Software Repositories*. 2014.

[15] Bogdan Vasilescu et al. "The Sky Is Not the Limit: Multitasking Across GitHub Projects". In: *ICSE*. 2016.

[16] Baishakhi Ray et al. "A Large Scale Study of Programming Languages and Code Quality in GitHub". In: *FSE*. 2014.

Chapter 13

[Required] Building and Sustaining Ethnically, Racially, and Gender Diverse Software Engineering Teams: A Study at Google

This paper investigates how software engineering teams can successfully build and sustain diversity across race, ethnicity, and gender within a large technology organization. Although prior research suggests that diverse teams can improve innovation, productivity, and software quality, many software engineering teams remain demographically homogeneous. The authors argue that while many organizations publicly commit to diversity goals, there is limited empirical evidence on the specific practices that actually help teams achieve and maintain diversity. To address this gap, the study examines the experiences and practices of highly diverse engineering teams at Google in order to understand what actions contribute to building inclusive and representative teams.

To explore this question, the researchers conducted 20 semi-structured interviews with engineers and managers from some of the most diverse engineering teams within the organization. Using qualitative thematic analysis, the study identifies practices related to recruiting, hiring, and creating inclusive work environments that help sustain diversity. The findings highlight two major themes: first, strategies used by managers to intentionally recruit diverse candidates, and second, the role of inclusive practices—particularly the concept of *technical allyship* — in supporting engineers from underrepresented groups. These insights provide actionable recommendations for engineers, managers, and organizational leaders seeking to improve diversity and inclusion in software engineering teams.

13.1 Motivation

Software engineering teams play a central role in shaping the technologies used by millions of people, yet these teams remain largely demographically homogeneous across many organizations. When teams lack diversity, the perspectives that influence how software is designed, implemented, and evaluated can become limited. **Research has shown that diversity within collaborative engineering teams can produce tangible benefits. For example, differences in cognitive styles among team members have been associated with increased novelty in software design decisions, while gender and tenure diversity have been linked with improved team productivity and performance outcomes** [1, 2]. These findings suggest that diversity is not only a social or ethical concern but also a factor that can shape the technical outcomes and effectiveness of software development teams.

Beyond productivity benefits, diversity can also influence the social and organizational dynamics of software teams. **Studies have found that teams with greater representation of women tend to**

experience fewer organizational issues known as community smells, which refer to problematic communication and collaboration patterns within development communities [3]. Other research indicates that increased gender diversity can help reduce biases among team members and foster more inclusive work environments [4]. These insights have motivated many large technology companies to publicly commit to improving diversity and representation in their engineering workforce, often through organizational diversity initiatives and hiring goals.

Despite these commitments, there remains a significant gap between the recognition of diversity's benefits and the availability of evidence-based practices for achieving it. *Much of the existing literature focuses on the consequences of diversity rather than on the processes that help organizations build diverse teams in the first place. As a result, recommendations for improving diversity often rely on anecdotal accounts or high-level organizational policies rather than empirically grounded practices that engineering teams and managers can implement in their day-to-day work* [5]. This lack of actionable knowledge makes it difficult for organizations to translate diversity goals into practical strategies.

Another motivation arises from the persistent challenges faced by developers from underrepresented groups in the software industry. Prior research has documented several structural and cultural barriers affecting these developers, including bias in evaluation processes, discrimination in professional environments, isolation within teams, and lower acceptance rates of technical contributions in collaborative projects [6, 7]. These challenges can discourage participation and retention, further reinforcing the lack of diversity in software engineering. Understanding how certain teams successfully overcome these challenges and cultivate inclusive environments therefore becomes essential for identifying practices that can help organizations recruit, support, and retain a more diverse engineering workforce.

13.1.1 Related Work

Research on diversity in software engineering and organizational settings has grown substantially in recent years. Much of this literature examines the consequences of diversity within teams, particularly how demographic or cognitive differences influence team performance, innovation, and collaboration outcomes. For example, studies of software development teams have shown that diversity in gender, experience, and tenure can positively affect productivity and collaborative dynamics [2]. Similarly, work on employee diversity in innovation-driven environments suggests that teams composed of individuals with varied perspectives and backgrounds are more likely to produce novel ideas and solutions [8]. These studies provide empirical support for the argument that diversity can enhance the effectiveness of collaborative technical work.

Beyond performance outcomes, researchers have also investigated how diversity shapes social interactions and organizational structures within software development communities. Prior work has documented that gender diversity can influence collaboration patterns and reduce negative organizational phenomena such as community smells, which represent problematic coordination and communication structures within development teams [3]. Other studies have explored the presence of implicit bias in professional software development environments, demonstrating that stereotypes and unconscious biases can influence evaluation, collaboration, and decision-making processes among developers [4]. These findings highlight that diversity not only affects technical outcomes but also interacts with social and cultural dynamics within engineering teams.

Another important line of research focuses on the experiences and challenges faced by developers from underrepresented groups in the software industry. Empirical studies have documented barriers such as discrimination, social isolation, and reduced recognition of technical contributions. For instance, investigations of contribution evaluation in open-source software communities reveal disparities in how contributions from developers of different genders or perceived identities are reviewed and accepted [7]. Similarly, analyses of demographic perceptions in collaborative software projects have shown that factors such as race and ethnicity can influence how contributions are evaluated and perceived by other developers [6]. These findings illustrate structural challenges that may discourage participation or retention among members of underrepresented groups.

Research has also examined the signals and environmental factors that influence developers' decisions to join or remain in software projects. For example, studies of open-source project participation have found that potential contributors evaluate factors such as team composition, communication style, and

inclusivity when deciding whether to contribute to a project [9]. In addition, broader research in diversity management emphasizes the role of organizational strategies, leadership commitment, and recruitment practices in shaping workforce diversity [5]. Together, these studies suggest that improving diversity requires not only addressing structural barriers but also creating environments that actively support inclusion and equitable participation.

13.2 Methodology

13.2.1 Matching Engineers with Teams

The study examines how engineers are matched with teams within a large technology company and how this process influences team composition and diversity. At the time of the study, teams could recruit engineers through two main pathways: external hiring and internal transfers. External candidates typically applied through role-based job postings such as software engineering positions. After completing screening and interview stages, qualified candidates entered a team-matching phase where they were paired with specific teams that had open positions. During this process, hiring managers could evaluate multiple candidates simultaneously, while candidates could also interact with several teams before selecting a role. This matching process created a two-sided selection mechanism in which both teams and candidates influenced the final placement decision.

In addition to external hiring, existing employees within the organization could transfer between teams. Internal candidates were able to browse job listings on an internal platform and apply directly to teams that had open roles. Unlike external applicants, internal candidates often had access to additional contextual information about teams and hiring managers before applying, such as internal profiles, documentation, or examples of recent work. This visibility allowed employees to better evaluate team culture, leadership style, and technical focus when deciding whether to transfer. As a result, internal mobility functioned as another mechanism through which engineers could select teams that aligned with their professional goals and working preferences.

13.2.2 Team Selection

To study how diversity is cultivated within software engineering teams, the researchers first **identified candidate teams that could meaningfully represent diverse environments. Teams were defined as groups of engineers reporting directly to the same manager**. The researchers applied several criteria to narrow down the list of potential teams in order to ensure that the teams selected were both stable and organizationally meaningful units.

1. One criterion involved team size: only teams consisting of five to fifteen engineers were considered. Smaller teams were excluded because diversity metrics would be unreliable with very few members, while very large teams were excluded because the manager-employee relationship might be less direct and the organizational structure less cohesive. Additionally, managers were required to have at least four years of management experience to ensure that they had sufficient experience managing teams across different organizational contexts and time periods.

2. The researchers also applied demographic criteria to ensure that selected teams exhibited meaningful diversity. Teams were required to demonstrate diversity across both race or ethnicity and gender. For race and ethnicity, the researchers used a richness metric requiring at least four distinct racial or ethnic groups to be represented within a team. For gender diversity, the Gini-Simpson index was used to measure the probability that two randomly selected team members would have different genders. Teams were included only if their gender diversity index exceeded a specified threshold.

3. In addition, at least one engineer from an underrepresented group needed to have remained on the team for more than two years and received a promotion during that period. This condition served as evidence that the team was capable not only of recruiting diverse engineers but also of supporting their retention and career advancement.

13.2.3 Interviewee Selection

After identifying candidate teams, the researchers selected interview participants from two key roles within those teams: managers and software engineers. **Managers were prioritized if they had longer tenure within the organization and experience managing multiple teams, since such experience would allow them to compare diversity-related practices across different teams and contexts**. This perspective was important because managerial decisions and leadership behaviors often influence hiring practices, team culture, and opportunities for professional development within engineering teams. *By interviewing managers with extensive organizational experience, the researchers aimed to gather insights into the strategies and decisions that contribute to building and sustaining diverse teams.*

For software engineers, the selection criteria emphasized individuals who belonged to one or more underrepresented groups, including women and individuals identifying with racial or ethnic groups other than White or Asian. Engineers were selected from different teams in order to capture a wide range of experiences rather than focusing deeply on a single team. Preference was also given to engineers who had spent longer periods within the organization or on their current team, since these participants could provide more detailed reflections on team culture and diversity practices over time. The researchers also aimed to ensure diversity within the interview sample itself by including participants from different racial, ethnic, and gender backgrounds. Additional steps were taken to include perspectives that might otherwise be underrepresented, such as engineers identifying as non-binary or belonging to demographic groups with very low representation in the workforce.

13.2.4 Interviews

The researchers conducted semi-structured interviews with selected participants in order to gather detailed insights about their experiences with diversity and inclusion within software engineering teams. All interviews were conducted remotely via teleconference and typically lasted approximately sixty minutes, although one interview lasted around thirty minutes. Semi-structured interviews were chosen to maintain consistency across participants while still allowing interviewees to discuss unexpected themes and experiences. Each interview began with an introduction to the study, followed by a request for permission to record the session and an explanation of how participants' identities and responses would be protected to ensure confidentiality.

The interview questions were developed iteratively through several stages. Initially, the researchers created questions focused on recruitment and hiring practices related to diversity. These questions were later refined through feedback from colleagues and through pilot interviews with both managers and engineers. The final interview guides included separate sets of questions for managers and engineers. Managers were asked about their experiences building diverse teams, their hiring strategies, and how they fostered inclusive team cultures. Engineers were asked about their experiences working on different teams, their perceptions of diversity within those teams, and the factors influencing their decisions to join or leave teams. Open-ended questions were deliberately used so that participants could describe their experiences without being guided toward specific diversity practices discussed in existing literature.

13.2.5 Data Analysis

The collected interview data was analyzed using thematic analysis, a qualitative research technique that identifies patterns and themes across textual data. Interview recordings were first transcribed using automated tools and then manually reviewed to remove identifying information. The researchers conducted multiple rounds of open coding to categorize segments of text into conceptual codes, which were gradually refined and merged into a structured codebook. Throughout this process, the research team met regularly to review emerging codes and discuss interpretations of the data. The coding process continued until both code saturation and meaning saturation were reached, indicating that no new themes were emerging from the data. To improve the credibility of the findings, the researchers also performed member-checking by sharing summarized results with participants and allowing them to verify or comment on the interpretations of their responses.

13.2.6 Self Disclosure

The authors also acknowledged their positionality in relation to the study, following established practices in qualitative research. At the time the research was conducted, all authors were employees of the same organization from which the interview participants were recruited. Because of this relationship, the authors recognized that their professional context could potentially influence the research process and interpretation of findings. Although the researchers maintained autonomy in conducting and publishing the study, they acknowledged that their work was inevitably shaped by their organizational environment and their relationships with colleagues and leadership within the company.

13.3 Results

13.3.1 Recruiting and Hiring

The first major results theme in the paper is that improving diversity in software engineering teams requires intentional action during recruiting and hiring, rather than treating representation as something determined only by the external pipeline. A recurring finding was that managers on highly diverse teams did not regard lack of diversity as an unavoidable outcome; instead, they saw themselves as active agents who could influence team composition. The paper describes this as a shift toward self-empowering hiring managers. *Participants explained that meaningful change began when managers first recognized that a diversity gap existed on their teams and then accepted responsibility for doing something about it.* In other words, the successful teams in the study were not simply lucky or passively diverse; their managers were described as people who reflected on their own incentives, assumptions, and prior practices, and then deliberately changed how they approached hiring. This is an important result because it frames diversity as something affected by everyday managerial choices rather than as a problem outsourced entirely to educational systems or upstream recruiting pipelines.

A second major result is that these managers broadened the pool of candidates while simultaneously questioning the assumptions they brought into hiring decisions. The paper shows that managers used several strategies to enlarge the candidate pool: *they asked engineers from underrepresented groups to tap into their external networks, advertised roles more broadly, and built longer-term relationships through mentorship or outreach programs. At the same time, they reflected on what qualifications were genuinely necessary for success in the role. Instead of relying on a rigid or idealized image of the "right" engineer, they tried to distinguish core requirements from preferences that might unnecessarily narrow the field.* The findings suggest that this dual process—expanding where candidates come from while pruning back biased assumptions about who looks qualified—was central to more inclusive hiring. The study presents this as a practical counterweight to fast, convenience-driven hiring, where managers may be tempted to choose the first acceptable candidate rather than the best fit from a diverse pool.

The paper also emphasizes that inclusive hiring often required managers to slow the process down. **Several participants noted that managers took extra time to make sure job messages reached people from a broader range of backgrounds and to ensure those candidates had time to apply. This mattered because a rapid hiring process could easily favor the most immediately visible or conventionally credentialed candidates, which in practice often reproduced existing demographic patterns**. The study therefore presents time itself as a diversity resource: the willingness to wait, gather a broader candidate set, and resist settling for the first fit created better conditions for equitable hiring. What is notable here is that the paper does not portray these practices as symbolic gestures. Rather, interviewees described them as deliberate interventions that changed who was considered, how candidates were evaluated, and ultimately who joined the team.

Another important result is that candidates from underrepresented groups were not only evaluating the technical role but also looking for evidence that managers were genuinely committed to inclusion. The paper finds that an ongoing commitment to diverse candidates mattered well beyond the mechanics of screening and interviews. Managers signaled this commitment by investing in early-career training programs, learning about the experiences of engineers from underrepresented groups, participating in allyship or diversity-related efforts, and explicitly discussing how they supported career growth on their teams. Engineers reported using these signals to judge whether a team would

be inclusive and whether they could succeed there. *In some cases, engineers even examined publicly visible signs of a manager's involvement in diversity-related activities. This means recruiting was not a one-way process in which only managers assessed applicants; applicants from underrepresented groups were also actively assessing whether managers and teams were trustworthy, aware, and serious about inclusion.* The paper further argues that once managers built a visibly diverse and inclusive team, that itself became a positive signal that attracted more diverse candidates, creating a virtuous cycle.

The final major result in this subsection is that individual manager effort was most effective when reinforced by organizational guidelines and accountability. Participants described leadership-level structures in which hiring managers had to consider multiple candidates and justify their choices to senior leaders such as vice presidents. These accountability mechanisms made it harder to rely uncritically on intuition or bias and increased the likelihood that candidates from underrepresented groups would remain seriously considered throughout the hiring process. At the same time, the paper is careful not to treat top-down policy as sufficient on its own. Interviewees stressed that accountability measures worked best when combined with genuine bottom-up commitment from managers who believed in the goal of increasing diversity. *The study also highlights a practical tension: equitable hiring often takes longer, but managers may feel pressure to fill roles quickly or risk losing headcount. As a result, the paper argues that leadership must do more than mandate inclusive processes; it must also protect the time and resources needed to carry them out.* This makes the recruiting and hiring findings especially strong, because they connect diversity not only to attitudes and intentions but also to institutional structures, incentives, and constraints.

13.3.2 Technical Allyship

Definition 13.3.1 (Technical Allyship)**.** *Technical allyship refers to the deliberate actions taken by individuals within a technical team to support and advocate for engineers from underrepresented groups in ways that promote equity in technical work environments. It involves identifying and addressing structural or interpersonal biases that may undermine the recognition of these engineers' technical contributions, while actively reinforcing confidence in their expertise and abilities. Unlike general expressions of support, technical allyship focuses specifically on behaviors within technical contexts—such as code reviews, task assignments, mentorship, and performance evaluations—that influence how engineers' skills are perceived and valued.*

The second major theme identified in the study is the concept of technical allyship, which refers to actions taken by team members to support engineers from underrepresented groups in technical contexts. **The authors describe technical allyship as a form of allyship that goes beyond expressing support and instead involves concrete behaviors that address inequities in technical work environments. These actions help reinforce confidence in the technical skills of engineers from underrepresented groups and advocate for them when their expertise may be overlooked due to biases related to gender, race, or ethnicity**. The study highlights that technical allyship can be practiced at multiple levels within a team, including by managers, senior leaders, and coworkers.

Managers were found to play a particularly important role in practicing technical allyship. **Their decisions regarding performance evaluation, task allocation, and career development directly influence the technical opportunities available to engineers. Participants reported that effective managers were careful to gather sufficient evidence before evaluating performance and remained aware of biases that could lead to underestimating the abilities of engineers from underrepresented groups**. Managers also created stretch opportunities by assigning challenging technical work that allowed these engineers to demonstrate their capabilities and develop new skills. These practices helped counter stereotypes and ensured that engineers were not limited to less technical roles or overlooked for advancement opportunities.

The study also highlights the importance of leadership-level allyship. Engineers from underrepresented groups often expressed concern about the lack of diversity among senior managers and technical leaders. Participants indicated that visible representation in leadership positions can signal that career advancement is achievable for individuals from diverse backgrounds. Leaders who intentionally promote diversity within management ranks therefore play

an important role in shaping organizational culture and long-term workforce diversity. Increasing representation at higher levels was described as having a cascading effect, influencing hiring practices and encouraging more inclusive environments across engineering teams.

Finally, coworkers were also found to contribute significantly to technical allyship through everyday interactions. Engineers described how supportive colleagues helped build confidence by acknowledging contributions, engaging respectfully during code reviews, and offering constructive feedback during technical discussions. These interactions helped create an environment where engineers from underrepresented groups felt comfortable sharing ideas and participating in decision-making. In addition, coworkers often provided mentorship, shared their own technical challenges, and advocated for colleagues' contributions during evaluation or promotion processes. Such actions helped mitigate impostor syndrome and strengthened engineers' sense of belonging within their teams.

13.4 Discussion and Implications

13.4.1 Discussion

The discussion section interprets the findings by emphasizing that the practices observed in highly diverse teams can potentially be transferred to other engineering teams seeking to improve diversity. The authors argue that diversity should not be treated as a fixed resource that teams compete for. *A common argument is that diversity is a zero-sum situation, where increasing representation in one team necessarily reduces it elsewhere because the number of engineers from underrepresented groups is limited. The authors challenge this view, suggesting that such reasoning shifts responsibility away from teams and organizations. Instead, they argue that although the current talent pool may appear limited at any given time, it is influenced by broader systemic factors and can expand over time through cultural and structural change.* Engineering teams therefore share responsibility for improving inclusivity within their own environments rather than attributing low diversity solely to upstream pipeline issues.

The authors also discuss practical challenges associated with implementing the practices identified in the study. *Although the practices appear effective in diverse teams, they may involve trade-offs that make adoption difficult in other organizational contexts. For example, inclusive hiring practices often require managers to spend more time identifying and evaluating candidates from diverse backgrounds. However, organizations frequently operate under pressure to fill open roles quickly, which can discourage managers from taking the additional time required to broaden candidate pools.* Additionally, the study found that the managers who successfully cultivated diverse teams were strongly committed to diversity goals and felt personal responsibility for improving representation. The authors suggest that managers who lack this level of commitment or who follow diversity initiatives only superficially may be less successful in achieving meaningful change.

Another important point raised in the discussion concerns the relationship between diversity practices and broader management behaviors. *Many of the practices identified in the study overlap with characteristics of effective engineering leadership described in prior research, such as supporting engineers' growth, encouraging experimentation, and fostering respectful collaboration. At the same time, the study highlights additional behaviors that are particularly important for inclusion, such as carefully evaluating performance to avoid bias and encouraging a culture of mutual respect and mentorship within teams.* The authors also emphasize the role of coworkers, noting that inclusive environments are not created solely through management decisions but also through everyday interactions among engineers. Actions such as recognizing colleagues' contributions, providing constructive feedback, and advocating for fair evaluation help reinforce inclusive team cultures and support the technical confidence and career progression of engineers from underrepresented groups.

13.4.2 Implications

13.4.2.1 Engineers

Engineers play an important role in shaping inclusive team environments through their everyday interactions and collaboration practices. Although diversity initiatives are often discussed at the managerial

or organizational level, the study highlights that individual engineers can actively contribute to more equitable and supportive technical environments.

- Act as technical allies by reinforcing confidence in colleagues from underrepresented groups and acknowledging their technical contributions during discussions, code reviews, and presentations.
- Advocate for fairness in team processes by speaking up when disparities in recognition, workload, or opportunity distribution are observed.
- Avoid reinforcing stereotypes or informal role assignments that may push certain engineers into less technical responsibilities.
- Encourage open collaboration by providing constructive feedback and fostering respectful technical discussions that value diverse perspectives.
- Request transparency from management regarding hiring, promotion, and evaluation processes to ensure equitable treatment within teams.

13.4.2.2 Managers

Managers have significant influence over team composition, culture, and career development opportunities. The study emphasizes that managers must actively examine their own practices and decisions to ensure they are supporting diversity and inclusion within their teams.

- Evaluate diversity and inclusion practices within the team and reflect critically on how hiring, evaluation, and task assignment decisions are made.
- Develop awareness and empathy regarding the experiences of engineers from underrepresented groups by engaging with training programs, allyship initiatives, and educational resources.
- Seek regular informal feedback from team members to understand their experiences and improve team culture.
- Implement inclusive hiring strategies by expanding candidate pools and examining potential biases when evaluating applicants.
- Carefully assess employee performance by gathering comprehensive evidence rather than relying on intuition or limited impressions.
- Provide stretch opportunities and challenging technical tasks to engineers from underrepresented groups to support their growth and visibility.
- Clearly communicate commitment to career development and mentorship when interacting with potential candidates during recruiting.

13.4.2.3 Leaders

Senior leaders and organizational decision-makers play a crucial role in shaping policies and structural mechanisms that influence diversity outcomes across teams. Their actions can reinforce inclusive practices and ensure accountability for diversity initiatives throughout the organization.

- Increase diversity in leadership positions to provide visible representation and demonstrate that career advancement is achievable for engineers from underrepresented groups.
- Establish structured hiring and promotion practices that require managers to consider diverse candidate pools during recruitment.
- Implement accountability mechanisms that require managers to document and justify hiring decisions, helping reduce the influence of bias.
- Recognize and reward managers who actively improve diversity and foster inclusive team cultures.
- Provide managers with sufficient time and organizational support to conduct inclusive hiring processes without the pressure to fill positions rapidly.

13.4.3 Limitations

The study has several limitations that affect the generalizability and interpretation of its findings. First, the research was conducted within a single large technology company, and all interview participants were employees working in the United States. As a result, the organizational context, hiring practices, and team structures examined in the study may differ significantly from those in smaller companies, startups, or open-source communities. For example, organizations with fewer resources or less formal recruitment processes may not be able to implement some of the practices identified in the study, such as extensive hiring pipelines or structured accountability mechanisms for recruitment decisions. Consequently, the transferability of the findings may be limited to organizations with similar scale, infrastructure, and workforce demographics [10].

Another limitation arises from the scope of diversity considered in the research and the potential for bias in qualitative self-report data. The study primarily focuses on diversity in terms of race, ethnicity, and gender using broad demographic categories. Other dimensions of diversity—such as socioeconomic background, disability, or cultural identity—were not examined and may require different strategies to address. Additionally, because the study relied on interviews, participants may have been influenced by social desirability bias, potentially presenting their teams or practices in a more positive light than reality. Finally, the study may also be subject to survivorship bias, since it interviewed current employees on successful teams rather than engineers who left the organization or the profession due to negative experiences [11].

References

[1] Carianne Pretorius et al. "Combined intuition and rationality increases software feature novelty for female software designers". In: *IEEE Software*. Vol. 38. 2. IEEE, 2020, pp. 64–69.

[2] Bogdan Vasilescu et al. "Gender and tenure diversity in GitHub teams". In: *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. 2015, pp. 3789–3798.

[3] Gemma Catolino et al. "Gender diversity and women in software teams: How do they affect community smells?" In: *ICSE Software Engineering in Society*. 2019, pp. 11–20.

[4] Yi Wang and David Redmiles. "Implicit gender biases in professional software development: An empirical study". In: *IEEE/ACM International Conference on Software Engineering: Software Engineering in Society*. 2019, pp. 1–10.

[5] Patricia A Kreitz. "Best practices for managing organizational diversity". In: *The Journal of Academic Librarianship* 34.2 (2008), pp. 101–120.

[6] Reza Nadri, Gema Rodríguez-Pérez, and Meiyappan Nagappan. "On the relationship between the developer's perceptible race and ethnicity and the evaluation of contributions in OSS". In: *IEEE Transactions on Software Engineering* 48.8 (2022), pp. 2955–2968.

[7] Josh Terrell et al. "Gender differences and bias in open source: Pull request acceptance of women versus men". In: *PeerJ Computer Science* 3 (2017), e111.

[8] Christian R Østergaard, Bram Timmermans, and Kari Kristinsson. "Does a different view create something new? The effect of employee diversity on innovation". In: *Research Policy* 40.3 (2011), pp. 500–509.

[9] Huilian Sophie Qiu et al. "The signals that potential contributors look for when choosing open-source projects". In: *Proceedings of the ACM on Human-Computer Interaction* 3.CSCW (2019), p. 122.

[10] Steve Easterbrook et al. "Selecting empirical methods for software engineering research". In: *Guide to Advanced Empirical Software Engineering*. Springer, 2008, pp. 285–311.

[11] Anton J Nederhof. "Methods of coping with social desirability bias: A review". In: *European Journal of Social Psychology* 15.3 (1985), pp. 263–280.

Chapter 14

[Required] The Costs and Benefits of Pair Programming

The paper "The Costs and Benefits of Pair Programming" investigates the effectiveness of pair programming, a software development practice where two programmers work together at a single computer to design and write code. The authors analyze both interview data and controlled experiments to understand whether pair programming provides measurable benefits compared to solo programming.

The study finds that although pair programming requires about 15% more development time, it significantly improves software quality and team dynamics. Programs produced by pairs tend to contain fewer defects, better design quality, and shorter code, indicating more efficient solutions. In addition, pair programming facilitates continuous code review, faster problem solving, knowledge transfer, and improved communication among team members.

The authors also highlight organizational benefits such as reduced project risk, improved team learning, and greater developer satisfaction. Overall, the results suggest that the modest additional development cost is offset by reduced debugging and maintenance effort, making pair programming economically and practically beneficial in many software development contexts.

14.1 Motivation

The motivation for investigating collaborative programming arises from the tension between traditional assumptions about software development and emerging anecdotal evidence suggesting that collaboration may improve programming outcomes. Software development has historically been viewed as an individual activity in which programmers independently design and implement code. Managers often assume that assigning two programmers to the same task would double development costs and reduce productivity. *However, experienced practitioners and advocates of collaborative development practices have long argued that working in pairs can lead to improved design quality, more robust solutions, and better overall productivity* [1, 2]. These conflicting perspectives create a need for empirical investigation to determine whether collaborative programming actually provides measurable benefits.

Another important motivation stems from the growing recognition that software development is fundamentally a human-centered and cognitive activity rather than purely a technical one. Research in distributed cognition suggests that complex problem solving is often more effective when knowledge and reasoning are shared across multiple individuals who bring different perspectives and experiences to a task [3, 4]. *When programmers work together, they may explore a broader range of design alternatives, challenge each other's assumptions, and identify errors earlier in the development process.* These collaborative interactions can potentially improve the quality of software artifacts while also enhancing the learning and skill development of the programmers involved.

Economic considerations also provide strong motivation for studying pair programming. Defects discovered late in the development lifecycle or after deployment can be extremely expensive to diagnose and repair. Empirical software engineering research has consistently shown that the cost of fixing defects increases dramatically as software moves through successive stages of development and deployment [5, 6]. **If collaborative programming can detect errors earlier through continuous peer review, it may reduce downstream costs associated with testing, debugging, and field support**. Therefore, understanding whether the modest increase in development effort associated with pair programming is offset by reductions in defect-related costs is a key question for both researchers and software managers.

Finally, the motivation for this research also includes organizational and educational considerations within software teams. **Modern software engineering practices increasingly emphasize teamwork, communication, and knowledge sharing as critical factors for project success**. Prior work on software development environments highlights the importance of collaboration and interpersonal dynamics in improving productivity and software quality [7, 8]. **Pair programming may serve as a mechanism to strengthen communication among developers, distribute knowledge more effectively across a team, and reduce project risks associated with reliance on a small number of experts**. Investigating these potential human and organizational benefits provides further justification for systematically studying the costs and benefits of pair programming.

14.2 Effect in Different Aspects

14.2.1 A Project Experience

The paper begins by presenting a real project experience from an organization adopting pair programming during a complex and high-risk software integration effort. The project required developers to modify a large number of files while simultaneously merging multiple code branches and performing significant architectural changes. Because such work involved both tedious code modifications and deep reasoning about system design, the team anticipated a high likelihood of subtle defects emerging during integration. Developers believed that collaborative programming could reduce the risk of hidden errors, broaden the scope of code review, and allow knowledge to spread across the team rather than remaining isolated with individual programmers.

Initially, the team did not fully implement true pair programming. Instead, developers practiced what was described as partner programming: programmers worked individually and later reviewed their changes together before committing them. While this approach helped identify some issues earlier than usual, it still required considerable time because each programmer had to explain the reasoning behind their changes to their partner during the review phase. Over time, several developers began working side by side continuously, performing design and coding together rather than sequentially. As this practice spread across the team, the developers realized that collaborating directly during development avoided duplicating effort and allowed them to catch mistakes immediately.

As the project progressed, the team reported dramatic improvements in software quality and confidence in the system. The integrated system passed quality assurance testing with surprisingly few defects, especially compared to previous projects that had experienced long debugging cycles. Developers also noted that working together helped them discover better solutions and maintain a shared understanding of the codebase. Pair rotation across subsystems further allowed knowledge to spread throughout the team, ensuring that multiple developers understood different parts of the system. This experience suggested that pair programming could improve both the technical outcomes of development and the collaborative dynamics within a team.

14.2.2 Investigative Paths

To understand the broader implications of pair programming, the authors frame their investigation around several analytical perspectives, referred to as investigative paths. These perspectives examine pair programming from both technical and organizational viewpoints, reflecting the multifaceted nature of software development. Instead of focusing solely on productivity or development speed, the investigation considers factors such as economic impact, programmer satisfaction, design quality, learning,

communication, and project management outcomes. This approach recognizes that software engineering performance depends on both technical efficiency and human collaboration.

The first investigative perspective focuses on economics. Managers often assume that assigning two programmers to the same task will significantly increase development costs. However, empirical evidence suggests that the cost increase is relatively small compared to the potential savings resulting from improved code quality and reduced debugging effort. Earlier studies reported that collaborative programming only modestly increases development time while producing code with fewer defects [9]. When the downstream costs of defect detection, testing, and maintenance are considered, these improvements can lead to significant overall savings.

Other investigative paths focus on human and cognitive aspects of software development. Pair programming may improve problem solving, facilitate continuous code reviews, and create opportunities for learning through collaboration. Cognitive science research suggests that problem solving often benefits from distributed cognition, where multiple individuals contribute different knowledge and perspectives to a task [3]. Additional perspectives examine how pair programming strengthens communication within teams, improves knowledge sharing, and reduces project risks by ensuring that more than one person understands each part of the codebase. By analyzing pair programming through these multiple lenses, the authors aim to build a comprehensive understanding of both its benefits and its costs.

14.2.3 Economics

The economic viability of pair programming is a central concern because organizations must justify development practices in terms of cost and productivity. A common misconception is that collaborative programming doubles the cost of software development since two developers are assigned to the same task. To test this assumption, a controlled experiment conducted with advanced undergraduate students compared projects completed individually with those completed by pairs. The results showed that after an initial adjustment period, pairs required only about fifteen percent more development time than individuals working alone [9]. This finding challenges the widely held belief that collaborative programming dramatically increases development costs.

While development time increased slightly, the experiment also revealed that programs written by pairs contained significantly fewer defects. Reduced defect rates have important economic implications because fixing errors later in the software lifecycle can be extremely expensive. Research in software engineering has demonstrated that defects discovered during testing or after deployment require substantially more effort to diagnose and repair than defects identified during development [5]. Because pair programming provides continuous review during coding, many mistakes are detected immediately, reducing the number of defects that reach later stages of the development process.

The paper illustrates these economic implications using a hypothetical example involving a large codebase. If developers working individually create thousands of defects during development, even a small percentage reduction in defect rates can translate into substantial savings in debugging and maintenance costs. When testing teams must spend hours diagnosing each defect, the cumulative cost of error correction can greatly exceed the modest increase in development effort associated with pair programming. From this perspective, collaborative programming may actually reduce overall project costs despite slightly longer development time.

14.2.4 Satisfaction

Beyond economic considerations, programmer satisfaction is an important factor in evaluating the viability of pair programming. If developers find the practice unpleasant or stressful, they are unlikely to adopt it consistently in real projects. Many programmers initially express skepticism or resistance to working closely with another person during coding. Programming has traditionally been viewed as a solitary activity, and experienced developers may feel uncomfortable sharing control of the coding process or exposing their reasoning to constant scrutiny.

However, surveys conducted among both professional programmers and students revealed that most participants ultimately preferred collaborative programming once they became accustomed to it. Developers reported that working with a partner made the programming process more enjoyable and reduced the

anxiety associated with making mistakes. Continuous peer review provided reassurance that errors would be detected early, allowing programmers to focus more confidently on solving complex problems. These results were statistically significant across several groups of programmers, indicating that satisfaction with pair programming was not limited to a specific environment or level of experience.

Another interesting observation came from students who initially believed that pair programming required them to work harder than individuals coding alone. Even though collaborative assignments required pairs to complete more work per cycle, the students declined an opportunity to return to solo programming. Their response suggested that the perceived benefits of collaboration—such as shared problem solving, mutual support, and increased confidence in the correctness of their programs—outweighed the additional effort required. These findings indicate that pair programming can enhance not only software quality but also the overall work experience of developers.

14.2.5 Design Quality

The paper argues that pair programming can significantly improve the design quality of software systems. Evidence for this claim comes from both practitioner reports and empirical observations. In interviews with development teams, programmers noted that collaborative work often led to better architectural decisions and clearer designs. When two programmers work together, they frequently propose alternative approaches and challenge each other's assumptions. This interaction forces both developers to justify their reasoning and explore multiple potential solutions before committing to a design. Such discussions can result in hybrid solutions that combine the strengths of different ideas, ultimately producing more robust and elegant designs.

Empirical observations also support this claim. Studies of collaborative programming show that pairs tend to implement the same functionality using fewer lines of code than individuals working alone. Shorter code can indicate more efficient and well-structured designs, as unnecessary complexity and redundant logic are reduced. Cognitive science research provides an explanation for this phenomenon: collaborative problem solving allows programmers to search through a larger space of possible solutions because each participant contributes different experiences, knowledge, and perspectives [4]. By evaluating multiple alternatives together, pairs are more likely to avoid poor design choices and converge on higher-quality solutions.

14.2.6 Continuous Reviews

One of the most significant advantages of pair programming is the presence of continuous code review during development. Traditional software engineering practices often rely on formal inspections or review meetings to identify defects after code has been written. Although these inspections have been shown to be effective, they are frequently neglected in practice because developers find them tedious or disruptive to their workflow. Studies of software inspections demonstrate that systematic reviews can significantly reduce defects in software systems [10, 11]. However, because these reviews occur after coding, they may still allow many errors to propagate through the development process before being discovered.

Pair programming addresses this limitation by integrating code review directly into the act of development. As one programmer writes code, the partner continuously observes the implementation, checks the logic, and considers alternative approaches. This process enables errors to be identified immediately rather than after the code has been completed. Early detection is critical because the cost of fixing defects increases dramatically as software moves through later stages of development and deployment [6]. Continuous review also encourages adherence to coding standards and improves shared understanding of the codebase, making the development process both more efficient and more reliable.

14.2.7 Problem Solving

Pair programming also enhances developers' ability to solve difficult problems. When programmers work alone, they may become stuck when faced with complex bugs or design challenges, particularly if they lack the necessary perspective to recognize the source of the problem. Collaborative programming provides an environment where both participants can contribute ideas and alternate between leading and supporting roles during problem solving. Developers often describe this dynamic as a form of

relay thinking, where one programmer works on the problem until reaching an impasse and the other continues the reasoning process from a different perspective. This collaborative dynamic enables pairs to maintain momentum when tackling challenging tasks. By discussing their reasoning aloud and examining each other's assumptions, programmers can identify errors in logic more quickly than they might when working individually. The constant exchange of ideas also creates opportunities for spontaneous insights that may not occur during solitary work. Practitioners frequently report that collaborative problem solving allows them to overcome obstacles more efficiently, suggesting that the cognitive interaction between two developers provides advantages beyond simple brainstorming.

14.2.8 Learning

Pair programming creates a powerful environment for learning and skill development within software teams. When two programmers collaborate closely, they constantly exchange knowledge about tools, programming languages, and design practices. This exchange occurs naturally during the coding process as developers explain their reasoning, suggest improvements, and discuss alternative approaches. The learning that occurs in this environment is often informal and immediate, allowing developers to acquire new knowledge without structured training sessions or formal instruction.

The effectiveness of this learning process can be explained through the concept of apprenticeship and situated learning. In apprenticeship models, novices learn skills by observing and participating alongside experienced practitioners, gradually increasing their responsibilities over time [12]. Pair programming closely resembles this structure because developers operate within direct visual and conversational proximity, allowing novices to observe how experienced programmers reason about problems and structure solutions. This "line-of-sight" learning enables the transfer of tacit knowledge that is difficult to convey through documentation or lectures.

Empirical observations further support the educational value of pair programming. In classroom environments where collaborative programming was used exclusively, students reported that working with partners allowed them to learn new technologies more quickly and independently. Many participants indicated that between the two partners they could usually solve problems without needing assistance from instructors. This suggests that pair programming not only improves individual learning but also creates a shared problem-solving capacity that allows teams to operate more autonomously.

14.2.9 Team Building and Communication

Pair programming contributes significantly to team cohesion and communication within software development projects. In many development environments, programmers work independently and interact with teammates only during meetings or code reviews. This limited interaction can lead to fragmented communication, misunderstandings, and knowledge silos within the team. Collaborative programming, in contrast, requires constant dialogue between partners as they design and implement code together. Through these interactions, developers gradually learn to express their ideas clearly, listen to alternative viewpoints, and coordinate their work more effectively.

Over time, these repeated interactions strengthen interpersonal relationships among team members. Developers who initially struggle to communicate often become more comfortable sharing ideas and discussing technical challenges. As pairs rotate across tasks, information spreads throughout the team, increasing the overall flow of knowledge within the project. Research on teamwork in software development emphasizes that communication and collaboration are key determinants of project success [7, 8]. By increasing both the frequency and richness of communication, pair programming can significantly improve the effectiveness and cohesion of software teams.

14.2.10 Staff and Project Management

Pair programming also provides advantages from a project management perspective by improving both skill distribution and project resilience. As developers work together on different components of the system, knowledge about the codebase becomes shared rather than concentrated in the hands of individual programmers. This distribution of knowledge increases the collective competence of the team and reduces the likelihood that critical expertise will be limited to a single individual.

This benefit is closely related to what project managers sometimes describe as the "truck number," which refers to the number of people who would need to leave a project before it becomes severely disrupted. When only one developer understands a particular subsystem, the project becomes vulnerable if that person leaves the team. Pair programming mitigates this risk by ensuring that at least two developers are familiar with each part of the system. As knowledge spreads across the team through collaboration, the project becomes more robust and easier to maintain over time.

14.3 Key Findings

1. Pair programming enables mistakes to be detected immediately while code is being written rather than later during testing or after deployment. This continuous observation and correction helps reduce the number of defects introduced during development.
2. The overall defect rate in programs produced by pairs is lower than in programs produced by individual programmers. Early detection and correction of errors during development leads to cleaner and more reliable code.
3. Collaborative work often results in better software designs. Pairs tend to evaluate multiple approaches before implementing a solution, which leads to simpler and more efficient implementations.
4. Pair programming improves problem-solving capabilities. When developers encounter difficult issues, they can alternate in reasoning about the problem and contribute different insights, allowing them to resolve challenges more quickly.
5. Developers working in pairs learn more about the system, programming techniques, and tools through continuous interaction. Knowledge is shared naturally as partners explain ideas, question decisions, and observe each other's approaches.
6. Knowledge about the system becomes distributed among multiple developers rather than being concentrated with a single individual. This reduces the risk associated with losing key personnel during a project.
7. Pair programming improves communication and collaboration within teams. Regular interaction between developers encourages discussion, increases transparency, and strengthens team cohesion.
8. Many developers report that working in pairs makes programming more enjoyable. The shared responsibility and collaborative environment reduce stress and increase confidence in the correctness of the code.
9. Although pair programming requires slightly more development time than solo programming, the reduction in debugging effort, testing time, and maintenance costs can compensate for this additional effort.

14.4 Implications

Improved Software Quality Through Continuous Collaboration Pair programming demonstrates that integrating review and collaboration directly into the development process can significantly improve software quality. Instead of relying solely on formal inspections or post-development testing, defects can be identified and corrected immediately during coding. This suggests that development methodologies that embed quality assurance into everyday development practices may produce more reliable software systems.

Knowledge Sharing and Skill Development in Teams The collaborative nature of pair programming facilitates continuous learning among developers. When programmers work together, they exchange knowledge about programming techniques, system architecture, and development tools. This process can accelerate skill development, particularly for less experienced developers, and ensures that expertise is distributed across the team rather than isolated within a few individuals.

Reduced Project Risk Through Shared Ownership of Code Pair programming encourages shared understanding of the codebase because multiple developers become familiar with each part of the system. This reduces the risks associated with relying on a single developer for critical components of the project. Teams that adopt collaborative development practices may therefore become more resilient to personnel changes and better equipped to maintain and evolve complex systems over time.

References

[1] Kent Beck. *Extreme Programming Explained: Embrace Change*. Addison-Wesley, 2000.

[2] John T. Nosek. "The Case for Collaborative Programming". In: *Communications of the ACM* 41.3 (1998), pp. 105–108.

[3] Gavriel Salomon. *Distributed Cognitions: Psychological and Educational Considerations*. Cambridge University Press, 1993.

[4] Nick V. Flor and Edwin L. Hutchins. "Analyzing Distributed Cognition in Software Teams: A Case Study of Team Programming During Perfective Software Maintenance". In: *Empirical Studies of Programmers*. Ablex Publishing, 1991.

[5] Watts S. Humphrey. *A Discipline for Software Engineering*. Addison-Wesley, 1995.

[6] Watts S. Humphrey. *Introduction to the Personal Software Process*. Addison-Wesley, 1997.

[7] Gerald M. Weinberg. *The Psychology of Computer Programming*. Dorset House, 1998.

[8] Tom DeMarco and Timothy Lister. *Peopleware: Productive Projects and Teams*. Dorset House, 1999.

[9] Laurie Williams et al. "Strengthening the Case for Pair Programming". In: *IEEE Software*. 2000.

[10] Michael E. Fagan. "Advances in Software Inspections to Reduce Errors in Program Development". In: *IBM Systems Journal* 15.3 (1976), pp. 182–211.

[11] Philip M. Johnson. "Reengineering Inspection: The Future of Formal Technical Review". In: *Communications of the ACM* 41.2 (1998), pp. 49–52.

[12] Jean Lave and Etienne Wenger. *Situated Learning: Legitimate Peripheral Participation*. Cambridge University Press, 1991.

Chapter 15

[Skim] Improving Communication Between Pair Programmers Using Shared Gaze Awareness

Remote collaboration has become increasingly common in software development, yet it often lacks the rich non-verbal cues that support effective communication in collocated settings. In pair programming, where two developers must continuously coordinate their attention and reasoning, the inability to see what a partner is focusing on creates challenges in referencing and understanding code locations. Existing remote tools such as screen sharing or distributed IDEs provide limited support for such coordination, often forcing developers to rely on explicit and sometimes inefficient communication strategies like verbalizing line numbers or function names. This gap highlights the need for mechanisms that can restore shared awareness and improve interaction quality in remote pair programming environments.

To address this limitation, the paper introduces a shared gaze awareness system that visualizes where each programmer is looking within the code. By leveraging eye-tracking technology, the system provides an unobtrusive visual cue that helps collaborators align their attention in real time. The study demonstrates that such visualization improves coordination by increasing the amount of time pairs focus on the same code regions, enabling more efficient and implicit communication. As a result, developers can respond more quickly and accurately to references, reducing the need for overly explicit descriptions. This work suggests that incorporating gaze-based awareness into remote collaboration tools can significantly enhance communication and make remote pair programming more effective and natural.

15.1 Motivation

Remote work has become increasingly prevalent due to its flexibility and ability to support diverse lifestyles, yet it lacks critical elements of face-to-face collaboration such as non-verbal cues and gestures. In collaborative programming tasks, especially pair programming, these cues play a crucial role in establishing shared understanding and coordinating attention. Without the ability to naturally perceive where a partner is looking, remote collaborators face difficulties in referencing specific locations in code and maintaining synchronized workflows. Prior research highlights that eye gaze is tightly coupled with communication, where speakers and listeners use gaze to establish common ground and disambiguate references [1, 2]. The absence of such cues in remote environments motivates the need for mechanisms that can restore this lost dimension of interaction.

Pair programming, a widely adopted collaborative practice, relies heavily on continuous coordination, shared attention, and effective communication between partners. It has been shown to improve code quality, focus, and developer satisfaction [3, 4]. However, its effectiveness is often tied to physical co-presence, making it less common in remote settings. Survey results reported in the paper indicate that while developers value pair programming, very few engage in it remotely due to limitations in current

tools. General-purpose tools like video conferencing and screen sharing fail to provide adequate support for non-verbal communication, while specialized tools offer only limited interaction capabilities. This disconnect between the benefits of pair programming and the constraints of remote work environments motivates the exploration of new interaction techniques.

To bridge this gap, prior work in collaborative systems has explored the role of shared gaze awareness as a means of enhancing coordination and communication. Studies have shown that sharing gaze information can improve task performance, increase joint attention, and enable more efficient use of implicit references such as deictic expressions [5, 6]. These findings suggest that gaze can serve as a powerful referential cue in collaborative tasks. Building on this foundation, the paper is motivated to investigate how shared gaze awareness can be applied specifically to remote pair programming, with the goal of improving communication efficiency, reducing ambiguity, and ultimately making remote collaboration as effective as its collocated counterpart.

15.2 Methodology

Overview The study employs a controlled experimental design to evaluate the effectiveness of a shared gaze awareness visualization in remote pair programming. The primary objective is to understand how displaying a collaborator's gaze influences coordination, communication, and task performance. The experiment follows a within-subjects design, where each pair of participants completes programming tasks under two conditions: with and without gaze awareness. This allows for direct comparison of behaviors across conditions while minimizing variability due to participant differences.

Study Design A 2×2 counterbalanced within-subjects design was used, incorporating two independent variables: gaze awareness (enabled vs. disabled) and task type. Each participant pair performed two refactoring tasks, experiencing both experimental conditions in different orders to control for learning effects. The tasks were designed to be equivalent in difficulty and duration, ensuring that differences in outcomes could be attributed to the presence of gaze visualization rather than task complexity.

Participants The study involved 24 professional software developers who were familiar with programming in C#. Participants were recruited from a large technology company and compensated for their time. They were paired and asked to collaborate in a simulated remote environment. All participants provided consent and had prior experience relevant to the programming tasks used in the experiment.

Experimental Setup To simulate remote pair programming, the researchers created a controlled environment using a single computer with dual monitors placed back-to-back, preventing participants from seeing each other while allowing verbal communication. Each participant had their own keyboard, mouse, and display, along with an eye-tracking device.

A custom extension for the development environment synchronized both participants' views of the code, ensuring that scrolling, highlighting, and navigation actions were mirrored. The system also captured gaze data from both users and displayed a visualization indicating where each partner was looking. This setup allowed the researchers to isolate the effect of gaze awareness while maintaining a realistic collaborative workflow.

Tasks Participants first completed a short familiarization task, making minor changes to a codebase to understand its structure. This task was not included in the analysis.

They then performed two refactoring tasks on a C# codebase resembling a simple game. Each task lasted approximately 15 minutes and required collaboration to improve code quality and structure. One task was completed with gaze visualization enabled, and the other without it.

Gaze Visualization Intervention The experimental condition introduced a shared gaze visualization that displayed a colored rectangular indicator representing the partner's focus area in the code. The visualization was designed to be subtle and non-intrusive, highlighting a small vertical region in the code

editor. It also changed color when both participants were looking at the same region, indicating shared attention.

The control condition removed this visualization while keeping all other aspects of the setup identical.

Data Collection Multiple forms of data were collected during the experiment:

- Eye-tracking logs capturing gaze positions and overlap between participants
- Screen recordings and video recordings of participants' interactions
- Interaction logs tracking navigation and editing behavior
- Post-task questionnaires measuring participants' perceptions of collaboration
- Interviews conducted after the tasks to gather qualitative feedback

Communication Analysis The recorded sessions were manually analyzed to identify references and acknowledgments during communication. Each reference to a code location was categorized based on how explicitly it was expressed, ranging from implicit (e.g., "this" or "here") to explicit (e.g., line numbers or specific function names).

Acknowledgments were classified as either successful or unsuccessful depending on whether the partner correctly understood the reference. This analysis enabled the researchers to study how gaze awareness influenced communication style and effectiveness.

Gaze Analysis A key metric in the study was gaze overlap, defined as the extent to which both participants were looking at the same region of code simultaneously. Each participant's focus area was approximated as a small vertical window centered on their gaze position. Overlap was recorded when these regions intersected for a minimum duration to avoid noise from rapid eye movements.

Additionally, gaze overlap was used to infer implicit acknowledgments of references when no verbal confirmation was given, providing insight into non-verbal coordination.

Evaluation Metrics The study evaluated the impact of gaze awareness using several quantitative and qualitative measures:

- Fraction of time spent in shared gaze overlap
- Distribution of reference types (implicit vs. explicit)
- Success rate of communication references
- Time taken to acknowledge references
- Participant perceptions from questionnaires and interviews

These metrics collectively assess how gaze awareness affects coordination, communication efficiency, and overall collaboration quality in remote pair programming.

15.3 Results

The results evaluate how shared gaze awareness affects coordination, communication, and efficiency in remote pair programming. The analysis focuses on three primary aspects: the extent of shared attention, the nature of communication between participants, and the effectiveness and speed of reference resolution. The findings consistently indicate that gaze awareness improves coordination and enables more efficient interaction, even though it does not significantly impact task completion time.

Gaze Overlap and Shared Attention The introduction of gaze visualization significantly increased the amount of time participants spent looking at the same region of code. Pairs using gaze awareness showed nearly double the proportion of shared gaze compared to when it was absent. This indicates stronger synchronization of attention between collaborators. Increased gaze overlap suggests that participants were better aligned in their understanding of the task and were more effectively coordinating their actions in real time.

Communication Patterns The presence of gaze awareness altered how participants referred to locations in the code. Specifically, there was a substantial increase in the use of implicit (deictic) references, such as "this" or "here," when gaze visualization was enabled. At the same time, the use of more explicit references, such as naming functions or specifying line numbers, decreased. This shift demonstrates that participants relied on gaze as a shared contextual cue, reducing the need for detailed verbal explanations. The visualization effectively acted as a referential pointer, allowing collaborators to communicate more naturally and with less effort.

Reference Success Rate The effectiveness of communication improved with gaze awareness. A higher proportion of references were successfully understood and acknowledged when the visualization was present. Notably, implicit references—typically more ambiguous—became significantly more reliable. This indicates that gaze awareness not only changes how participants communicate but also enhances the clarity of that communication. By providing a shared visual context, the system reduces ambiguity and increases the likelihood that references are interpreted correctly.

Speed of Acknowledgment Participants were faster at responding to references when gaze visualization was enabled. On average, references were acknowledged more quickly compared to the condition without gaze awareness. Interestingly, while implicit references did not show a significant improvement in speed individually, more explicit references (such as line numbers or specific names) were acknowledged faster. This suggests that gaze awareness enhances overall responsiveness, even when traditional explicit communication is used.

Task Completion Time Despite improvements in coordination and communication, there was no significant difference in task completion time between the two conditions. All participant pairs used the full allotted time for the tasks regardless of whether gaze awareness was available. This outcome is attributed to participants' tendency to continue refining and improving the code beyond the minimum requirements, reflecting professional standards rather than inefficiencies in collaboration.

Participant Perceptions Post-task questionnaire results showed a slight improvement in participants' perception of their partner's focus when gaze awareness was enabled. Participants felt that their collaborators were more attentive and aligned with the task.However, most other subjective measures of collaboration quality did not show significant differences. This suggests that while the benefits of gaze awareness are measurable in behavior, they may be subtle and not immediately perceived by users.

Qualitative Insights Interviews with participants revealed that gaze visualization was particularly useful for understanding where a partner's attention was directed. Many participants described it as a substitute for pointing gestures, helping them quickly establish shared context. Some participants also noted that the color change indicating shared focus reinforced their sense of being "on the same page." Interestingly, several participants reported that they were not consciously aware of using the visualization, even though their behavior reflected its influence. This highlights the subtle and unobtrusive nature of the design.

15.4 Implications

The findings suggest that shared gaze awareness can significantly enhance coordination in remote pair programming by restoring an important non-verbal communication channel that is typically lost outside collocated settings. By enabling developers to see where their partner is looking, the system supports faster grounding of shared context and reduces ambiguity in communication. This allows collaborators to

rely more on natural, implicit forms of interaction rather than verbose and explicit descriptions, making remote collaboration feel more fluid and intuitive. As a result, gaze-based cues can act as a lightweight yet powerful mechanism to improve communication efficiency without disrupting existing workflows.

Beyond pair programming, the study highlights broader design implications for collaborative systems. The effectiveness of a subtle and non-intrusive visualization suggests that awareness tools should be carefully integrated to complement, rather than distract from, the primary task. The approach can be extended to other collaborative domains such as document editing, learning environments, and distributed teamwork, where shared attention is critical. Additionally, the results emphasize the importance of designing interaction techniques that align with task structure and user behavior, indicating that gaze awareness systems should be adaptable and context-sensitive to maximize their utility.

15.5 Threats to Validity

One key limitation lies in the accuracy and reliability of eye-tracking technology. Eye trackers are inherently imprecise and can be affected by user movement, calibration issues, and hardware differences. Although the study accounted for this by designing a broader visualization region, inaccuracies in gaze data could still influence the results. Additionally, the use of two different eye-tracking devices introduces potential inconsistencies in measurement, which may affect the comparability of gaze data across participants.

Another threat concerns the experimental setup and generalizability of results. The study simulated remote collaboration using a controlled environment with no communication latency and synchronized displays, which differs from real-world remote settings that often involve network delays and varying system configurations. Furthermore, the experiment focused on a mirrored-view setup rather than fully distributed workflows where collaborators may work on different parts of the codebase. These factors limit the external validity of the findings and suggest that further studies are needed to evaluate the approach in more realistic and diverse environments.

References

[1] Zenzi M. Griffin and Kathryn Bock. “What the eyes say about speaking”. In: *Psychological Science* 11.4 (2000), pp. 274–279.

[2] Joy E. Hanna and Susan E. Brennan. “Speakers’ eye gaze disambiguates referring expressions early during face-to-face conversation”. In: *Journal of Memory and Language* 57.4 (2007), pp. 596–615.

[3] Jo E. Hannay et al. “The effectiveness of pair programming: A meta-analysis”. In: *Information and Software Technology* 51.7 (2009), pp. 1110–1122.

[4] Andrew Begel and Nachiappan Nagappan. “Pair Programming: What’s in It for Me?” In: *ESEM*. 2008, pp. 120–128.

[5] Susan E. Brennan et al. “Coordinating cognition: The costs and benefits of shared gaze during collaborative search”. In: *Cognition* 106.3 (2008), pp. 1465–1477.

[6] Bertrand Schneider and Roy Pea. “Real-time mutual gaze perception enhances collaborative learning and collaboration quality”. In: *International Journal of Computer-Supported Collaborative Learning* 8.4 (2013), pp. 375–397.

Chapter 16

[Required] Expectations, Outcomes, and Challenges of Modern Code Review

This paper investigates how modern, tool-based code review is actually practiced in industry, focusing on developers at Microsoft. It examines the gap between what developers and managers expect from code reviews—primarily defect detection—and what reviews actually achieve in practice.

Using a mixed-method empirical approach that includes observations, interviews, comment analysis, and large-scale surveys, the authors find that modern code reviews provide value beyond bug detection. While finding defects remains the main motivation, most review activity centers on improving code quality, sharing knowledge, increasing team awareness, and generating alternative solutions. A key insight is that understanding code changes and their context is the central challenge in reviews, and current tools often fail to adequately support this need.

16.1 Motivation

The motivation for studying modern code review arises from its long-standing role as a key quality assurance practice in software engineering. **Traditional code inspections, formalized by Fagan [1], were designed to systematically detect defects through structured, synchronous meetings, and have been shown to be effective in improving software quality [2].** However, these approaches are often time-consuming and cumbersome, limiting their adoption in fast-paced industrial environments [3]. As a result, organizations have shifted toward more lightweight and tool-supported review practices.

This shift toward modern code review introduces uncertainty about whether prior knowledge from formal inspections still applies. Contemporary practices are informal, asynchronous, and heavily supported by tools, which fundamentally changes how developers interact with code and each other during reviews [4]. **While these newer approaches improve efficiency and scalability, it is unclear whether they achieve the same goals as traditional inspections, particularly in terms of defect detection and overall code quality**.

Additionally, modern code review is increasingly embedded in large-scale, collaborative development environments, where it serves not only as a defect detection mechanism but also as a medium for communication and coordination. Prior research suggests that review processes can capture valuable design rationale and support knowledge sharing among developers [5], and that developers often struggle with understanding code changes and team knowledge distribution [6]. These aspects highlight the broader socio-technical role of code review beyond simple error detection.

Given these evolving practices and expectations, there is a need for an empirical investigation into what

developers and managers actually expect from code review, what outcomes are realized in practice, and what challenges arise. Understanding these factors can help practitioners better integrate code review into their workflows and guide researchers in improving tools and methodologies to better support modern development environments.

16.1.1 Related Work

Prior work on code review has largely focused on traditional code inspections and early forms of distributed review. For instance, studies on distributed and asynchronous inspections explored how tools could support geographically separated reviewers in identifying and discussing defects [7]. Surveys of inspection techniques have also provided taxonomies and comparative analyses of different approaches, highlighting factors such as team size, inspection type, and session structure in determining effectiveness [8, 9]. Additionally, research examining open source development has shown how peer review influences project management decisions and collaborative workflows [10]. These studies, however, primarily examined formal or semi-formal inspection processes rather than modern, lightweight practices.

A key limitation of traditional inspection methods identified in prior research is their inefficiency, particularly due to the need for synchronous meetings and scheduling overhead. For example, it has been shown that a significant portion of inspection time can be wasted due to coordination delays [11]. This inefficiency motivated the development of early tool-based solutions such as ICICLE, CAIS, and Scrutiny, which aimed to enable asynchronous and more efficient review processes [12, 13, 14]. These systems laid the groundwork for modern code review tools by emphasizing collaboration, reduced overhead, and support for distributed teams.

More recent research has shifted toward understanding lightweight and tool-supported review practices, particularly in open source environments. Empirical studies of projects such as Apache have shown that modern reviews tend to be small, frequent, and involve brief, independent contributions from multiple reviewers [15]. Furthermore, research has highlighted that code review discussions often capture valuable design rationale, making them useful for future maintenance and comprehension tasks [5]. Complementary studies on developer behavior indicate that many challenges faced by developers stem from difficulties in understanding code changes and accessing team knowledge, reinforcing the importance of review processes in supporting comprehension [6].

Overall, while prior literature provides substantial insights into both traditional inspections and emerging lightweight practices, it does not fully address how modern, tool-based code review operates in large-scale industrial settings. In particular, there remains a gap in understanding how motivations, outcomes, and challenges of contemporary code review compare to earlier expectations, motivating further empirical investigation.

16.2 Methodology

16.2.1 Research Questions

The study is guided by three primary research questions that aim to explore the motivations, outcomes, and challenges of modern code review in practice:

1. What are the motivations and expectations for modern code review? Do they change from managers to developers and testers?
2. What are the actual outcomes of modern code review? Do they match the expectations?
3. What are the main challenges experienced when performing modern code reviews relative to the expectations and outcomes?

16.2.2 Research Setting

The study is conducted within a large industrial context at Microsoft, where software development spans diverse domains such as enterprise systems, mobile applications, and search engines. This diversity allows the authors to observe code review practices across teams with varying cultures, workflows, and

policies, making the setting suitable for understanding modern code review in realistic, heterogeneous environments. The focus is on teams that have widely adopted a common tool for code review, CodeFlow, which is used by tens of thousands of developers. This tool represents a typical modern code review system, similar in functionality to other widely used platforms such as Mondrian, Phabricator, and Gerrit, enabling asynchronous, tool-based, and collaborative review processes.

CodeFlow structures the review process by allowing authors to submit a package of code changes along with a textual description, select reviewers, and initiate a discussion around the changes. Reviewers can inspect diffs, annotate code inline, and engage in threaded discussions that may occur synchronously or asynchronously. The system centralizes all review-related data, including comments and interactions, which provides a rich and unobtrusive source of empirical data for analysis without interfering with developer behavior. This environment allows the researchers to study code reviews "in situ," capturing both the social and technical aspects of the practice as it naturally occurs in industry.

16.2.3 Research Method

The study adopts a mixed-methods approach, combining qualitative and quantitative techniques to triangulate findings and improve validity [16]. The methodology consists of multiple phases: (1) analysis of a prior internal study, (2) observations and interviews with developers, (3) card sorting of interview data, (4) card sorting of code review comments, (5) construction of an affinity diagram, and (6) large-scale surveys of managers and developers.

Analysis of Previous Research The first phase involves analyzing transcripts from an earlier study conducted at Microsoft, focusing on responses to questions about the goals of code review. Using coding techniques inspired by grounded theory, responses are broken into smaller units and grouped into concepts and categories [17, 18]. This initial analysis helps identify preliminary motivations for code review, which are later refined through subsequent phases.

Interviews, while conducting Code Reviews The second phase consists of direct observations and semi-structured interviews with developers actively performing code reviews. Participants are selected across multiple teams and experience levels to ensure diversity. During each session, researchers first observe developers conducting real reviews, encouraging them to think aloud, and then follow up with interviews guided by evolving research questions. Semi-structured interviews are chosen to allow flexibility and exploration of emerging themes [19, 20]. The collected data is transcribed and segmented into meaningful units for further analysis.

Card Sorting and Affinity Diagram To organize and abstract the qualitative data, the researchers employ open card sorting, a technique commonly used in information architecture to derive themes and hierarchies from unstructured data [21]. This process is applied both to interview data (over a thousand coded units) and to a sampled set of 570 code review comments, enabling the identification of recurring patterns and categories. The resulting categories are then synthesized using an affinity diagram, which groups related concepts and helps derive higher-level themes through collaborative analysis [22].

Finally, the study validates and generalizes its findings through two surveys: one targeting managers and another targeting developers. The surveys are designed following established guidelines for opinion-based empirical studies [23]. The manager survey gathers qualitative insights into motivations for code review, while the developer survey uses structured questions to quantify the importance of different motivations and experiences. High response rates from both groups strengthen the reliability of the results. By combining these diverse data sources, the methodology ensures a comprehensive and well-supported understanding of modern code review practices.

16.3 Key Findings

The paper aims to answer three research questions regarding the motivations, outcomes, and challenges of modern code review. The key findings are organized around these questions, revealing insights into the complex role that code review plays in contemporary software development.

16.3.1 Why do Programmers do Code Review?

Finding Defects Finding defects emerges as the primary and most widely agreed-upon motivation for code review among both developers and managers. Many practitioners view code review as an efficient way to catch bugs early, sometimes even considering it cheaper than testing for certain issues. This includes both low-level logical errors and higher-level design flaws. However, while this motivation is dominant in perception, it does not fully capture the breadth of what code reviews accomplish in practice. Developers often approach reviews expecting to identify flaws in logic or correctness, but this expectation represents only one dimension of the activity.

Code Improvement Code improvement is a closely ranked motivation, focusing on enhancing readability, maintainability, and adherence to coding standards rather than correctness. Reviewers frequently comment on formatting, naming conventions, and removal of redundant or unused code. This type of feedback is often the first pass during review and is considered essential for maintaining consistency across a team. However, the paper highlights a subtle concern: because these improvements are easier to identify, reviewers may focus disproportionately on them, sometimes overlooking deeper issues such as architectural flaws or security concerns.

Alternative Solutions This motivation involves suggesting better or more efficient ways to implement a solution. Developers value the opportunity to receive feedback that leads to improved designs or alternative approaches, reflecting the collective intelligence of the team. However, there is a notable disconnect between developers and managers on this aspect—developers consider it moderately important, while managers rarely emphasize it. In practice, suggestions for alternative solutions tend to be less frequent and often remain at a high level rather than deeply explored.

Knowledge Transfer Code review serves as a significant mechanism for learning and knowledge sharing within teams. Both reviewers and authors benefit from exposure to different parts of the codebase, APIs, and design decisions. This bidirectional learning helps distribute expertise, onboard new developers, and reinforce best practices. Interestingly, this motivation often emerges organically rather than being explicitly planned, indicating that developers perceive code review as an educational process embedded within their workflow.

Team Awareness and Transparency Another important motivation is maintaining awareness of ongoing changes within the team. Code reviews act as a communication channel that keeps team members informed about new features, modifications, and design decisions. This transparency helps prevent unexpected changes, fosters alignment, and enables developers to stay updated on relevant parts of the system. It also allows team members to discover new functionalities that they can reuse or integrate into their own work.

Share Code Ownership Closely related to awareness, shared code ownership emphasizes collaboration and reduces reliance on individual experts. Through code review, multiple developers become familiar with different parts of the codebase, mitigating knowledge silos and increasing system resilience. This also has a cultural impact, making developers less protective of their code and more open to feedback. Over time, this fosters a sense of collective responsibility and encourages a more collaborative development environment.

Summary Overall, while defect detection is the most prominent stated motivation, code review fulfills a much broader role in practice. It supports code quality, learning, communication, and collaboration within teams. The motivations span both technical and social dimensions, highlighting that modern code review is not just a verification activity but a central component of team coordination and knowledge sharing.

16.3.2 The Outcomes of Code Review

The paper presents a detailed empirical analysis of what actually happens during modern code review, contrasting sharply with earlier expectations. By examining 570 review comments across multiple

threads, the authors categorize and quantify the types of feedback given during reviews. The findings reveal that the most common outcome of code review is not defect detection, but rather code improvement. Nearly one-third of all comments focus on improving code practices, readability, and removing unnecessary or redundant code. These types of changes are relatively easy to identify and act upon, which likely contributes to their high frequency. This establishes code review as a mechanism primarily used for refining and polishing code rather than deeply analyzing correctness.

In contrast, defect detection—despite being the most frequently cited motivation—accounts for a significantly smaller portion of review outcomes. Only a small fraction of comments are related to defects, and even within this category, most issues are minor and localized. These include logical mistakes, simple configuration errors, or overlooked corner cases. High-level issues such as architectural flaws, security vulnerabilities, or complex design problems are rarely identified during reviews. This indicates that while developers expect code review to uncover meaningful defects, in practice it tends to surface only superficial problems.

The authors explore the reasons behind this mismatch between expectations and outcomes. One key explanation lies in the nature of modern, tool-based reviews. Reviewers often prioritize quick, visible issues—such as formatting or small logic errors—over deeper analysis, partly because these are easier to detect and require less contextual understanding. Interviews suggest that reviewers sometimes default to commenting on minor issues when they lack sufficient familiarity with the code. Additionally, some reviewers acknowledge that focusing on easy improvements can mask the presence of more significant problems, which may go unnoticed.

Another important observation is that the ability to detect meaningful defects is closely tied to the reviewer's level of understanding of the code and its context. When reviewers lack familiarity with the codebase or the purpose of the changes, their feedback tends to remain shallow. This reinforces the idea that deeper insights require not only time but also contextual knowledge, which is often limited in modern, distributed review environments. As a result, the depth and quality of review outcomes vary significantly depending on the reviewer's expertise and involvement with the code.

Although less prominent, the study also finds evidence of knowledge transfer occurring during reviews. Some comments direct authors to documentation or suggest learning resources, indicating that reviews occasionally serve as opportunities for sharing expertise. However, such outcomes are relatively rare in the recorded data, likely because many learning interactions happen outside the review tool, such as through direct communication.

Overall, the paper concludes that modern code review does not fully meet its primary expectation of defect detection. Instead, it functions more effectively as a tool for incremental code improvement and team-level knowledge exchange. The discrepancy between expected and actual outcomes is not due to flawed practices but rather reflects the inherent limitations of lightweight, tool-based reviews. These findings highlight the need to reconsider the role of code review within the broader development process and suggest that it should be complemented with other techniques to ensure comprehensive defect detection.

16.3.3 Challenges of Code Review

Code Review is Understanding The central challenge identified in this section is that code review is fundamentally an exercise in understanding. Across interviews and observations, developers consistently report that the most difficult aspect of reviewing code is grasping the rationale behind changes—why the change was made, what problem it solves, and how it fits into the broader system. Even when descriptions are provided, they are often insufficient or incomplete, leaving reviewers to infer intent from the code itself. This lack of clarity makes it difficult to provide meaningful feedback. As a result, much of the review effort is spent not on evaluating correctness, but on reconstructing context. This explains why understanding-related comments (questions, clarifications) are highly prevalent and why deeper issues are often missed—without sufficient understanding, reviewers cannot effectively assess design or correctness.

The authors also highlight that understanding is the dominant factor influencing the time and quality of reviews. Other challenges such as scheduling or time pressure are often secondary and can be traced

back to the effort required to comprehend the code. Developers indicate that understanding consumes the majority of review time, reinforcing that improving comprehension is key to improving review effectiveness. This insight reframes code review from a verification task into a cognitive task centered on information processing and context-building.

A Priori Understanding The authors emphasize the importance of prior familiarity with the codebase in determining review effectiveness. Developers who are already familiar with the files under review—such as code owners or contributors to that component—are able to review faster and provide more insightful feedback. Their comments tend to be deeper, more conceptual, and more actionable, often identifying subtle issues or suggesting better design alternatives. In contrast, reviewers unfamiliar with the code must invest additional time to understand its purpose, dependencies, and constraints, which often limits their feedback to surface-level observations.

The findings show that unfamiliarity significantly increases the effort required for review. Most developers report that reviewing unfamiliar code requires exploring additional context beyond the changes themselves, such as reading related files or understanding system behavior. Even then, their understanding remains shallow compared to that of experienced contributors. This results in feedback that is less confident, less detailed, and focused on easily observable aspects like style or syntax. The subsection clearly establishes that prior knowledge is a critical enabler of effective code review.

Dealing with Understanding Needs Given the challenges of understanding, developers adopt various strategies to gather the necessary context. These include reading change descriptions, exploring related code, executing the code, consulting documentation, and directly communicating with the author or other team members. In many cases, reviewers resort to synchronous communication—such as face-to-face discussions or real-time messaging—to clarify doubts more efficiently. This highlights a limitation of current code review tools, which primarily support asynchronous, text-based interactions and offer limited assistance for deeper comprehension.

The authors underscore that existing tools provide only basic features such as diff views, inline comments, and syntax highlighting, which are insufficient for supporting complex understanding tasks. As a result, reviewers must rely on external resources and communication channels to fill the gaps. This fragmented approach to understanding reduces efficiency and may lead to incomplete or superficial reviews. The findings suggest that improving tool support for context and comprehension could significantly enhance the effectiveness of code review, enabling reviewers to move beyond surface-level feedback and address more complex issues.

16.4 Implications

The paper concludes by translating its empirical findings into a set of actionable recommendations for practitioners and broader implications for researchers. These are grounded in the observed mismatch between expectations and outcomes of code review, and the central role of understanding in shaping review effectiveness.

16.4.1 Recommendations for Practitioners

16.4.1.1 Rethinking Code Review as a Quality Assurance Mechanism

One of the primary recommendations is that teams should reconsider relying on code review as a strong defect detection mechanism. The study shows that reviews tend to uncover only superficial issues rather than deep or critical defects. As a result, treating code review as a primary line of defense for quality assurance can be misleading. Instead, teams should complement code reviews with other practices such as automated testing, static analysis, and formal verification techniques to ensure comprehensive defect detection.

16.4.1.2 Improving Reviewer Understanding

Since understanding is identified as the key challenge in code review, improving the reviewer's context is critical. The paper recommends that teams ensure reviewers have sufficient familiarity with the code being reviewed. This can be achieved by involving code owners or developers with relevant expertise in the review process. Additionally, authors should provide clear, detailed descriptions of changes, including the rationale and expected impact, to reduce the cognitive burden on reviewers. Enhancing understanding directly improves the depth and usefulness of feedback.

16.4.1.3 Recognizing Benefits Beyond Defect Detection

The study emphasizes that code review provides multiple benefits beyond finding bugs, such as improving code quality, facilitating knowledge transfer, increasing team awareness, and encouraging shared ownership. Teams should explicitly acknowledge and leverage these broader benefits when designing their review processes. For instance, reviews can be structured to encourage discussion, learning, and collaboration rather than focusing solely on correctness.

16.4.1.4 Enhancing Communication During Reviews

Another key recommendation is to improve communication mechanisms within the review process. While modern tools support asynchronous interactions, they often fall short in enabling rich, high-bandwidth communication. Developers frequently resort to in-person discussions or synchronous messaging to clarify complex issues. Teams should therefore incorporate opportunities for direct communication—either through integrated tools or organizational practices—to support better understanding and more effective reviews.

16.4.2 Implications for Researchers

16.4.2.1 Opportunities for Automating Routine Review Tasks

The findings suggest that a significant portion of review effort is spent on identifying code improvements and simple defects, which are relatively straightforward and repetitive. This presents an opportunity for research in automated tools that can handle such tasks, such as enforcing coding standards, detecting common bugs, or identifying dead code. By automating these low-level checks, developers can focus their attention on more complex and meaningful aspects of code review, such as design and architecture.

16.4.2.2 Advancing Program Comprehension Support

A major implication is the need for better tool support for code understanding. Despite extensive research in program comprehension and the availability of advanced features in modern IDEs, current code review tools provide only minimal support for understanding code changes. This gap highlights a significant opportunity for researchers to design tools that integrate contextual information, dependency analysis, and historical insights directly into the review process. Improving comprehension support could directly address the core challenge identified in the study.

16.4.2.3 Exploring Socio-Technical Aspects of Code Review

The study reveals that code review has important social and collaborative dimensions, such as knowledge sharing, team awareness, and cultural shifts toward shared ownership. However, these aspects are difficult to measure and remain underexplored. This opens up avenues for research into the socio-technical effects of code review, including how it influences team dynamics, learning, and long-term productivity. Developing methods to capture and evaluate these outcomes could provide a more holistic understanding of the value of code review.

16.4.2.4 Bridging the Gap Between Expectations and Practice

Finally, the mismatch between expected and actual outcomes of code review highlights a broader research challenge: aligning tools and practices with real-world needs. Researchers are encouraged to focus on practical problems faced by developers, particularly those related to understanding and collaboration.

By addressing these gaps, future work can help evolve code review into a more effective and impactful practice in modern software development.

Overall, the recommendations and implications underscore a shift in perspective—from viewing code review as a defect-finding activity to recognizing it as a multifaceted process that combines technical evaluation with communication, learning, and collaboration.

16.5 Limitations

The paper acknowledges that, as a qualitative study, there are inherent challenges in establishing the validity and generalizability of its findings. Although the authors attempt to mitigate this through triangulation—combining observations, interviews, surveys, and analysis of review comments—the interpretation of qualitative data can still introduce subjectivity. The study relies on coding, categorization, and thematic analysis, which depend on researcher judgment, and therefore may be influenced by bias despite efforts to ensure consistency and agreement among researchers [24].

Another limitation concerns the context of the study, which is conducted entirely within Microsoft. While the company provides a diverse set of teams, projects, and development practices, findings from a single organization may not generalize to other industrial or open-source environments. Differences in team dynamics, project size, or development culture could lead to different code review behaviors. However, the authors argue that insights from single-case studies can still contribute meaningfully to scientific understanding, as supported by prior research in case study methodology [25, 26].

The study also highlights limitations related to the observability of certain outcomes. Some motivations and benefits of code review—such as knowledge transfer, team awareness, and learning—are inherently difficult to measure through artifacts like review comments. While survey responses and interviews suggest that these outcomes exist, there is limited "hard evidence" to quantify their frequency or impact. This means that some conclusions rely on self-reported perceptions rather than directly observable data.

Finally, the analysis of outcomes is based largely on review comments recorded in the CodeFlow tool, which may not capture the full extent of interactions during code review. Developers often engage in offline or synchronous communication, such as face-to-face discussions or real-time messaging, especially when addressing complex understanding needs. These interactions are not fully represented in the dataset, potentially leading to an incomplete picture of the review process and its outcomes.

References

[1] Michael E. Fagan. "Design and code inspections to reduce errors in program development". In: *IBM Systems Journal* (1976).

[2] A. Frank Ackerman, Priscilla J. Fowler, and Robert G. Ebenau. "Software inspections and the industrial production of software". In: *Proceedings of a symposium on Software validation: inspection-testing-verification-alternatives*. 1984, pp. 13–40.

[3] Forrest Shull. "Inspecting the history of inspections". In: *IEEE Software* (2008).

[4] Peter Rigby et al. "Open source peer review—lessons and recommendations for closed source". In: *IEEE Software* (2012).

[5] Alistair Sutherland and Gina Venolia. "Can peer code reviews be exploited for later information needs?" In: *Proceedings of the International Conference on Software Engineering*. 2009.

[6] Thomas LaToza, Gina Venolia, and Robert DeLine. "Maintaining mental models: a study of developer work habits". In: *Proceedings of the International Conference on Software Engineering*. 2006.

[7] Mark Stein et al. "Distributed software inspection". In: 1997.

[8] Oliver Laitenberger. "A survey of software inspection technologies". In: *Handbook on Software Engineering and Knowledge Engineering*. 2002, pp. 517–555.

[9] Adam Porter, Harvey Siy, and Lawrence Votta. "A review of software inspections". In: *Advances in Computers* 42 (1996), pp. 39–76.

[10] Justin P. Johnson. "Collaboration, peer review, and open source software". In: *Information Economics and Policy* 18 (2006), pp. 477–497.

[11] Lawrence G. Votta. "Does every inspection need a meeting?" In: *ACM SIGSOFT Software Engineering Notes* 18 (1993), pp. 107–114.
[12] Lisa Brothers, V. Sembugamoorthy, and Michael Muller. "ICICLE: groupware for code inspection". In: *Conference on Computer-Supported Cooperative Work*. 1990, pp. 169–181.
[13] Vahid Mashayekhi, Chris Feulner, and John Riedl. "CAIS: collaborative asynchronous inspection of software". In: *SIGSOFT Software Engineering Notes* 19 (1994), pp. 21–34.
[14] John Gintell et al. "Scrutiny: A collaborative inspection and review system". In: *Proceedings of the 4th European Software Engineering Conference*. 1993, pp. 344–360.
[15] Peter Rigby, Daniel German, and Margaret-Anne Storey. "Open source software peer review practices: a case study of the Apache server". In: *Proceedings of ICSE*. 2008.
[16] John W. Creswell. *Research Design: Qualitative, Quantitative, and Mixed Methods Approaches*. Sage, 2009.
[17] Bruce L. Berg. *Qualitative Research Methods for Social Sciences*. Pearson, 2004.
[18] Steve Adolph, Wendy Hall, and Philippe Kruchten. "Using grounded theory to study the experience of software development". In: *Empirical Software Engineering* 16.4 (2011), pp. 487–513.
[19] Thomas Lindlof and Bryan Taylor. *Qualitative Communication Research Methods*. Sage, 2002.
[20] Robert S. Weiss. *Learning From Strangers: The Art and Method of Qualitative Interview Studies*. Free Press, 1995.
[21] Ian Barker. *What is information architecture?* 2005.
[22] Stuart J. Janis et al. *Improving Performance Through Statistical Thinking*. McGraw-Hill, 2000.
[23] Barbara A. Kitchenham and Shari L. Pfleeger. "Personal opinion surveys". In: *Guide to Advanced Empirical Software Engineering*. Springer, 2007.
[24] Nahid Golafshani. "Understanding reliability and validity in qualitative research". In: *The Qualitative Report* 8 (2003), pp. 597–607.
[25] Bent Flyvbjerg. "Five misunderstandings about case-study research". In: *Qualitative Inquiry* 12.2 (2006), pp. 219–245.
[26] Victor Basili, Forrest Shull, and Filippo Lanubile. "Building knowledge through families of experiments". In: *IEEE Transactions on Software Engineering* 25.4 (1999), pp. 456–473.

Chapter 17

[Skim] Engineering Impacts of Anonymous Author Code Review: A Field Experiment

This paper investigates the practical implications of anonymous author code review, a technique intended to reduce bias by hiding the identity of code authors during review. Motivated by prior evidence of bias in software engineering and other domains, the authors conduct a large-scale field experiment at Google involving over 5,000 code reviews by 300 engineers. The study examines key aspects of the review process, including the ability of reviewers to infer author identity, the impact on review speed and quality, and perceptions of fairness. While anonymization is theoretically expected to mitigate biases related to gender, experience, or reputation, the paper highlights that its real-world effectiveness depends on how it interacts with existing development practices and social dynamics .

The findings show that anonymous author code review has nuanced effects rather than transformative ones. Reviewers were often able to correctly guess the author's identity—especially in familiar contexts—limiting the full potential of anonymization. Despite this, the practice reduced emphasis on hierarchical relationships and encouraged more neutral evaluation behavior. Importantly, the study finds that anonymization had little to no measurable impact on review quality or objective review speed, though participants sometimes perceived it as slowing communication due to reduced opportunities for direct interaction. Overall, the paper concludes that while anonymous author review does not drastically change engineering outcomes, it offers modest benefits in promoting fairness and reducing bias, with trade-offs related to context loss and communication overhead .

17.1 Motivation

The motivation for this work stems from growing evidence that code review—despite being perceived as an objective, quality-driven process—is influenced by social and cognitive biases related to the identity of the author. While developers often believe that code changes are evaluated purely on technical merit, prior research shows that factors such as gender [1, 2], race [3], age [4], and even physical attractiveness [5] can influence decision-making outcomes in professional settings. In the context of software engineering, studies have demonstrated that women's contributions to open-source projects are less likely to be accepted when their gender is identifiable, suggesting that implicit bias plays a role in code review decisions [1]. These findings challenge the assumption that code review is inherently fair and motivate the need for mechanisms that can reduce the influence of such biases.

To address similar concerns in other domains, anonymization has been widely adopted as a strategy to promote fairness. For example, blind auditions in orchestras have been shown to increase the representation of women by removing visual cues about the performer [6], and double-blind peer review in academia has been found to reduce advantages associated with famous authors or prestigious institutions

[7]. Inspired by these successes, the idea of anonymous author code review—where the reviewer does not know the identity of the code author—has been proposed as a potential solution to mitigate bias in software engineering [8]. However, despite its theoretical appeal and some early exploratory tools (e.g., anonymization extensions for GitHub), there is a lack of empirical evidence on how such a practice would function in real-world engineering environments. This gap is critical, as practitioners have expressed skepticism, with concerns that anonymization might hinder communication, reduce efficiency, or be impractical in collaborative development settings.

Consequently, the primary motivation of this paper is to bridge this empirical gap by systematically studying the real-world effects of anonymous author code review. The authors aim to move beyond theoretical arguments and anecdotal claims by conducting a large-scale field experiment to understand not only whether anonymization can reduce bias, but also how it impacts key engineering outcomes such as review velocity, code quality, and developer experience. Importantly, the study is not limited to fairness alone; it seeks to uncover the broader trade-offs involved in adopting anonymization in practice. By doing so, the paper positions itself at the intersection of software engineering practices and human-centered concerns, aiming to provide evidence-based guidance on whether and how anonymous author code review should be implemented in modern development workflows.

17.2 Methodology

17.2.1 Research Questions

The paper formulates six research questions to systematically evaluate the practical implications of anonymous author code review, focusing on both technical outcomes and human factors in the review process.

RQ (1) How often can reviewers guess author identities?

RQ (2) How does anonymous author code review change reviewers' velocity?

RQ (3) How does anonymous author code review change review quality?

RQ (4) What effect does anonymous author code review have on reviewers' and authors' perceptions of fairness?

RQ (5) What do engineers perceive as the biggest advantages and disadvantages of anonymous author code review?

RQ (6) What features are important to an implementation of anonymous author code review?

The first research question (**RQ (1)**) investigates whether anonymization is effective in practice by examining how often reviewers can correctly guess the identity of the author. This question is critical because prior work in double-blind peer review suggests that anonymity is effective when identities are difficult to infer [9, 10]. If reviewers can frequently deduce the author based on contextual cues such as code ownership or prior collaboration, the benefits of anonymization may be undermined. Thus, RQ1 establishes a foundational understanding of whether anonymity can realistically be preserved in software engineering environments.

The second (**RQ (2)**) and third (**RQ (3)**) research questions focus on core engineering outcomes: review velocity and review quality. RQ2 explores whether hiding author identity slows down the review process, as prior studies have identified review speed as a key concern in industrial settings [11]. Anonymity could introduce friction by limiting direct communication between reviewers and authors. RQ3 examines whether anonymization affects the quality of reviews, drawing on evidence from academic peer review where double-blind processes have been shown to yield equal or higher-quality evaluations [12, 13, 14, 15]. Together, these questions assess whether anonymous author review introduces trade-offs between fairness and efficiency or quality.

The fourth research question (**RQ (4)**) shifts focus to perceptions of fairness, a central motivation behind anonymization. Rather than attempting to measure fairness objectively, the study follows prior work in evaluating perceived fairness among participants [16, 17]. This question examines whether both reviewers and authors feel that the process becomes more equitable when identities are hidden.

The fifth research question (**RQ (5)**) broadens the analysis by identifying the perceived advantages and disadvantages of anonymous author code review from the perspective of engineers. This includes understanding practical trade-offs, such as reduced bias versus loss of context or communication barriers. Finally, the sixth research question (**RQ (6)**) investigates what features are necessary for effectively implementing anonymous author code review in real-world systems, providing actionable insights for tool and process design.

17.2.2 Method Overview

To systematically study anonymous author code review in practice, the authors conducted a large-scale field experiment at Google involving hundreds of engineers and thousands of code reviews. The approach combines controlled experimentation with real-world deployment, enabling both causal comparison (via control and treatment groups) and ecological validity. The study is designed to answer multiple research questions by integrating quantitative metrics (e.g., review time, rollback rates) with qualitative feedback (e.g., perceptions of fairness, quality, and usability). Statistical analyses were conducted using regression models with multiple covariates to isolate the effect of anonymization from other influencing factors .

Implementation of Anonymous Author Code Review The authors implemented anonymization using a browser extension that removed author-identifying information from Google's internal code review tool, Critique. This included hiding usernames in the review interface, associated emails, and notifications. However, due to practical limitations, anonymization could not be enforced across all channels (e.g., mobile notifications, external devices, or linked bug reports), requiring participants to follow guidelines such as filtering emails and avoiding certain interfaces. The extension also provided a "break-the-glass" feature that allowed reviewers to reveal the author's identity when necessary, acknowledging that complete anonymity may not always be practical. Importantly, while reviewers experienced anonymization, authors continued to see the standard interface.

Participants and Experimental Design Participants were recruited from Google engineers who met specific criteria, including sufficient tenure, team familiarity, and recent code review activity. This ensured that participants were experienced with the review process and could provide meaningful data. A stratified sampling strategy was used to include a subset of "readability reviewers," who review code for adherence to coding standards. Volunteers were randomly assigned to either a treatment group (anonymous review) or a control group (standard review), enabling comparative analysis. The study ultimately included 300 active participants in the treatment group and 95 in the control group, generating over 5,000 reviewed changelists in the treatment condition alone.

17.2.3 Data Collection

17.2.3.1 Quantitative Measures

The study employs a mixed-methods data collection strategy, combining large-scale quantitative logging with structured qualitative feedback to capture both behavioral and perceptual aspects of anonymous author code review. Quantitatively, the researchers instrumented their browser extension and internal tooling to collect fine-grained interaction data during the review process. One key metric collected was whether and when reviewers chose to deanonymize the author, which directly informs the effectiveness of anonymization and helps answer how often anonymity is practically preserved. This logging provides an objective signal of when anonymity breaks down in real-world usage .

Another major quantitative measure is review velocity, operationalized as "active reviewing time." This metric captures the amount of time a reviewer spends actively engaging with a changelist, including reading code, commenting, and consulting related resources such as documentation or APIs. By focusing on active engagement rather than elapsed time, the study avoids noise from idle periods and provides a more accurate estimate of effort and efficiency. This measure is critical for evaluating whether anonymization introduces friction into the review process .

To assess review quality, the authors use rollback rates as a proxy. Specifically, they track whether a changelist is reverted within a long-term window (up to several months after the review). The underlying

assumption is that higher-quality reviews are more likely to catch issues early, reducing the need for later rollbacks. While indirect, this measure allows the researchers to evaluate quality at scale without requiring manual inspection of code or subjective judgments. It also aligns with prior empirical work linking review outcomes to software quality.

17.2.3.2 Qualitative Measures

In addition to quantitative metrics, the study collects rich qualitative data through two types of structured reports. "CL reports" are completed immediately after each review and capture the reviewer's perceptions of identity guessability, velocity, quality, and fairness. These near-real-time responses reduce recall bias and reflect the reviewer's immediate experience. A second instrument, the "final report," is administered at the end of the study and captures more reflective feedback, including perceived advantages, disadvantages, and desired features of anonymized review systems. Together, these qualitative instruments provide insight into the human and experiential dimensions of the process that cannot be inferred from logs alone .

Finally, the study also gathers fairness perceptions from authors of the reviewed changelists. Importantly, authors were not informed whether their code was reviewed under anonymous conditions, reducing bias in their responses. This design allows the researchers to compare perceived fairness across conditions while minimizing expectancy effects. Overall, the data collection approach is comprehensive, triangulating across behavioral logs, self-reports, and multiple stakeholder perspectives to build a holistic understanding of anonymous author code review.

17.2.4 Data Analysis

The analysis phase relies on a combination of statistical modeling and qualitative coding to interpret the collected data. The authors primarily use regression-based techniques to isolate the causal effects of anonymization while controlling for confounding variables. Two main types of models are employed: review-level regressions (RLR), which analyze individual code review instances, and participant-level regressions (PLR), which aggregate behavior at the reviewer level. This dual approach allows the study to capture both fine-grained and broader patterns in the data .

A key strength of the analysis is the inclusion of extensive control variables. These include reviewer characteristics (e.g., tenure, seniority, role), author characteristics, and properties of the changelist (e.g., size, number of reviewers, type of change). The models also account for the prior relationship between reviewer and author (e.g., insider vs. outsider), which is known to influence review dynamics. By incorporating these covariates, the analysis mitigates confounding effects and ensures that observed differences between control and treatment groups can be more confidently attributed to anonymization rather than underlying differences in context .

The regression models also include random effects to account for individual variability among reviewers. This is important because different engineers may have distinct reviewing styles, speeds, or biases. By modeling reviewer identity as a random effect, the analysis captures this heterogeneity and avoids overgeneralizing from individual behavior. Statistical significance is evaluated using a standard alpha threshold (0.05), and model fit is reported using metrics such as adjusted (R^2) or pseudo-(R^2), depending on the type of regression used.

For qualitative data, the authors perform thematic coding of open-ended responses from participants. Responses are categorized into recurring themes (e.g., benefits, drawbacks, reasons for deanonymization), allowing the researchers to quantify and interpret common patterns in subjective experiences. While coding is primarily conducted by a single researcher—introducing potential subjectivity—the large volume of responses and systematic categorization provide meaningful insights into how developers perceive anonymous author review.

Overall, the data analysis approach is methodologically rigorous and well-aligned with the study's goals. By combining controlled statistical modeling with qualitative interpretation, the authors are able to not only measure the effects of anonymization but also explain the mechanisms behind those effects. This integrated approach strengthens the validity of the findings and enables a nuanced understanding of both technical outcomes and human factors in code review.

17.3 Results

17.3.1 Participation and Representativeness

Since participation in the study was voluntary, the authors first analyze who chose to participate and whether any systematic biases exist. They find that engineers with "readability" certification were significantly more likely to volunteer, while more experienced engineers (with longer tenure) were less likely to participate. Additionally, some regional differences were observed (e.g., higher participation from certain geographic regions), though most other factors did not show statistically significant effects. This suggests that while the sample is reasonably diverse, it is not perfectly representative and may slightly overrepresent certain subgroups.

The authors then examine whether participants consistently reported on all the code reviews they performed during the study period. Although participants were instructed to report every changelist (CL), compliance was not perfect. However, the reporting rates between the control and treatment groups were similar and not statistically different, indicating no systematic bias introduced by the experimental condition itself. That said, other factors—such as the size of the changelist, the number of reviewers, or whether the reviewer had prior familiarity with the author—did influence the likelihood of reporting. This reinforces the need for regression models later in the analysis to control for such confounding variables.

Another aspect of representativeness concerns how typical the reviewed changelists were compared to participants' normal work. The majority of participants in both groups reported that the changelists they reviewed during the study were either "very typical" or "somewhat typical" of their usual work. The treatment group reported slightly higher "very typical" ratings than the control group, suggesting that the anonymization process did not significantly disrupt the nature of the work being reviewed. This supports the ecological validity of the experiment, indicating that the findings are grounded in realistic development scenarios.

Finally, the authors analyze the structure of code reviews themselves, particularly the number of reviewers per changelist and the overlap between participants. Most changelists were reviewed by one or two reviewers, which aligns with standard industry practices. Only a very small fraction of reviews involved multiple study participants or a mix of control and treatment participants, minimizing the risk of cross-contamination between experimental conditions. Overall, this section concludes that while there are some participation biases inherent to voluntary studies, the dataset is sufficiently representative and well-controlled to support meaningful analysis of the research questions.

17.3.2 RQ1: Ability to Guess Author Identity

The results show that anonymization is often ineffective in typical development settings. In normal code reviews, reviewers were able to correctly guess the author in a large majority of cases, while uncertainty was relatively low. In contrast, in readability reviews—where reviewers are less familiar with authors—reviewers were uncertain about the author's identity most of the time. This highlights that anonymization works better in low-context scenarios but breaks down in environments with strong familiarity and ownership patterns .

Reviewers primarily inferred identities using contextual cues, such as the part of the codebase being modified, programming style, or ongoing work. Communication outside the tool (e.g., prior discussions) and limitations in the anonymization system also contributed to identity leakage. In some cases, reviewers explicitly chose to reveal the author to gain necessary context or communicate more effectively. Overall, RQ1 concludes that anonymity is difficult to preserve in collaborative, context-rich software development workflows.

17.3.3 RQ2: Impact on Review Velocity

From a subjective perspective, many participants believed that anonymization slowed down reviews or introduced friction, mainly due to reduced communication and lack of contextual information. While most reported no change in individual reviews, a significant portion expected negative effects on overall engineering velocity, indicating a perception that anonymity makes the process less efficient .

However, objective analysis using active reviewing time shows no significant impact of anonymization on review speed. Both control and treatment groups experienced similar changes in review time, suggesting that observed slowdowns were likely due to external factors (e.g., study effects) rather than anonymization itself. Thus, RQ2 highlights a gap between perception and reality: developers feel that anonymization slows them down, but measurable effects are minimal.

17.3.4 RQ3: Impact on Review Quality

Participants generally perceived that anonymization had no significant effect on review quality, with some indicating slight improvements due to increased focus on the code rather than the author. Few participants reported negative effects, and overall sentiment leaned toward neutrality or modest benefit.

Objective analysis using rollback rates supports this perception, showing no significant difference in quality between anonymous and non-anonymous reviews. This indicates that anonymization does not harm the effectiveness of code reviews in identifying issues or maintaining code correctness. Overall, RQ3 concludes that anonymous author code review preserves quality while potentially encouraging more objective evaluation behavior.

17.3.5 RQ4: Effect on Perceived Fairness

Anonymous author code review shows a largely neutral to mildly positive impact on perceived fairness. From the reviewer perspective, most participants reported no change in fairness during individual reviews, but a meaningful subset perceived improvements due to reduced influence of identity-based biases. In post-study reflections, this positive perception became stronger, suggesting that while the immediate effect may be subtle, developers conceptually associate anonymization with fairer evaluation practices .

From the author perspective, fairness perceptions remained largely unchanged across anonymous and non-anonymous conditions. Since authors were unaware of whether anonymization was applied, this result is less likely to be biased by expectations. An exception appears in readability reviews, where control reviews were perceived as more fair, though further analysis suggests this is due to unusually high control ratings rather than a negative effect of anonymization itself. Overall, anonymization does not significantly shift fairness perceptions but aligns with developers' expectations of improved impartiality.

17.3.6 RQ5: Perceived Advantages and Disadvantages

Participants identified fairness and objectivity as the primary advantages of anonymization. Reviewers reported focusing more on the code itself rather than the author, leading to more careful and unbiased evaluations. Some also noted improved critical feedback and reduced influence of hierarchy or prior relationships. However, many participants observed little benefit in practice because they could still infer author identity through contextual cues, limiting the effectiveness of anonymization in real-world settings.

The main disadvantages center around loss of context and communication barriers. Reviewers rely on knowledge of the author (e.g., expertise, intent, ownership) to interpret changes efficiently, and anonymization removes this signal. This often led to slower reviews, difficulty in decision-making, and reduced ability to initiate quick interactions. Additional drawbacks include tool limitations, inconvenience from over-redaction, and reduced awareness of team activity. Overall, RQ5 highlights a trade-off: anonymization can reduce bias but at the cost of efficiency and collaboration.

17.3.7 RQ6: Important System Features

The results emphasize that strict anonymity is impractical, and successful implementations require flexibility. The most critical feature identified is the ability to "break the glass" and reveal the author when necessary. This allows reviewers to recover essential context in situations where identity is important, balancing fairness with usability.

Participants also favored partial anonymization, such as revealing non-sensitive contextual signals (e.g., team or expertise) while hiding identity. Additionally, flexible adoption models—such as opt-in usage at the team or individual level—and delayed identity revelation (e.g., after approval) were seen as useful.

Overall, the findings suggest that effective systems should not enforce complete anonymity but instead provide configurable mechanisms that preserve workflow efficiency while reducing bias.

17.4 Limitations

The authors acknowledge several limitations related to the experimental design and execution, particularly concerning the incomplete nature of anonymization. Despite the use of a browser extension to hide author identities within the code review tool, reviewers could still infer identities through various external channels such as email notifications, bug tracking systems, or prior communication. In addition, contextual cues such as code ownership, familiarity with specific components, or ongoing tasks often enabled reviewers to correctly guess the author. This limitation directly affects the internal validity of the study, as the treatment condition does not represent a fully anonymized environment. As a result, the measured effects of anonymization may underestimate its potential impact in a more controlled or isolated setting.

Another important limitation arises from the voluntary participation and representativeness of the sample. Since engineers self-selected into the study, the participant pool may not fully reflect the broader engineering population at Google. For example, the study found that certain groups, such as readability reviewers, were more likely to participate, while more experienced engineers were less likely to volunteer. This introduces potential selection bias, which may influence both behavioral and perceptual outcomes. Additionally, not all participants consistently submitted reports for every code review they conducted, leading to incomplete data coverage. Although the authors attempt to control for these factors in their statistical models, the possibility of residual bias remains.

The study also relies on proxy and self-reported measures, which introduce additional sources of uncertainty. Review quality is approximated using rollback rates, which may not fully capture the nuances of code quality or the effectiveness of a review. Rollbacks can occur for reasons unrelated to review quality, and conversely, some issues may not result in a rollback. Similarly, qualitative measures such as perceived fairness, velocity, and quality are based on participant self-reports, which are subject to biases such as recall bias or social desirability bias. While collecting both objective and subjective data strengthens the analysis, these measures still have inherent limitations in accurately reflecting underlying constructs.

Finally, the authors highlight limitations related to generalizability and experimental context. The study is conducted within a single large organization with specific tools, workflows, and cultural practices, which may not generalize to other companies or open-source environments. The experimental setup itself may also introduce behavioral changes, such as participants being more cautious or attentive because they know they are being studied (a form of observer effect). Additionally, the relatively short duration of the experiment may not capture long-term adaptations to anonymization, such as changes in collaboration patterns or communication strategies. Together, these factors suggest that while the findings are informative, they should be interpreted with caution when considering broader adoption of anonymous author code review.

17.5 Implications

The discussion synthesizes the findings by emphasizing that anonymous author code review produces nuanced and context-dependent effects rather than fundamentally transforming the review process. A central takeaway is that anonymity is difficult to achieve in practice due to strong contextual signals embedded in software development workflows. Reviewers frequently rely on knowledge of code ownership, team structures, and prior interactions, which enables them to infer author identity even when explicit identifiers are removed. As a result, the potential benefits of anonymization—particularly in reducing bias—are inherently limited in environments where familiarity and collaboration are high. This aligns with prior work on anonymization and double-blind review, which shows that anonymity is most effective when contextual cues are minimal [10, 9].

At the same time, the discussion highlights a gap between perception and measurable outcomes. While developers often believe that anonymization slows down reviews and introduces friction, the empirical results show no significant impact on objective review velocity or quality. This suggests that concerns

about efficiency may be more perceptual than real, potentially influenced by changes in workflow or discomfort with reduced context. Similarly, although fairness perceptions do not change dramatically in the short term, participants broadly agree that anonymization promotes more objective evaluation in principle. These findings reflect broader insights from peer review research, where anonymization does not always lead to large measurable differences but is still valued for its role in mitigating bias [7, 16].

The authors also discuss the inherent trade-offs between fairness and practicality. Removing author identity can reduce bias and encourage more careful evaluation, but it also removes useful contextual information that reviewers depend on for efficient decision-making and communication. This tension suggests that fully anonymous systems may not be suitable for all scenarios. Instead, the discussion advocates for flexible approaches, such as partial anonymization or selective identity disclosure, which can balance the benefits of reduced bias with the need for context and collaboration. This perspective reinforces the idea that anonymization should be treated as a configurable tool rather than a one-size-fits-all solution.

Finally, the discussion situates the findings within the broader landscape of software engineering practices and human factors. The results suggest that social dynamics, familiarity, and communication patterns play a significant role in code review, often outweighing the effects of structural interventions like anonymization. The authors argue that future work should focus on designing systems that better support both fairness and usability, as well as exploring contexts where anonymization may be more effective, such as cross-team or low-familiarity reviews. Overall, the discussion underscores that while anonymous author code review has potential benefits, its impact is constrained by the deeply social nature of software development.

References

[1] Josh Terrell et al. "Gender differences and bias in open source: Pull request acceptance of women versus men". In: *PeerJ Computer Science* 3 (2017), e111.

[2] H Kristen Davison and Michael J Burke. "Sex discrimination in simulated employment contexts: A meta-analytic investigation". In: *Journal of Vocational Behavior* 56.2 (2000), pp. 225–248.

[3] Jeffrey H Greenhaus, Saroj Parasuraman, and Wayne M Wormley. "Effects of race on organizational experiences, job performance evaluations, and career outcomes". In: *Academy of Management Journal* 33.1 (1990), pp. 64–86.

[4] Robert A Gordon and Richard D Arvey. "Age bias in laboratory and field settings: A meta-analytic investigation". In: *Journal of Applied Social Psychology* 34.3 (2004), pp. 468–492.

[5] Megumi Hosoda, Eugene F Stone-Romero, and Gwen Coats. "The effects of physical attractiveness on job-related outcomes: A meta-analysis of experimental studies". In: *Personnel Psychology* 56.2 (2003), pp. 431–462.

[6] Claudia Goldin and Cecilia Rouse. "Orchestrating impartiality: The impact of "blind" auditions on female musicians". In: *American Economic Review* 90.4 (2000), pp. 715–741.

[7] Andrew Tomkins, Min Zhang, and William D Heavlin. "Reviewer bias in single- versus double-blind peer review". In: *Proceedings of the National Academy of Sciences* 114.48 (2017), pp. 12708–12713.

[8] Ji Young Kim, Gráinne M Fitzsimons, and Aaron C Kay. "Lean in messages increase attributions of women's responsibility for gender inequality". In: *Journal of Personality and Social Psychology* 115.6 (2018), p. 974.

[9] Stephen J Ceci and Douglas Peters. "How blind is blind review?" In: *American Psychologist* 39.12 (1984), pp. 1491–1494.

[10] Claire Le Goues et al. "Effectiveness of anonymization in double-blind review". In: *Communications of the ACM* 61.6 (2018), pp. 30–33.

[11] Caitlin Sadowski et al. "Modern code review: A case study at Google". In: *Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice*. 2018, pp. 181–190.

[12] Murad Alam et al. "Blinded vs. unblinded peer review of manuscripts submitted to a dermatology journal". In: *British Journal of Dermatology* 165.3 (2011), pp. 563–567.

[13] Mary Fisher, Stephen B Friedman, and Bernard Strauss. "The effects of blinding on acceptance of research papers by peer review". In: *JAMA* 272.2 (1994), pp. 143–146.

[14] Amy C Justice et al. “Does masking author identity improve peer review quality?” In: *JAMA* 280.3 (1998), pp. 240–242.
[15] Susan van Rooyen et al. “Effect of blinding and unmasking on the quality of peer review”. In: *JAMA* 280.3 (1998), pp. 234–237.
[16] Robert A McNutt et al. “The effects of blinding on the quality of peer review”. In: *JAMA* 263.10 (1990), pp. 1371–1376.
[17] Mark Ware. “Peer review in scholarly journals: Perspective of the scholarly community”. In: *Information Services & Use* 28.2 (2008), pp. 109–112.

Chapter 18

[Required] Are Automated Debugging Techniques really helping Programmers?

The paper investigates whether modern automated debugging approaches genuinely improve developers' effectiveness in practice. While a wide range of techniques—particularly those focused on fault localization through ranking suspicious code statements—have advanced the state of the art, the authors argue that these methods are often built on unrealistic assumptions about how programmers debug. In particular, many tools assume that identifying and inspecting a faulty statement in isolation is sufficient for understanding and fixing a bug. However, debugging in real-world settings is a complex, iterative process that involves navigating dependencies, examining program state, and repeatedly executing the program with different inputs. To address the gap between theoretical effectiveness and practical utility, the authors conduct human-centered empirical studies to evaluate how developers actually use automated debugging tools and whether these tools meaningfully support debugging tasks.

To explore this, the study involves controlled experiments with developers performing debugging tasks with and without a representative automated debugging tool based on statistical fault localization. The findings reveal a nuanced picture: while automated tools can sometimes help experienced developers complete simpler debugging tasks more quickly, their benefits do not consistently extend to more complex scenarios. Moreover, developers do not follow the assumptions embedded in these tools—for instance, they do not inspect ranked statements sequentially but instead rely on intuition and search strategies. The study also shows that simply pointing to a faulty statement is insufficient for bug understanding, as developers require additional context and exploration. Overall, the paper highlights a disconnect between how automated debugging techniques are designed and how debugging actually occurs in practice, emphasizing the need for tools that better align with human workflows and support richer interaction, context, and exploration.

18.1 Motivation

The motivation for this work stems from the well-recognized difficulty and cost associated with debugging in software development. Debugging is not only time-consuming but also cognitively demanding, often accounting for a substantial portion of software maintenance effort [1]. As modern software systems grow in complexity, configurability, and scale, the challenges associated with debugging are further amplified. This has driven significant research interest in developing automated debugging techniques aimed at improving developer efficiency and effectiveness [2, 3, 4]. Despite this progress, there remains an open question as to whether these techniques truly deliver practical benefits when used by real developers.

There exists a mismatch between how automated debugging techniques are evaluated and how debugging

is actually performed in practice. Many existing approaches focus primarily on fault localization[1] - identifying and ranking potentially faulty statements—while implicitly assuming that developers can easily understand and fix a bug once the faulty statement is identified[2]. This assumption of *perfect bug understanding* simplifies evaluation but overlooks the inherently complex and iterative nature of debugging, which involves reasoning about program behavior, exploring dependencies, and validating hypotheses through repeated execution [5]. As a result, current evaluation metrics may not accurately reflect the real-world usefulness of these techniques.

Secondly, a lack of empirical, human-centered studies in the debugging literature. Although foundational techniques such as program slicing have been studied for decades, only a limited number of works have investigated how developers actually use these tools in practice, and most of these studies involve small sample sizes and simplified programs [6, 7]. This scarcity of empirical evidence makes it difficult to assess whether automated debugging tools scale to realistic development scenarios or whether their underlying assumptions hold when applied to real-world tasks. Consequently, there is a need for systematic studies that examine developer behavior and tool usage in realistic debugging contexts.

Finally, the need to bridge the gap between research and practice in automated debugging. While many techniques demonstrate improvements in controlled or theoretical settings, their adoption in real development workflows remains limited. This suggests that existing tools may not align well with how developers think, explore code, and solve problems. By studying how developers interact with automated debugging tools, the authors aim to uncover the limitations of current approaches and identify directions for designing more effective, developer-centric debugging support. Ultimately, the goal is not to prove the superiority of any single technique, but to better understand what makes debugging tools useful in practice and how they can be improved to meet the needs of real users.

18.1.1 Automated Debugging Techniques

Early work in this area is rooted in **program slicing, introduced by [5, 8], which identifies all statements that may affect the value of a variable at a given program point**. The key idea is that if a variable exhibits incorrect behavior, the fault must lie within its slice, allowing developers to ignore irrelevant parts of the code. However, in practice, slices tend to be large and difficult to use effectively, which led to the development of more refined variants such as dynamic slicing [9] and other optimizations like relevant, critical, and pruned slices [10, 11]. While these techniques reduce the size of the candidate set, they still often produce too many statements to be practically useful.

To address the limitations of slicing-based approaches, a second class of techniques emerged that rely on analyzing program executions. **These techniques compare failing and passing runs to identify suspicious code regions, using various forms of execution data such as path profiles [12], counterexamples from model checking [3, 13], statement coverage [14], and predicate evaluations** [4]. Some approaches further enhance effectiveness by applying clustering techniques to reduce redundancy in execution traces and highlight meaningful patterns [15, 16, 17]. Collectively, these methods aim to rank statements based on their likelihood of being faulty, thereby guiding developers toward potential root causes more efficiently.

Despite their sophistication, a common limitation across these techniques is their narrow focus on reducing the number of statements developers must inspect. **They generally assume that once a suspicious statement is identified, the developer can easily determine whether it is faulty and fix the issue**. This assumption overlooks the broader cognitive and investigative processes involved in debugging, such as understanding execution context, analyzing dependencies, and exploring program state. In many cases, developers must rerun the program or gather additional information to make sense of a statement's behavior, which these techniques do not directly support.

While automated debugging techniques have significantly advanced in terms of algorithmic capability, their practical effectiveness remains uncertain. The lack of consideration for how developers actually interact with code during debugging raises concerns about their usability and real-world impact. This gap between technical design and human-centered usage motivates the need for empirical studies that

[1] Fault Localisation refers to identifying the lines of code responsible for the failure.
[2] Fault Understanding refers to understanding the root cause of the failure.

evaluate these techniques in realistic settings and inform the development of more effective debugging tools.

18.1.2 Related Work with Programmers

Early foundational work by [5] explored how programmers interact with program slices, showing that developers are more likely to recognize meaningful code fragments when they align with execution flow. However, these initial studies did not directly measure debugging effectiveness. Subsequent work by [7] attempted to evaluate whether slicing improves debugging performance, but found no significant benefit when developers used slicing tools on small programs. Interestingly, modifications to the experimental setup—such as using smaller programs and simplified representations like printed slices—led to some improvements, suggesting that the effectiveness of such techniques is highly sensitive to context and presentation.

Further studies investigated how developers naturally approach debugging and whether slicing aligns with their cognitive strategies. For instance, [6] found that only a small subset of programmers instinctively applied slicing-like reasoning during debugging, but those who did demonstrated better understanding and more systematic behavior. Similarly, [18] conducted controlled experiments comparing groups with and without slicing tools, finding mixed results: while some improvements in debugging time were observed for certain tasks, these benefits were not consistent across all scenarios. These findings indicate that even well-established techniques like slicing do not uniformly translate into improved debugging performance.

More recent work has explored alternative debugging aids and their impact on developer effectiveness. For example, [19] studied an Eclipse plugin that visualized differences between execution traces, finding some evidence that novice developers using the tool could outperform experts without it. Another notable system, Whyline, integrates visualization, slicing, and interactive querying to help developers understand program behavior. In a study by [20], participants using Whyline were able to debug significantly faster than those using traditional tools, demonstrating the potential of richer, more interactive debugging environments. These results suggest that combining multiple forms of support—rather than relying solely on statement ranking or slicing—may be key to improving debugging effectiveness.

Overall, the section underscores that empirical evidence for the effectiveness of automated debugging techniques remains sparse and inconclusive. Many studies involve small sample sizes, simplified programs, or limited task diversity, making it difficult to generalize findings to real-world development contexts. As a result, there is still a significant gap in understanding how these tools perform in practice and which features truly benefit developers. This lack of comprehensive evaluation motivates the need for more robust, large-scale, and realistic studies that examine both tool effectiveness and developer behavior, ultimately guiding the design of more usable and impactful debugging techniques.

18.2 Methodology

18.2.1 Program Subjects

The study uses two program subjects chosen to balance familiarity and complexity. The first subject is a Java implementation of Tetris, consisting of around 2,400 lines of code. This program represents a domain that most participants can easily understand due to its intuitive gameplay mechanics, allowing them to map program behavior to expected outcomes quickly. The second subject is NanoXML, an XML parsing library of approximately 4,400 lines of code. Unlike Tetris, NanoXML represents a less familiar and more technical domain, requiring deeper reasoning about parsing logic, namespaces, and structured data. Together, these two subjects allow the study to evaluate debugging behavior in both familiar and unfamiliar contexts.

18.2.2 Participants

Participants were 34 graduate students enrolled in software engineering courses, with varying levels of programming and debugging experience. Some participants had prior industry experience, including internships or full-time roles, while others primarily had academic experience. This diversity allowed the

study to capture a broad spectrum of debugging behaviors, from novice to more experienced developers. Since debugging is a universal activity across experience levels, the authors considered this variation beneficial for understanding how different types of programmers interact with debugging tools.

18.2.3 Instruments

The primary automated debugging technique used in the study is Tarantula, a statistical fault localization method that ranks program statements based on their likelihood of being faulty. This technique was selected because it is representative of many modern debugging approaches, relies on well-established statistical principles, and has shown competitive performance in prior evaluations. Importantly, Tarantula embodies the common paradigm of ranking suspicious statements, making it suitable for investigating how developers interact with such ranked outputs.

To make the technique accessible to participants, the authors developed a custom Eclipse plugin that presents the ranked list of suspicious statements. The interface was intentionally kept simple: it displays statements along with their file names, line numbers, and suspiciousness scores, and allows users to navigate directly to code by clicking entries. Additional navigation controls enable sequential traversal of the list. This minimalistic design isolates the effect of ranking-based debugging without introducing confounding factors from more complex visualizations or interfaces.

The ranked statements used by the plugin were generated using coverage data and pass/fail information from test executions. For NanoXML, existing test suites were used, while for Tetris, the authors created a capture-replay system to record gameplay sessions as test cases. These test cases were then used to compute suspiciousness scores. The study also provided all necessary artifacts, including the plugin, subjects, and instructions, to support reproducibility and future research.

18.2.4 Method

The experimental method is designed to evaluate three hypotheses related to the effectiveness of automated debugging tools. Participants were divided into multiple groups and assigned debugging tasks under different conditions—some using the automated tool and others relying on traditional debugging techniques. By comparing task completion times and success rates across groups, the study assesses whether automated tools improve debugging performance and whether their effectiveness varies with task difficulty.

Additional experimental variations were introduced to examine the impact of statement ranking. In some groups, the rank of the faulty statement was artificially modified to test whether higher ranking correlates with better performance. The study also collected detailed behavioral data, including navigation logs and questionnaire responses, to analyze how developers interact with the tool, how they interpret ranked information, and what challenges they encounter. This combination of quantitative and qualitative data enables a deeper understanding of debugging behavior beyond simple performance metrics.

18.2.5 Procedure

The experiment was conducted in controlled environments such as classrooms or labs, with participants given time to familiarize themselves with the debugging tool beforehand. During the study, participants were assigned two debugging tasks and given a fixed time limit for each. They were required to reproduce the failure, investigate the code, identify the fault, and propose a fix while recording their progress. After completing the tasks, participants filled out a questionnaire describing their experience and strategies. This structured procedure ensured consistency across participants while capturing both performance data and subjective insights into tool usage.

18.3 Results

18.3.1 Experts are faster when using tools

The results show that automated debugging tools can improve performance, but primarily for more experienced developers and simpler tasks. For the Tetris task, participants using the tool completed the

task faster on average than those using traditional debugging, although the difference was not always statistically significant across all participants. However, when focusing on high-performing participants, the improvement became significant, indicating that expertise plays a key role in leveraging such tools effectively. In contrast, for the more complex NanoXML task, no clear performance benefit was observed, suggesting that the tool's effectiveness is context-dependent and may not generalize across different types of debugging challenges.

18.3.2 The tools failed to perform harder tasks

Contrary to expectations, the automated debugging tool did not provide greater benefits for harder tasks. Although NanoXML was clearly more difficult—evidenced by lower completion rates—participants using the tool did not perform significantly better than those using traditional debugging. In fact, relative performance improvements were smaller for the harder task, indicating that the tool did not scale well with task complexity. This suggests that factors such as unfamiliarity with the codebase, increased cognitive load, or lack of contextual information may limit the usefulness of automated debugging techniques in more challenging scenarios.

18.3.3 Rank does not impact performance

The study examined whether the rank of the faulty statement influences debugging performance by artificially adjusting its position in the ranked list. Participants were divided into groups where the faulty statement's rank was either preserved or modified—lowered for one task and raised for another. The expectation was that higher-ranked faults would lead to faster debugging, while lower-ranked faults would slow participants down.

The results, however, showed no significant impact of rank on performance. Even when the faulty statement was moved much higher in the list, participants did not complete the task faster. Similarly, lowering the rank did not significantly hinder performance. These findings suggest that developers do not rely strictly on ranking when navigating suspicious statements and may use other strategies to guide their debugging process.

A closer analysis revealed that participants often compensated for poor ranking by identifying related or nearby statements, particularly those within the same function or context as the fault. This indicates that ranking alone is insufficient to drive debugging effectiveness, as developers rely on broader contextual reasoning and exploration. The findings challenge the common assumption that improving rank directly translates to improved debugging performance.

18.3.4 Programmers search statements

The analysis of navigation logs revealed that developers do not follow a linear, top-down approach when inspecting ranked statements. Instead, they frequently jump between positions in the list, often skipping multiple entries at a time. A significant portion of navigation steps involved large jumps, indicating that developers actively search for relevant code rather than systematically inspecting each item.

Additionally, participants exhibited non-linear navigation patterns characterized by frequent changes in direction, or "zigzags," as they moved up and down the list. Questionnaire responses confirmed that developers used intuition, hypotheses, and keyword-based reasoning to guide their exploration. These behaviors highlight that debugging is an active search process, not a passive consumption of ranked results, and that tools should better support exploratory workflows.

18.3.5 No Perfect Bug Understanding

The study found strong evidence against the assumption of perfect bug understanding. Even after encountering the faulty statement, most participants continued to explore other parts of the code for a significant amount of time. On average, they spent more than half of their debugging time after first seeing the fault, indicating that identifying a suspicious statement is not sufficient for understanding or fixing the bug. This underscores the importance of providing additional context, such as execution state or dependencies, to support effective debugging.

18.3.6 Benefits and Challenges

Participants reported that the primary benefit of the automated debugging tool was its ability to point them in the right direction rather than directly identify the fault. The tool helped narrow down the search space and suggest starting points for investigation, which participants then explored further using traditional debugging techniques such as breakpoints. This suggests that automated tools are most useful as complementary aids rather than standalone solutions.

At the same time, several challenges were identified that limited the tool's effectiveness. Participants noted the lack of contextual information, such as runtime values, program structure, and relationships between statements, which made it difficult to interpret the ranked results. Additionally, long lists of suspicious statements and false positives led to frustration and reduced trust in the tool, causing some participants to abandon it altogether. These findings highlight the need for richer, more integrated debugging tools that provide context, explanation, and better interaction mechanisms.

18.4 Implications

18.4.1 Programmers' Behavior

Programmers Without Tools Sometimes Fix the Failure Instead of the Fault A key observation from the study is that developers using traditional debugging approaches sometimes resolve visible symptoms of a bug rather than addressing its underlying cause. Instead of identifying and correcting the root fault, they may modify parts of the code in a way that prevents the failure from occurring, effectively masking the issue. This behavior reflects a pragmatic but potentially problematic debugging strategy, where immediate functionality is restored without ensuring long-term correctness or robustness. Such fixes may introduce hidden issues or technical debt, especially in larger systems where the root cause remains unaddressed.

Interestingly, this behavior was predominantly observed among participants who did not use the automated debugging tool. In contrast, those who used the tool were more likely to locate and fix the actual fault. This suggests that automated debugging aids can guide developers toward more principled debugging practices by narrowing their focus to relevant code regions. By highlighting suspicious statements, the tool may encourage developers to investigate deeper causes rather than applying superficial fixes. As a result, automated tools may not only improve efficiency but also promote better debugging quality and correctness.

Programmers Want Values, Overviews, and Explanations The study reveals that developers rely heavily on contextual information when debugging and that current automated tools often fail to provide sufficient support in this regard. While ranking-based tools help identify suspicious code locations, developers still turn to traditional debugging techniques—such as setting breakpoints—to inspect runtime values and understand program behavior. This indicates that fault localization alone is not enough; developers need access to dynamic information and contextual cues to interpret why a piece of code is faulty. Without this, the debugging process remains incomplete and requires additional effort outside the tool.

Furthermore, developers expressed a strong need for better organization and explanation of debugging information. They preferred higher-level overviews, such as grouping suspicious statements by methods or files, and desired explanations that connect statements to execution traces, test cases, and program slices. The lack of coherence in ranked lists makes it difficult for developers to form a mental model of the problem, reducing trust in the tool. These findings suggest that future debugging tools should move beyond simple ranking and incorporate richer, more interactive features that support exploration, explanation, and understanding of program behavior.

18.4.2 Threats to Validity

Time Constraints and Participant Performance One important threat to validity arises from the time limits imposed during the experiment. Participants were given a fixed duration to complete each debugging task, which was necessary to keep the study manageable but may have influenced the results.

Less experienced participants, in particular, may not have had sufficient time to fully understand the code and identify the fault, leading to incomplete or skewed performance outcomes. As a result, the findings may be biased toward more experienced or efficient participants, limiting the generalizability of the conclusions across different skill levels.

Participant Demographics and Experience Another potential threat concerns the use of graduate students as participants instead of professional developers. Although some participants had industry experience, their overall expertise and real-world exposure may not fully represent that of experienced software engineers working in production environments. Differences in debugging habits, familiarity with tools, and domain knowledge could affect how participants interact with automated debugging techniques. Consequently, the observed behaviors and results may not entirely reflect how such tools would perform in professional settings.

Task and Program Selection The choice of only two programs and specific debugging tasks introduces a limitation in terms of external validity. The selected subjects—Tetris and NanoXML—represent particular types of applications and may not capture the diversity of real-world software systems. Additionally, participants were unfamiliar with the codebases, which may have affected their ability to debug effectively compared to situations where developers work on familiar systems. Furthermore, differences in failure information, such as the presence of a stack trace in one task but not the other, could have influenced the relative effectiveness of the debugging tool. These factors limit the extent to which the results can be generalized to other programs, domains, and debugging scenarios.

18.5 Implications

Percentages Will Not Cut It A major finding of the study is that commonly used evaluation metrics for automated debugging—particularly percentage-based rankings—do not accurately reflect practical usefulness. Many techniques report success by showing that a faulty statement appears within a small percentage of the codebase (e.g., top 2%), which may seem promising in theory. However, in practice, developers do not inspect large portions of ranked results. Instead, they tend to focus only on the top few entries and quickly lose interest if those do not appear helpful. This creates a disconnect between reported effectiveness and actual developer behavior.

The study suggests that absolute rank is a far more meaningful metric than percentage rank. Developers care about how quickly they can reach useful information, not how small the relative search space is. Even when the rank of the faulty statement was improved significantly (e.g., from position 83 to 16), no noticeable performance improvement was observed. This indicates that beyond a certain threshold, improvements in ranking do not translate into real benefits. Therefore, future research should prioritize techniques that consistently place faults among the very top results, rather than optimizing for percentage-based metrics that may be misleading.

Focus More on Search Another important implication is that debugging tools should better support search-oriented behaviors rather than relying solely on ranking. The study found that developers naturally engage in searching and filtering when navigating suspicious statements, often using domain knowledge or hypotheses about the bug to guide their exploration. Instead of following the ranked list sequentially, they jump across entries and look for patterns or keywords that align with their expectations about the failure.

This suggests that combining ranking with search capabilities could significantly enhance debugging effectiveness. For example, allowing developers to filter statements based on relevant terms or automatically highlighting potentially useful keywords could help them quickly narrow down the search space. In some cases, search-based strategies may even be more effective than ranking alone. By incorporating mechanisms that align with how developers think and explore code, debugging tools can become more intuitive and better support real-world debugging workflows.

Grow the Ecosystem The study also highlights the importance of considering the broader debugging ecosystem rather than focusing on isolated techniques. Many automated debugging approaches require

additional setup steps, such as collecting execution traces, labeling test cases, and computing suspiciousness scores. In practice, the time and effort required to perform these steps may outweigh the benefits provided by the tool, especially if the integration with existing workflows is limited.

To be truly effective, debugging tools must provide end-to-end support that seamlessly integrates all aspects of the debugging process. This includes automating data collection, managing test cases, enabling easy reruns, and supporting code inspection within a unified environment. By reducing friction and improving usability, such an ecosystem can make automated debugging tools more practical and appealing for everyday use. The findings emphasize that technical improvements alone are insufficient; attention must also be given to usability, integration, and workflow alignment.

References

[1] I. Vessey. "Expertise in debugging computer programs". In: *International Journal of Man-Machine Studies* 23.5 (1985), pp. 459–494.

[2] A. Zeller and R. Hildebrandt. "Simplifying and isolating failure-inducing input". In: *IEEE Transactions on Software Engineering*. Vol. 28. 2. 2002, pp. 183–200.

[3] T. Ball, M. Naik, and S. K. Rajamani. "From symptom to cause: localizing errors in counterexample traces". In: *Proceedings of POPL*. 2003, pp. 97–105.

[4] B. Liblit et al. "Scalable statistical bug isolation". In: *PLDI*. 2005.

[5] M. Weiser. "Program slicing". In: *Proceedings of ICSE*. 1981, pp. 439–449.

[6] M. A. Francel and S. Rugaber. "The value of slicing while debugging". In: *Science of Computer Programming* 40 (2001), pp. 151–169.

[7] M. Weiser and J. Lyle. "Experiments on slicing-based debugging aids". In: *Workshop on Empirical Studies of Programmers*. 1986, pp. 187–197.

[8] Mark Weiser. "Program slicing". In: *IEEE Transactions on Software Engineering* 10.4 (1984), pp. 352–357.

[9] Bogdan Korel and Janusz Laski. "Dynamic program slicing". In: *Information Processing Letters* 29.3 (1988), pp. 155–163.

[10] Tibor Gyimóthy, Árpád Beszédes, and István Forgács. "An efficient relevant slicing method for debugging". In: *Proceedings of ESEC/FSE*. 1999, pp. 303–321.

[11] Xiangyu Zhang, Neelam Gupta, and Rajiv Gupta. "Pruning dynamic slices with confidence". In: *Proceedings of PLDI*. 2006, pp. 169–180.

[12] Thomas Reps et al. "The use of program profiling for software maintenance with applications to the year 2000 problem". In: *Proceedings of ESEC/FSE*. 1997, pp. 432–449.

[13] Alex Groce, Daniel Kroening, and Flavio Lerda. "Understanding counterexamples with explain". In: *Proceedings of CAV*. 2004, pp. 453–456.

[14] James A. Jones, Mary Jean Harrold, and John Stasko. "Visualization of test information to assist fault localization". In: *Proceedings of ICSE*. 2002, pp. 467–477.

[15] Manos Renieris and Steven P. Reiss. "Fault localization with nearest neighbor queries". In: *Proceedings of ASE*. 2003, pp. 30–39.

[16] Chao Liu and Jiawei Han. "Failure proximity: A fault localization-based approach". In: *Proceedings of FSE*. 2006, pp. 286–295.

[17] Dan Hao et al. "A similarity-aware approach to testing based fault localization". In: *Proceedings of ASE*. 2005, pp. 291–294.

[18] Shinji Kusumoto et al. "Experimental evaluation of program slicing for fault localization". In: *Empirical Software Engineering* 7 (2002), pp. 49–76.

[19] K. D. Sherwood and G. C. Murphy. *Reducing code navigation effort with differential code coverage*. Tech. rep. University of British Columbia, 2008.

[20] Andrew J. Ko and Brad A. Myers. "Finding causes of program output with the Java Whyline". In: *Proceedings of CHI*. 2009, pp. 1569–1578.

Part III

Reading Notes - April

Chapter 19

[Required] A Large-Scale Survey on the Usability of AI Programming Assistants: Successes and Challenges

The rapid rise of AI programming assistants such as GitHub Copilot and ChatGPT has introduced a fundamental shift in how software is developed, transforming programming into a collaborative process between humans and AI systems. These tools are capable of generating code, assisting with debugging, and suggesting implementations, leading to growing interest in their impact on developer productivity and software quality. While some studies highlight their ability to produce high-quality code and improve perceived productivity, others report mixed results regarding actual task completion and code quality. Notably, developers do not accept a large proportion of AI-generated suggestions, indicating a gap between the potential of these tools and their practical usability.

To better understand this gap, Liang et al. conduct a large-scale empirical study investigating how developers use AI programming assistants and the usability challenges they encounter. Through a survey of 410 developers, the study explores usage patterns, motivations, and barriers associated with these tools. The findings reveal that developers primarily use AI assistants for accelerating routine tasks—such as reducing keystrokes and recalling syntax—rather than for higher-level problem solving. At the same time, significant usability issues persist, including difficulties in controlling generated outputs and ensuring that code meets functional and non-functional requirements. These insights highlight the need for improved interaction design and alignment between AI-generated code and developer expectations.

19.1 Motivation

The rapid adoption of AI programming assistants has significantly reshaped modern software development, prompting researchers to investigate their real-world effectiveness and usability. Tools such as GitHub Copilot and ChatGPT have gained widespread attention due to their ability to generate code and assist developers in various tasks. Prior studies suggest that these systems can produce **high-quality code suggestions** [1, 2], and in some cases are associated with **improvements in developers' perceived productivity** [3]. However, conflicting evidence exists, as other work reports **no significant improvements in task completion or code quality** when using such tools [4, 5]. This inconsistency motivates a deeper investigation into how these tools perform in practice.

Despite their technical capabilities, developers often do not fully adopt AI-generated suggestions. Empirical findings indicate that only a relatively small proportion of suggestions are accepted in real-world usage—for example, acceptance rates of around **23–29% across common programming languages** [3]. This raises important questions about the usability of AI programming assistants. Prior work suggests that developers may hesitate due to concerns such as **incorrect or low-quality code**, lack of adherence to project-specific requirements, or difficulty in understanding generated outputs [5]. Addi-

tionally, developers may struggle with **debugging and adapting AI-generated code**, particularly when it relies on unfamiliar frameworks or APIs [6].

Existing literature has identified several challenges in using AI programming assistants, but it lacks a comprehensive understanding of how frequently these issues occur and which ones matter most in practice. Most prior studies either focus on controlled environments or examine limited aspects of usability. As a result, there is a need to systematically study **the prevalence and importance of usability factors** across a broader developer population. Understanding these aspects is critical because improvements in model performance alone may not translate into better developer experiences if the tools remain difficult to use or misaligned with developer needs [7].

Motivated by these gaps, the paper aims to provide a large-scale, empirical analysis of how developers interact with AI programming assistants and the challenges they face. By quantifying usability issues and usage patterns, the study seeks to inform both researchers and tool designers about **which aspects of these systems are effective and which require improvement**. Ultimately, this work is driven by the goal of enabling better tool design that reduces cognitive overhead, improves alignment with developer expectations, and increases the practical adoption of AI-assisted programming tools.

19.1.1 Related Work

The paper situates itself within a growing body of work on the usability of AI programming assistants, spanning both early program synthesis systems and modern transformer-based tools. Prior studies on programming-by-demonstration and neural code generation systems have highlighted usability challenges such as difficulty in correcting generated code and mismatches between user expectations and tool capabilities [8, 9]. Other research has explored the learnability of such tools, particularly for novice programmers, and identified interaction challenges such as refining generated outputs and disambiguating user intent [10, 11]. Compared to these earlier approaches, the current paper focuses specifically on widely deployed, transformer-based assistants like GitHub Copilot and Tabnine, which have demonstrated strong performance in handling both natural language and code inputs [12, 1]. This shift motivates a need to understand usability not just in controlled settings, but in real-world developer workflows.

More recent empirical studies have examined how developers interact with transformer-based AI programming assistants in practice. Research has shown that users often struggle with expressing intent in natural language queries and that generated code may assume familiarity with specific APIs or frameworks [6]. User studies on GitHub Copilot reveal additional challenges, including difficulties in understanding and debugging generated code, as well as varying usage patterns such as "exploration" and "acceleration" modes [5, 13]. Other work has identified common developer activities when using such tools, including verifying suggestions, debugging, and consulting documentation [14]. Large-scale analyses of usage data further indicate relatively low acceptance rates of generated suggestions, despite improvements in perceived productivity [3]. Building on these findings, the current study extends prior work by *quantifying the prevalence and importance of usability challenges across a broader set of tools and a larger developer population*, providing a more comprehensive understanding of how these issues manifest in practice.

19.2 Methodology

Participants The study recruited a large and diverse group of developers to capture a broad range of experiences with AI programming assistants. Participants were identified from GitHub repositories related to tools such as Copilot and Tabnine, resulting in an initial pool of over **33,000 users**, which was later filtered to **10,530 invited participants** based on available contact information. Ultimately, the survey received **410 responses**, providing a sufficiently large dataset for analysis. The participants represented a wide geographic distribution across **57 countries** and included developers with varying levels of experience, ranging from **1 to 41 years**, with a median of 6 years.

The participant pool also reflected diverse development contexts and technical backgrounds. Respondents reported programming in multiple languages such as **Python, JavaScript, TypeScript, and Java**, and contributed to projects in professional, academic, open-source, and hobby settings. Many participants had direct experience using AI programming assistants, including tools like GitHub Copilot, ChatGPT, and organization-specific systems. This diversity ensures that the findings reflect **real-world**

usage patterns across different domains and experience levels, strengthening the external validity of the study.

Survey The authors designed a structured survey to systematically capture developers' experiences with AI programming assistants. The survey was administered using Qualtrics and took approximately **15 minutes to complete**. It included both **closed-ended questions** (for quantitative analysis) and **open-ended questions** (to capture qualitative insights). Participants were asked to reflect on a specific project where they used AI tools, helping ground their responses in **concrete, real-world experiences** rather than abstract opinions.

The survey covered multiple aspects of tool usage, including **frequency of use, motivations for using or not using the tools, usability challenges, and reasons for abandoning generated code**. It also included questions about strategies for improving tool output and participant concerns about AI-generated code. To ensure quality, the survey was **piloted with 11 developers** and refined iteratively based on feedback, improving clarity and coverage of usability factors. All questions were optional, and participants were incentivized through a sweepstakes reward.

Analysis The study employed a combination of **quantitative and qualitative analysis techniques** to interpret the survey data. For quantitative analysis, the authors focused on **frequency-based measures**, such as how often participants selected certain responses or rated items as important. These measures were used to estimate the **perceived importance and prevalence** of different usability issues, rather than attempting to capture exact behavioral frequencies.

For qualitative analysis, the authors conducted **open coding on free-text responses**, following established best practices. Multiple rounds of coding were performed, where responses were labeled, merged into themes, and refined into a shared codebook through consensus. This process emphasized **identifying recurring patterns and themes** in developer experiences, such as common frustrations or successful use cases. By combining both analytical approaches, the study provides a **comprehensive and nuanced understanding** of how developers interact with AI programming assistants.

19.3 Key Findings

19.3.1 Usage Characteristics

19.3.1.1 Usage Patterns

The study shows that AI programming assistants are widely used but primarily in a supportive role rather than as primary code generators. Among the tools, GitHub Copilot was the most commonly used, with developers reporting that a median of **30.5% of their code is written with assistance from the tool**. Interestingly, even though tools like ChatGPT had a higher proportion of frequent users, they contributed a smaller portion of actual code generation. Overall, developers tend to use these tools frequently but selectively, leveraging them mainly for **specific parts of development rather than end-to-end coding**. This indicates that AI assistants are integrated into workflows as **partial productivity enhancers rather than complete automation tools**.

19.3.1.2 Motivation

Developers are primarily motivated to use AI programming assistants for efficiency and convenience. The most important reasons include **reducing keystrokes (autocomplete)**, **finishing tasks faster**, and **recalling syntax without searching online**. These motivations highlight the role of AI assistants in accelerating routine development tasks. On the other hand, developers are less motivated to use these tools for higher-level activities such as **discovering new solutions or identifying edge cases**. For those who avoid using AI assistants, the main reasons include **generated code not meeting functional or non-functional requirements**, **difficulty in controlling outputs**, and the **effort required to debug or modify generated code**. Overall, the findings emphasize that while AI assistants are valued for speed and convenience, their limitations in control and reliability significantly impact adoption.

19.3.1.3 Successful Use Cases

The study finds that developers are most successful when using AI programming assistants for **repetitive and well-defined tasks**. Common use cases include generating **boilerplate code, CRUD operations, and simple utility functions**, where the logic is straightforward and predictable. These tools are also effective for **autocomplete and small code completions**, allowing developers to quickly extend patterns they have already started. In such scenarios, AI assistants function as a strong **acceleration mechanism**, helping developers save time without requiring deep reasoning from the model.

Beyond routine tasks, developers also use AI assistants in slightly more exploratory contexts, though with less consistency. These include **generating test cases, handling edge cases, creating proof-of-concepts, and learning new programming languages or APIs**. The tools are particularly useful when developers need a **starting point or reference implementation**, especially in unfamiliar domains. However, their effectiveness decreases as task complexity increases, with developers noting that AI-generated code often struggles with **complex algorithms or non-trivial logic**, reinforcing that current tools are best suited for **simple, structured, and repetitive coding scenarios**.

19.3.1.4 User Input Strategies

Developers employ several strategies to improve the quality of AI-generated code, with the most common being providing **clear and explicit instructions**. This often involves writing detailed comments, docstrings, or step-by-step descriptions of the intended functionality. Many users also enhance outputs by **adding partial code as context**, allowing the assistant to infer patterns and complete the remaining implementation. Additionally, following **consistent naming conventions and coding styles** helps the model generate more relevant and accurate suggestions.

Another important strategy is breaking down problems into smaller, manageable parts. Developers often **decompose complex tasks into simpler steps** and iteratively refine prompts to guide the assistant more effectively. Some users also rely on **existing project context**, such as open files or previously written code, to improve suggestions. While a subset of developers reported using no specific strategy, others engaged in forms of **prompt engineering**, adjusting inputs dynamically until satisfactory results were obtained. Overall, these strategies highlight that effective use of AI assistants requires **active user involvement and careful input design**, rather than passive reliance on generated outputs.

19.3.2 Usability of AI Assistants

The study highlights that usability remains a central challenge in the adoption of AI programming assistants, despite their growing popularity. A key issue is the **misalignment between developer intent and generated output**, where suggestions often fail to meet functional or contextual requirements. Developers frequently report that while code may be syntactically correct, it lacks relevance to the specific task or project constraints. This creates additional overhead, as users must refine, correct, or discard suggestions, reducing the overall efficiency gains promised by these tools.

Another major usability concern is the **lack of control and predictability** in how AI assistants generate code. Developers struggle to guide the system toward desired outputs, often needing multiple attempts or rephrasing inputs. The inability to precisely influence results leads to frustration, particularly when dealing with non-trivial tasks. Additionally, users find it difficult to **understand and trust generated code**, especially when it introduces unfamiliar constructs, dependencies, or hidden assumptions. This lack of transparency further complicates debugging and integration into existing codebases.

The study also emphasizes the broader impact of usability issues on developer workflows. AI assistants are most effective when they align with **existing development practices and cognitive processes**, but current limitations often disrupt this alignment. Developers must invest effort in verification, testing, and adaptation, which can offset productivity gains. As a result, usability is not just a secondary concern but a **critical factor determining adoption and long-term effectiveness**. Improving aspects such as contextual awareness, explainability, and interaction design is essential for making these tools more reliable and developer-friendly.

19.3.3 Additional Feedback

Participants provided additional feedback that sheds light on their expectations and concerns regarding AI programming assistants. A recurring theme is the desire for **greater reliability and accuracy**, with developers expressing frustration over incorrect or incomplete code suggestions. Many users emphasized the importance of tools that can better understand **project-specific context, coding standards, and constraints**, rather than generating generic solutions. This reflects a need for assistants that can operate more effectively within real-world development environments.

Another key area of feedback relates to the **interaction experience and usability improvements**. Developers expressed interest in features that allow for better control over outputs, such as more interactive refinement mechanisms, explanations of generated code, and the ability to specify constraints more clearly. There is also a demand for improved support in tasks such as **debugging, testing, and maintaining code**, indicating that users want AI assistants to go beyond generation and provide more holistic development support.

Finally, participants raised broader concerns about the long-term implications of using AI programming assistants. These include potential risks such as **over-reliance on AI tools, reduced learning opportunities for developers, and the introduction of subtle bugs or security vulnerabilities**. Some developers also noted ethical and professional considerations, such as code ownership and trust in AI-generated outputs. Overall, the feedback suggests that while developers recognize the benefits of these tools, their future success depends on addressing both **technical limitations and human-centric concerns**.

19.4 Implications

The paper outlines several directions for future research to build on its findings. One key area is the need for more **longitudinal and observational studies** that capture how developers use AI programming assistants over time. Such studies could provide a deeper understanding of actual usage patterns, complementing self-reported data and addressing limitations related to recall bias. Additionally, future work could explore **controlled experiments** to better measure the impact of these tools on productivity, code quality, and developer behavior.

Another important direction is improving the design and capabilities of AI programming assistants. The study highlights the need for systems that offer **better alignment with developer intent, improved contextual awareness, and greater controllability**. Research could focus on enhancing interaction mechanisms, such as enabling more precise input specifications, providing explanations for generated code, and supporting iterative refinement. These improvements would help address many of the usability challenges identified in the study.

Finally, future work should consider broader implications and emerging use cases of AI-assisted programming. This includes investigating **how these tools affect learning, collaboration, and software engineering practices over time**. There is also a need to study potential risks, such as over-reliance on AI or the introduction of subtle errors, in more depth. By expanding the scope of research, future studies can contribute to developing **more reliable, effective, and human-centered AI programming tools**.

19.5 Threats to Validity

The study acknowledges several limitations related to its design and data collection. A primary concern is **selection bias**, as participants were recruited from GitHub repositories associated with AI programming assistants, which may overrepresent developers who are already familiar with or favorable toward such tools. Additionally, the response rate is relatively small compared to the number of invited participants, raising the possibility of **non-response bias**. This means that the findings may not fully generalize to the broader population of developers, especially those who do not actively engage with AI tools.

Another threat arises from the reliance on self-reported data. Participants were asked to recall their experiences and estimate usage patterns, which introduces risks of **recall bias and subjective in-**

terpretation. Developers may overestimate or underestimate their reliance on AI assistants or the effectiveness of these tools. Furthermore, since the study captures perceptions rather than direct observations, there is a gap between **reported behavior and actual usage**, which could influence the accuracy of conclusions drawn about productivity and usability.

Finally, there are concerns related to construct and external validity. The survey design, while carefully structured, may not fully capture all relevant aspects of usability, leading to potential **measurement limitations**. Additionally, the study focuses on a specific set of tools and time period, which may limit its applicability as AI programming assistants rapidly evolve. As a result, while the findings provide valuable insights, they should be interpreted with an understanding of these **contextual and methodological constraints**.

References

[1] Frank F. Xu et al. "A systematic evaluation of large language models of code". In: *MAPS*. 2022.

[2] Burak Yetistiren, Isik Ozsoy, and Eray Tuzun. "Assessing the quality of Github Copilot's code generation". In: *PROMISE*. 2022.

[3] Albert Ziegler et al. "Productivity assessment of neural code completion". In: *MAPS*. 2022.

[4] Saki Imai. "Is GitHub Copilot a substitute for human pair-programming?" In: *ICSE Companion*. 2022.

[5] Priyan Vaithilingam et al. "Expectation vs. experience: Evaluating usability of code generation tools". In: *CHI*. 2022.

[6] Frank F. Xu et al. "In-IDE code generation from natural language: Promise and challenges". In: *TOSEM* (2022).

[7] Brad A. Myers et al. "Programmers are users too: Human-centered methods for improving programming tools". In: *Computer* (2016).

[8] Kasra Ferdowsifard et al. "Small-step live programming by example". In: *UIST*. 2020.

[9] Xi Victoria Lin et al. *Program synthesis from natural language using recurrent neural networks*. Tech. rep. University of Washington, 2017.

[10] Dhanya Jayagopal et al. "Exploring the learnability of program synthesizers by novice programmers". In: *UIST*. 2022.

[11] Andrew McNutt et al. "On the design of AI-powered code assistants for notebooks". In: *CHI*. 2023.

[12] Ashish Vaswani et al. "Attention is all you need". In: *NeurIPS*. 2017.

[13] Shraddha Barke et al. "Grounded Copilot: How programmers interact with code-generating models". In: *arXiv* (2022).

[14] Hussein Mozannar et al. "Reading between the lines: Modeling user behavior and costs in AI-assisted programming". In: *arXiv* (2022).

Chapter 20

[Skim] Evolving with AI: A Longitudinal Analysis of Developer Logs

The rapid integration of AI-powered coding assistants into modern development environments has transformed software engineering into a form of human-AI collaboration. While tools such as GitHub Copilot and JetBrains AI Assistant promise improvements in productivity and efficiency, their long-term impact on real-world developer workflows remains insufficiently understood. Most prior research has relied on short-term experiments or self-reported perceptions, leaving a gap in understanding how developers' behaviors evolve over time when AI becomes embedded in everyday programming practices.

To address this gap, the paper presents a mixed-method longitudinal study combining telemetry data from 800 developers over two years with survey and interview insights. By analyzing five key workflow dimensions—productivity, code quality, code editing, code reuse, and context switching—the study offers a comprehensive view of how AI reshapes development practices. Importantly, it contrasts what developers actually do (behavioral logs) with what they perceive (survey responses), revealing nuanced differences between observed and perceived impacts.

20.1 Motivation

The primary motivation of this work arises from the gap between widespread adoption of AI coding assistants and the limited understanding of their long-term effects on developer workflows. While tools such as GitHub Copilot and similar systems are increasingly embedded in IDEs, most prior studies focus on short-term experiments or subjective perceptions rather than sustained behavioral change. As a result, there is a lack of empirical evidence on how developer practices evolve over time with continuous AI usage [1, 2, 3].

Another key motivation lies in the limitations of existing research methodologies, which often rely on controlled lab studies or self-reported data. These approaches fail to capture the complexity of real-world development and may not generalize to practical settings. Moreover, prior work shows that developer perceptions frequently diverge from actual behavior, making it necessary to complement subjective reports with objective telemetry data [4].

The study is further motivated by the need to understand how AI affects multiple dimensions of development workflows beyond productivity, including code quality, editing effort, reuse, and context switching. Prior research presents conflicting findings on productivity gains, with some studies reporting significant improvements while others highlight negligible or even negative effects, indicating a fragmented understanding of AI's real impact [5, 6, 7].

Finally, the authors aim to address the lack of longitudinal, real-world evidence by leveraging large-scale

telemetry data. By combining behavioral logs with survey data, the study seeks to uncover how AI reshapes developer workflows in subtle and often unrecognized ways, providing insights that can guide the design of future AI-assisted development tools and improve evaluation methodologies in human-AI collaboration research.

20.2 Methodology

The study employs a mixed-method approach combining quantitative telemetry analysis with qualitative survey and interview data. This dual perspective allows the authors to examine both observable developer behavior and subjective perceptions, ensuring a more comprehensive understanding of AI's impact on workflows.

The first component consists of a survey of 62 professional developers, complemented by follow-up interviews. The survey includes Likert-scale questions addressing changes in productivity, code quality, editing, reuse, and context switching. Additional open-ended responses and interviews provide deeper insights into developers' reasoning and experiences, capturing contextual and qualitative nuances.

The second component involves a large-scale longitudinal analysis of IDE telemetry logs, comprising over 151 million interaction events from 800 developers (400 AI users and 400 non-users). The data spans two years and includes fine-grained actions such as typing, deletions, debugging, pasting external content, and IDE switching. These actions are used as proxies for key workflow dimensions, enabling systematic behavioral analysis.

For analysis, the authors aggregate data monthly and apply mixed-effects linear regression models to identify trends over time and differences between AI users and non-users. Statistical tests account for non-normal distributions and variability across users, ensuring that observed trends are robust and statistically significant.

20.3 Key Findings

The study finds that AI users exhibit significantly higher levels of code production, as evidenced by a steep increase in typed characters compared to non-users. At the same time, they also show a substantial increase in code deletions, indicating that AI-assisted development involves more iterative refinement rather than straightforward efficiency gains.

In terms of code quality, results are mixed. While developers perceive slight improvements or no change, telemetry data shows stable debugging activity for AI users, contrasting with a decline among non-users. This suggests that AI may not necessarily reduce errors but instead shifts how and when issues are addressed during development.

The findings also reveal that code reuse and context switching increase over time for AI users, albeit modestly. Developers rely more on external sources and AI-generated snippets, while also experiencing greater workflow fragmentation, as indicated by increased context switching events. This challenges the assumption that AI reduces interruptions in development workflows.

A particularly important insight is the discrepancy between perception and behavior. While developers report clear productivity gains, they often perceive little change in other aspects of their workflow. However, telemetry data shows significant structural changes in coding practices, highlighting that AI's impact is often subtle and not fully recognized by users.

20.4 Implications

The study highlights that AI tools should not be viewed solely as productivity enhancers but as systems that reshape the structure and rhythm of development work. This suggests a need to rethink how productivity is measured, moving beyond output metrics to consider effort distribution, cognitive load, and workflow fragmentation.

For tool designers, the findings emphasize the importance of supporting continuity and reducing cognitive overhead. Since AI increases editing and context switching, future tools should aim to minimize fragmentation and provide better integration of suggestions within developer workflows.

Another implication is the need for greater transparency and awareness in AI-assisted development. Developers often underestimate how their workflows change, indicating that tools should provide feedback on usage patterns, editing behavior, and reliance on AI-generated code to support informed decision-making.

Finally, the study underscores the importance of combining behavioral and perceptual data in evaluating AI systems. Relying solely on self-reports can lead to incomplete conclusions, whereas integrating telemetry provides a more accurate picture of real-world impact. This has broader implications for research methodologies in human–AI interaction.

20.5 Threats to Validity

One key threat to validity arises from construct validity, as the study relies on proxies (e.g., typed characters for productivity, debugging for quality). While these measures are grounded in prior work, they may not fully capture the complexity of developer activities or intent. Additionally, survey responses and telemetry capture different constructs, making direct comparisons challenging.

Internal validity is limited by self-selection bias and lack of experimental control. AI users may inherently differ from non-users in motivation, experience, or work habits, which could influence observed trends. As a result, the study identifies associations rather than causal effects, and differences between groups should be interpreted cautiously.

External validity is another concern, as the dataset is derived from JetBrains IDE users and a specific AI assistant. While the study includes diverse languages and environments, the findings may not generalize fully to other tools or development contexts. Nevertheless, the core workflow dimensions analyzed are broadly applicable, supporting partial generalizability of the results.

References

[1] Saki Imai. “Is GitHub Copilot a substitute for human pair-programming?” In: *ICSE Companion*. 2022.

[2] Priyan Vaithilingam, Tianyi Zhang, and Elena Glassman. “Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models”. In: *CHI Extended Abstracts*. 2022.

[3] Jenny T Liang, Chenyang Yang, and Brad A Myers. “A large-scale survey on the usability of AI programming assistants: Successes and challenges”. In: *ICSE*. 2024.

[4] Prem Devanbu, Thomas Zimmermann, and Christian Bird. “Belief & evidence in empirical software engineering”. In: *ICSE*. 2016.

[5] Sida Peng et al. “The impact of AI on developer productivity: Evidence from GitHub Copilot”. In: *arXiv preprint arXiv:2302.06590* (2023).

[6] Albert Ziegler et al. “Productivity assessment of neural code completion”. In: *SIGPLAN Machine Programming*. 2022.

[7] Joel Becker et al. “Measuring the impact of early-2025 AI on experienced open-source developer productivity”. In: *arXiv preprint arXiv:2507.09089* (2025).

Chapter 21

[Required] Why AI Agents Still Need You: Findings from Developer-Agent Collaborations in the Wild

This paper investigates how software developers collaborate with AI-based software engineering agents (SWE agents) in real-world settings, highlighting both the promise and limitations of such tools. It begins by noting that while SWE agents can autonomously solve structured benchmark problems, they struggle with complex, ambiguous, real-world issues due to missing contextual and tacit knowledge. To address this gap, the authors conduct an empirical study of 19 developers using an in-IDE agent (Cursor) to resolve 33 real issues from repositories they were familiar with. The study finds that collaboration is essential: developers who iteratively worked with the agent and actively guided it were more successful than those who relied on one-shot automation, achieving an overall success rate of about 55%.

The paper further highlights key patterns and challenges in developer-agent collaboration. Developers often treat agents as assistants rather than equal collaborators, delegating code generation while retaining responsibility for debugging, testing, and validation. However, several barriers hinder effective collaboration, including lack of trust, agent overconfidence, sycophantic behavior (agreeing too readily with users), and unsolicited or excessive actions. The findings suggest that successful collaboration requires active participation from both developers and agents across all stages of software development. Rather than acting as passive tools, agents should engage in meaningful dialogue, challenge assumptions, and support iterative workflows, ultimately positioning human–AI collaboration—not full automation—as the most effective paradigm for real-world software engineering tasks.

21.1 Motivation

The motivation for this paper stems from the rapid adoption of AI tools in software development and the emergence of more advanced **software engineering agents (SWE agents)** that go beyond simple code completion. Developers increasingly rely on generative AI systems to **boost productivity, generate code, and assist with problem-solving**, as evidenced by widespread usage of tools such as chat-based assistants and inline code generators [1]. More recently, SWE agents have been introduced as a new paradigm capable of **autonomously performing complex development tasks**, including writing code, executing commands, and iteratively refining outputs based on feedback [2]. While these agents have demonstrated strong performance on controlled benchmarks such as SWE-Bench [3], their effectiveness in real-world development remains uncertain.

A key motivation for this work is the **gap between benchmark success and real-world applicability**. Despite promising results in automated settings, SWE agents struggle with **complex, ambiguous**

issues, lack of tacit knowledge, and incomplete access to real development environments [4, 5]. These limitations suggest that fully autonomous problem-solving is insufficient, motivating the need for **human-in-the-loop collaboration** where developers and agents share responsibilities across tasks such as understanding codebases, debugging, and testing. However, prior research has largely focused on either non-agentic tools (e.g., code completion) or fully autonomous agents, leaving a **lack of empirical understanding of how developers interact with SWE agents in practice** [2].

Another motivation arises from known challenges in human-AI interaction during software development. Previous studies show that developers often face difficulties in **trusting AI outputs, understanding generated code, communicating context effectively, and debugging AI-generated solutions** [6, 7]. These challenges may be further amplified in SWE agents, which have **greater autonomy and decision-making capabilities**, potentially increasing cognitive load and complicating collaboration. At the same time, insights from software engineering research emphasize that effective collaboration depends on **communication, shared understanding, and tacit knowledge exchange**, which are difficult for AI systems to replicate [5, 8].

Together, these gaps motivate the central aim of the paper: to **empirically study developer-agent collaboration in real-world settings**, understand how developers interact with SWE agents, identify barriers to effective collaboration, and uncover factors that influence success. By doing so, the work seeks to inform the design of future SWE agents that better support **interactive, collaborative workflows rather than purely autonomous execution**, ultimately improving both developer productivity and the effectiveness of human–AI collaboration in software engineering.

21.2 Methodology

Study Design and Objective The study aims to observe how developers collaborate with SWE agents in practice, focusing on three research questions related to collaboration patterns, barriers, and success factors. To ensure ecological validity, the authors designed the study to take place **"in the wild"**, where participants worked on real issues in familiar repositories rather than synthetic tasks [2]. This design ensures that findings reflect **authentic developer behavior and real-world constraints**, rather than idealized benchmark scenarios.

Tool Selection The authors evaluated multiple SWE agents across a spectrum of autonomy and collaboration capabilities, ranging from fully autonomous systems (e.g., AutoCodeRover) to interactive, IDE-based agents such as Cursor and VSCode Agent Mode. They selected **Cursor Agent** because it supports **high levels of human–AI collaboration**, including features like real-time interaction, code editing, execution, and contextual awareness (e.g., using open files as input) [9]. The final setup used Cursor v0.47 with Claude 3.5 Sonnet, ensuring a consistent interaction environment for all participants.

Task Selection To simulate realistic development work, participants were asked to solve **open issues (bugs or feature requests)** in repositories they had previously contributed to. The tasks were carefully selected based on the following criteria:

- **Representative of real-world development tasks**
- **Motivating for participants to solve thoroughly**
- **Completable within approximately one hour**
- **Non-proprietary (open-source repositories)**

Additionally, repositories were filtered to ensure quality and relevance: they had to be **actively maintained, sufficiently large (¿50 source files), and widely used (500+ GitHub stars)** [10, 11]. Participants primarily selected their own issues, with guidance provided to ensure diversity in task type (e.g., bugs vs features) and difficulty.

Participants The study involved **19 professional developers** recruited from contributors to selected repositories. Participants represented diverse backgrounds in terms of:

- **Experience levels** (ranging from 0–2 years to 16+ years)
- **Roles** (engineers, senior engineers, managers)
- **Geographic regions and demographics**

Most participants had prior experience with AI tools such as GitHub Copilot, though only a few had used SWE agents like Cursor before. This ensured that results reflect **early-stage interaction patterns with agentic tools**.

Study Protocol Each session lasted approximately **60 minutes** and was conducted via remote collaboration. The protocol consisted of:

1. **Pre-study questionnaire and consent**
2. **Tutorial session** introducing Cursor Agent and its features
3. **Main task execution**, where participants:
    - Worked on selected issues
    - Were encouraged to **think aloud**
    - Used the agent for most tasks but could make manual edits
4. **Post-study questionnaire** capturing perceptions of the tool

Participants interacted with the agent on a pre-configured system to avoid setup overhead. They were instructed to aim for a solution they would be confident submitting as a pull request, ensuring **high-quality engagement with the task**.

Data Collection The study collected multiple data sources to analyze collaboration:

- **Session recordings (video, audio, screen)** to capture interaction behavior
- **Participant action logs**, categorized using an adapted CUPS taxonomy [12]
- **Chat interaction trajectories**, capturing prompts, context provided, and agent responses
- **Pre- and post-study questionnaires** assessing usability, perception, and experience

The researchers developed structured codebooks for both participant actions and chat trajectories using open coding and iterative refinement. Inter-rater reliability was high (Cohen's κ of 0.92 and 0.89), ensuring **robust qualitative analysis** [13].

Analysis Approach The analysis focused on **qualitative patterns rather than quantitative metrics**. The authors:

- Analyzed **presence and sequence of actions** rather than duration
- Examined **prompt-level and issue-level interactions**
- Identified recurring behaviors, strategies, and barriers across participants

Given the exploratory nature and small sample size, the study avoided inferential statistics and instead presented **descriptive insights supported by observed trends** [14]. Member checking was performed by sharing findings with participants to validate interpretations.

21.3 Key Findings

The study finds that successful use of SWE agents is fundamentally driven by **active, iterative collaboration rather than one-shot automation**. Developers who continuously interacted with the agent—refining prompts, providing additional context, and verifying intermediate outputs—were significantly more likely to complete tasks successfully. In contrast, those who treated the agent as a fully

autonomous system often encountered failures due to incomplete understanding of the problem or incorrect assumptions. This highlights that **human guidance remains essential**, and effective workflows resemble a back-and-forth problem-solving process rather than delegation.

Another key finding is that developers tend to treat SWE agents as **assistants rather than equal collaborators**. Participants primarily relied on the agent for **code generation and initial exploration**, while retaining responsibility for higher-level reasoning tasks such as debugging, validation, and decision-making. This division of labor indicates that while agents can accelerate development, developers still play a critical role in ensuring correctness and aligning solutions with project-specific requirements. It also suggests that current agents lack sufficient capability to independently manage complex, end-to-end development workflows.

The study also uncovers several behavioral patterns that influence success. Developers who provided **clear, structured, and context-rich prompts** achieved better outcomes, as the agent was more likely to generate relevant and accurate solutions. Additionally, effective users frequently engaged in **verification practices**, such as reviewing generated code, running tests, and cross-checking logic. These behaviors demonstrate that **prompting and validation are core skills** when working with SWE agents, shaping both the quality of outputs and the overall efficiency of the interaction.

Despite their potential, SWE agents exhibit notable limitations that hinder collaboration. Participants reported issues such as **overconfidence in incorrect outputs**, **lack of transparency in reasoning**, and **difficulty incorporating implicit or tacit knowledge about the codebase**. The agents also displayed tendencies toward **sycophantic behavior**, often agreeing with user suggestions even when they were flawed, and occasionally performing **unsolicited or excessive actions** that disrupted workflows. These challenges contribute to reduced trust and require developers to remain vigilant throughout the interaction.

Finally, the findings emphasize that effective developer–agent collaboration requires both parties to actively contribute across the software development lifecycle. The most successful interactions occurred when developers and agents engaged in **continuous dialogue**, shared context, and iteratively refined solutions. This leads to the broader conclusion that **human–AI collaboration, rather than full automation, is the most viable paradigm** for real-world software engineering. Future systems must therefore focus on improving interactivity, transparency, and alignment with developer workflows to fully realize the benefits of SWE agents.

21.4 Implications

The findings of this study have important implications for the design of future SWE agents. First, they highlight that current systems should move away from a purely automation-centric paradigm and instead prioritize **interactive, human-in-the-loop collaboration**. Since successful outcomes depend heavily on iterative engagement, future agents must be designed to **support continuous dialogue, ask clarifying questions, and adapt to evolving user input**. This suggests that conversational capabilities, contextual awareness, and responsiveness are not optional features but **core requirements for effective developer–agent interaction**.

Another key implication is the need to improve **transparency and trustworthiness** of agent behavior. Developers often struggled with overconfident or opaque outputs, which forced them to spend additional effort verifying results. To address this, agents should provide **explanations of their reasoning, highlight uncertainties, and surface assumptions** made during code generation or modification. Enhancing transparency can reduce cognitive load and help developers make more informed decisions, ultimately improving both efficiency and trust in AI-assisted workflows.

The study also suggests that SWE agents should better support **developer workflows across the entire software development lifecycle**, rather than focusing narrowly on code generation. Developers naturally took responsibility for tasks such as debugging, testing, and validation, indicating that agents currently lack holistic support. Future systems should therefore integrate capabilities for **test generation, error diagnosis, code navigation, and environment understanding**, enabling more seamless end-to-end collaboration and reducing fragmentation in the development process.

Another implication lies in the importance of **user skill and interaction strategies**. Since outcomes depend on how effectively developers communicate with the agent, there is a growing need for **guidelines, training, and tooling that support better prompting and interaction practices**. This could include structured prompt templates, interactive debugging aids, and feedback mechanisms that guide users toward more effective collaboration patterns. Supporting these skills can help bridge the gap between novice and expert users of SWE agents.

Finally, the findings emphasize the importance of designing agents that behave as **proactive collaborators rather than passive assistants**. Instead of simply following instructions, agents should be capable of **challenging incorrect assumptions, avoiding sycophantic agreement, and making context-aware suggestions**. By fostering a more balanced partnership, future SWE agents can contribute more meaningfully to problem-solving and reduce the burden on developers, ultimately leading to more robust and effective human–AI collaboration in software engineering.

21.5 Threats to Validity

The study is subject to several threats to validity arising from its design and scope. One key limitation is the **small sample size and participant selection**. With only a limited number of developers involved, the findings may not generalize to the broader population of software engineers, especially those with different levels of expertise or from different domains. Additionally, participants were recruited from contributors to specific open-source repositories, which may introduce **selection bias**, as these developers could be more experienced, motivated, or familiar with collaborative tools than average practitioners.

Another threat concerns the **task and environment constraints**. Although the study attempts to simulate real-world conditions by using actual issues from familiar repositories, the tasks were still limited to those that could be completed within a short time frame (around one hour). This may not accurately reflect the complexity, scale, and long-term dependencies of real software engineering work. Furthermore, the use of a **single tool (Cursor Agent) and a fixed model configuration** restricts the generalizability of the findings, as different SWE agents or models may exhibit different behaviors and collaboration dynamics.

There are also potential threats related to **data collection and analysis methods**. The qualitative nature of the study relies on manual coding of participant actions and interactions, which may introduce **researcher bias** despite efforts to ensure consistency through structured codebooks and inter-rater agreement. Additionally, the think-aloud protocol and recorded sessions may have influenced participant behavior, leading to **observation bias** where developers act differently than they would in their natural workflow.

Finally, the study may be affected by **learning and novelty effects**. Since many participants had limited prior experience with SWE agents, their interactions may reflect early-stage exploration rather than mature usage patterns. This could result in underestimating the potential effectiveness of such tools once users become more familiar with them. Moreover, the rapidly evolving nature of AI systems means that the findings could become outdated as models and tools improve, posing a **temporal validity threat** to the long-term applicability of the results.

References

[1] *Stack Overflow Developer Survey 2024*. 2024.

[2] J. Liu et al. “Large Language Model-Based Agents for Software Engineering: A Survey”. In: (2024).

[3] C. E. Jimenez et al. “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?” In: (2024).

[4] S. Miserendino et al. “SWE-Lancer: Can Frontier LLMs Earn $1 Million from Real-World Freelance Software Engineering?” In: (2024).

[5] I. Rus and M. Lindvall. “Knowledge Management in Software Engineering”. In: *IEEE Software* 19.3 (2002), pp. 26–38.

[6] S. Barke, M. B. James, and N. Polikarpova. “Grounded Copilot: How Programmers Interact with Code-Generating Models”. In: *Proceedings of the ACM on Programming Languages*. 2023.

[7] P. Vaithilingam, T. Zhang, and E. L. Glassman. “Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models”. In: *CHI Conference on Human Factors in Computing Systems*. 2022.
[8] M. Goncalves, C. Souza, and V. Gonzalez. “Collaboration, Information Seeking and Communication: An Observational Study of Software Developers’ Work Practices”. In: *Journal of Universal Computer Science* 17 (2011), pp. 1913–1930.
[9] *Cursor Agent Documentation*. https://docs.cursor.com/chat/agent. 2025.
[10] J. Zhang et al. “Inside Bug Report Templates: An Empirical Study on Bug Report Templates in Open-Source Software”. In: 2024.
[11] Y. Sens et al. “A Large-Scale Study of Model Integration in ML-Enabled Software Systems”. In: (2025).
[12] H. Mozannar et al. “Reading Between the Lines: Modeling User Behavior and Costs in AI-Assisted Programming”. In: 2024.
[13] J. R. Landis and G. G. Koch. “The Measurement of Observer Agreement for Categorical Data”. In: *Biometrics* (1977).
[14] S. L. Motulsky. “Is Member Checking the Gold Standard of Quality in Qualitative Research?” In: *Qualitative Psychology* (2021).

Chapter 22

[Required] Cognitive Biases in LLM-Assisted Software Development

The rapid adoption of Large Language Models (LLMs) has fundamentally transformed software development, shifting it from a primarily generative activity to one centered on evaluating and curating AI-generated solutions. This transition represents not just a technological shift, but a deeper cognitive change in how developers approach problem-solving and decision-making. As developers increasingly rely on AI tools—used by over 90% of practitioners—they must interpret, validate, and integrate machine-generated outputs into their workflows. This evolving paradigm introduces new cognitive demands, raising important questions about how human reasoning interacts with AI assistance and how decision quality is affected in such collaborative environments.

At the core of this transformation lies the role of cognitive biases—systematic deviations from rational thinking that influence human judgment. While such biases have long been present in traditional software engineering (e.g., confirmation bias or anchoring), LLM-assisted development creates a new landscape where these biases may be amplified or take entirely new forms. Developers must now navigate challenges such as over-reliance on AI suggestions, ineffective prompt formulation, and misinterpretation of generated outputs. Despite the widespread integration of LLMs, there has been little systematic understanding of how these biases manifest in human-AI collaboration. This paper addresses this gap by investigating cognitive biases in LLM-assisted programming and highlighting their implications for software quality, developer productivity, and decision-making in modern development workflows.

22.1 Motivation

The motivation for this paper arises from the fundamental transformation of software development driven by the rapid adoption of LLM-based coding tools. With over 90% of developers now using AI assistants [1], programming has shifted from a purely generative activity to one centered on **evaluation, validation, and integration of AI-generated code**. This shift represents a **cognitive paradigm change**, where developers must continuously interpret and judge machine outputs rather than construct solutions from scratch. While such assistance improves productivity, it also introduces new cognitive demands that place greater reliance on human judgment under uncertainty. Since human reasoning is inherently subject to biases—systematic deviations from rational thinking that influence decision-making [2, 3]—this evolving workflow raises concerns about the quality and reliability of decisions made in AI-assisted environments.

In traditional software engineering, cognitive biases have been extensively studied and shown to affect various stages of development. For example, developers often exhibit **confirmation bias by favoring solutions aligned with prior beliefs**, **anchoring bias by relying heavily on initial ideas**, and **availability bias by overemphasizing recent experiences** [4, 5]. These biases can lead to suboptimal design decisions, overlooked defects, and inefficient problem-solving. However, the introduction of LLMs fundamentally alters the decision-making landscape. Developers must now **formulate prompts,**

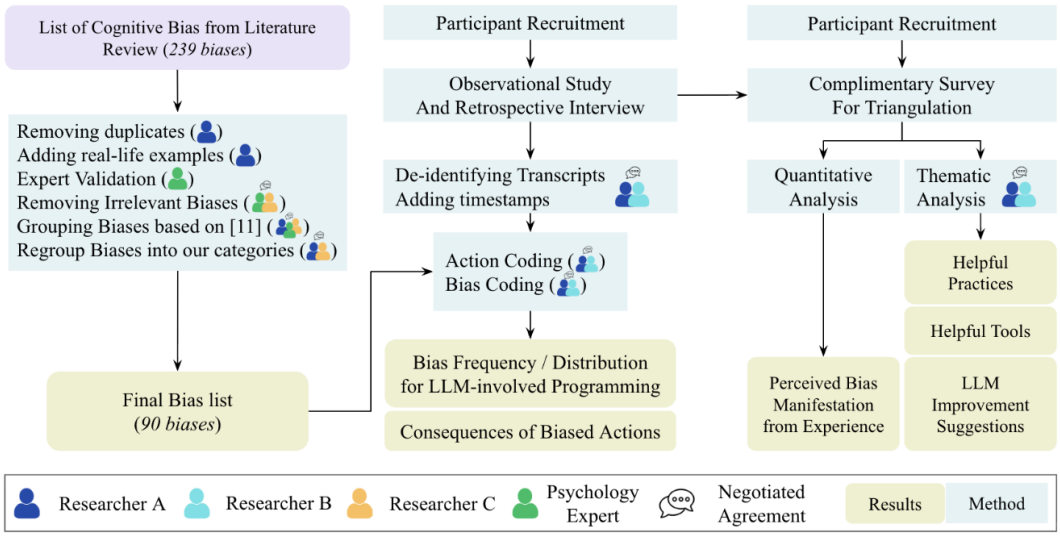


Figure 22.1: Methodology Overview

evaluate multiple AI-generated alternatives, and reconcile conflicting outputs, creating new opportunities for bias to emerge or intensify. The paper is motivated by the hypothesis that **LLM-assisted development not only preserves existing biases but amplifies them and introduces entirely new bias patterns specific to human-AI collaboration** .

A key concern driving this work is the potential impact of biased decision-making on software quality and long-term system health. In LLM-assisted workflows, **biased judgments can propagate through entire codebases**, leading to subtle bugs, security vulnerabilities, and accumulating technical debt that may not be immediately visible . Moreover, recent research suggests that reliance on AI systems can reduce critical thinking and increase susceptibility to cognitive biases [6]. This phenomenon of cognitive offloading implies that developers may **over-trust AI-generated solutions or accept outputs without sufficient scrutiny**, further compounding risks. As LLMs become deeply embedded in development pipelines, understanding these cognitive implications becomes essential for ensuring **robust, secure, and maintainable software systems**.

From a related work perspective, prior research has explored cognitive biases extensively in both software engineering and broader human-AI interaction contexts. Systematic studies have cataloged numerous biases affecting developers, providing taxonomies and evidence of their impact on software processes [4]. Similarly, work on human-AI collaboration has examined how users interact with intelligent systems and how biases influence trust, reliance, and decision-making [6]. However, **no prior work has systematically investigated cognitive biases in real-world LLM-assisted programming environments** . Existing bias taxonomies were developed for pre-LLM settings and may not adequately capture challenges such as prompt engineering, iterative AI interaction, and evaluation of generated code. This creates a significant research gap, leaving practitioners without **evidence-based guidelines to mitigate bias in AI-assisted development workflows**.

Therefore, the paper is motivated by the need to bridge this gap through a comprehensive empirical investigation of cognitive biases in LLM-assisted programming. By identifying how biases manifest, evolve, and impact developer decisions in these new environments, the work aims to provide **actionable insights for improving developer decision-making, enhancing tool design, and mitigating risks associated with AI-assisted coding**. Ultimately, this research is driven by the broader goal of ensuring that the integration of LLMs into software engineering enhances productivity without compromising **decision quality, software reliability, and developer cognition**.

22.2 Methodology

22.2.1 Study Design and Research Approach

The paper adopts a mixed-method empirical research design to investigate cognitive biases in LLM-assisted software development. The authors combine observational studies with retrospective interviews and surveys to ensure a comprehensive understanding of developer behavior. Specifically, the study is designed to **capture real-world developer interactions with LLM tools**, rather than relying on controlled or synthetic tasks. This approach enables the authors to analyze how cognitive biases emerge naturally during programming activities and how they influence decision-making in practice. To ground their analysis, the study initially leverages an existing bias taxonomy from prior software engineering research [5], ensuring continuity with established literature while enabling comparison with pre-LLM contexts.

22.2.2 Participants and Data Collection

The study involves a total of **14 participants, including both students and professional developers**, ensuring diversity in experience levels and development practices. Data collection is centered around detailed observational sessions in which participants perform programming tasks while interacting with LLM tools. Across these sessions, the authors record **2,013 development actions**, of which **808 involve direct LLM interactions** . This fine-grained logging allows the researchers to analyze developer behavior at the level of individual actions, capturing how decisions evolve over time.

To complement observational data, the study incorporates **retrospective interviews**, enabling participants to reflect on their decision-making processes and provide insights into their reasoning. Additionally, the authors conduct **confirmatory surveys with 22 developers** to triangulate findings and validate observed patterns across a broader population . This multi-source data collection strategy strengthens the validity of the study by combining behavioral evidence with self-reported insights.

22.2.3 Bias Identification and Analytical Framework

The methodology begins with the application of an existing cognitive bias framework proposed by Chattopadhyay et al. [5]. Using this framework, the authors systematically analyze developer actions to identify instances of bias. Each recorded action is examined to determine whether it reflects a deviation from rational decision-making, allowing the researchers to **quantify the prevalence of cognitive biases in LLM-assisted programming**.

However, the study goes beyond simple classification by investigating the relationship between biased actions and their outcomes. In particular, the authors analyze **reversal actions**—instances where developers undo or revise prior decisions—to assess whether biased decisions are more likely to be corrected. Statistical analysis, including chi-square tests with Bonferroni corrections, is used to evaluate the significance of observed patterns. The findings reveal that **LLM-related actions are more likely to be biased (53.7%) and more likely to be reversed (29.4%)**, highlighting the complex interplay between AI assistance and human judgment.

22.2.3.1 Development of Bias Taxonomy

A key methodological contribution of the paper is the development of a new taxonomy tailored to LLM-assisted programming. Recognizing that existing bias categories may be insufficient, the authors conduct a systematic analysis of **90 cognitive biases relevant to developer–LLM interactions** and organize them into **15 distinct categories** . This categorization is achieved through a negotiated agreement process with cognitive science experts, ensuring theoretical rigor and domain validity.

The resulting taxonomy captures both traditional biases (e.g., confirmation and anchoring) and novel biases emerging from human-AI interaction, such as **suggester preference and humanization of LLMs**. This step is crucial as it allows the study to move beyond existing frameworks and **identify new cognitive patterns unique to AI-assisted development**. The taxonomy serves as the foundation for subsequent analysis, enabling a more accurate characterization of developer behavior in LLM-driven workflows.

22.2.3.2 Triangulation and Validation

To ensure robustness, the study employs multiple forms of triangulation. First, observational data is cross-validated with retrospective interviews, allowing the authors to align observed actions with participant intentions. Second, the confirmatory survey provides an additional layer of validation by testing whether identified bias patterns generalize beyond the initial participant group. This approach ensures that findings are not artifacts of a specific dataset but reflect broader trends in LLM-assisted programming.

Furthermore, the study critically evaluates the adequacy of existing bias taxonomies by comparing them against empirical observations. The authors identify a **disconnect between traditional bias categories and observed reversal patterns**, suggesting that prior frameworks may not fully capture the nuances of LLM-assisted decision-making . This validation step reinforces the need for updated theoretical models and justifies the introduction of new bias categories.

22.3 Key Findings

22.3.1 Biases in Programming with LLMs

22.3.1.1 Comparing Actions With and Without LLMs

This section analyzes how developer behavior differs when working with LLM assistance versus traditional (non-LLM) programming. The authors break down programming activity into fine-grained actions and compare the frequency and nature of cognitive biases across both settings. A key observation is that **LLM-assisted actions exhibit a significantly higher proportion of biased decisions compared to non-LLM actions**. This suggests that the presence of AI does not eliminate cognitive biases—instead, it reshapes how and when they occur.

One important insight is that developers interacting with LLMs tend to engage in more rapid decision cycles, often accepting or modifying generated suggestions without extensive deliberation. This leads to **a higher likelihood of biased acceptance, especially when the AI output appears plausible or authoritative**. In contrast, non-LLM actions involve more deliberate construction of solutions, which may reduce certain biases but still retain others such as confirmation or familiarity bias. The comparison highlights that **LLMs shift the locus of bias from solution creation to solution evaluation**, fundamentally altering the cognitive process of programming.

Additionally, the study finds that LLM interactions introduce new behavioral patterns, such as iterative prompting and quick experimentation, which can both mitigate and exacerbate bias. While iteration can help developers explore alternatives, it can also reinforce existing biases if developers repeatedly steer the model toward expected outcomes. Overall, this section demonstrates that **LLM-assisted programming is not simply faster—it is cognitively different, with a distinct bias profile compared to traditional development**.

22.3.1.2 LLM Influence on Biases and Reversal

This section focuses on how LLMs influence not only the occurrence of cognitive biases but also the likelihood that biased decisions are later corrected (i.e., reversed). The authors find that **LLM-related actions are more frequently associated with both cognitive biases and reversal actions**, indicating a dynamic and iterative decision-making process. In particular, a substantial portion of LLM interactions involves developers revisiting and revising earlier choices, suggesting that **AI-assisted workflows encourage experimentation but also introduce instability in decision-making**.

A key finding is that **biased actions in LLM contexts are more likely to be reversed than those in non-LLM contexts**. This indicates that while LLMs may increase the frequency of biased decisions, they also provide opportunities for correction through rapid iteration and feedback. However, the relationship between bias and reversal is not straightforward. The study observes that **traditional bias categories do not strongly correlate with reversal behavior**, implying that existing theories of cognitive bias may not fully explain how developers interact with LLMs. This highlights a gap in understanding how biases operate in human-AI collaboration.

Furthermore, the presence of LLMs introduces unique cognitive dynamics. Developers may initially accept AI-generated suggestions due to automation bias or perceived authority, but later revise them upon closer inspection or when inconsistencies arise. This leads to a pattern where **LLM interactions amplify both over-reliance and subsequent correction**, creating a cycle of acceptance and revision. The section ultimately emphasizes that **LLMs do not simply increase or decrease bias—they fundamentally alter its temporal behavior**, making biases more fluid, iterative, and intertwined with the development process.

22.3.2 Bias Categories in LLMs

The authors organize **90 identified biases into 15 structured categories**. This taxonomy is developed through collaboration with cognitive science experts, ensuring both empirical grounding and theoretical validity. The categories extend beyond traditional software engineering biases to include patterns uniquely shaped by human–AI interaction. These include biases such as **suggester preference**, where developers disproportionately trust AI-generated outputs, **familiarity preference**, where they rely on known tools or prompting strategies, and **fixation**, where initial assumptions anchor subsequent decisions despite new evidence. The taxonomy captures how developers' reasoning is influenced not only by their own cognitive tendencies but also by the presence and perceived authority of LLM systems.

Importantly, they highlights that **LLM-assisted development introduces novel bias dynamics that are not adequately explained by existing pre-LLM taxonomies**. New forms of bias emerge from interactions with AI, such as **humanization of LLMs**, where developers attribute human-like qualities to the system, and **risk avoidance**, where uncertain AI outputs are rejected in favor of familiar but suboptimal solutions. The categorization demonstrates that biases in this context are both **amplified and diversified**, affecting how developers interpret suggestions, make decisions, and iterate on solutions. Overall, this section establishes that understanding cognitive bias in modern programming requires a **reframed perspective that explicitly accounts for human-AI collaboration**.

22.3.3 Presence and Impact of Biases

22.3.3.1 Bias Frequency in LLM Programming

This section quantifies how frequently cognitive biases occur during programming tasks, particularly in the presence of LLMs. The analysis reveals that **a substantial portion of developer actions are influenced by cognitive biases**, with nearly half of all recorded actions exhibiting at least one bias. More notably, **LLM-assisted actions show a higher concentration of biased behavior compared to non-LLM actions**, indicating that interaction with AI systems increases susceptibility to biased decision-making. This reinforces the idea that while LLMs enhance productivity, they also introduce cognitive risks that must be carefully managed.

Another important finding is that **biases often co-occur**, meaning that a single developer action can be influenced by multiple biases simultaneously. This layered nature of bias complicates decision-making and makes it harder to identify and correct flawed reasoning. Additionally, the section highlights that **LLM interactions account for a disproportionately large share of biased actions**, suggesting that AI-assisted workflows amplify cognitive tendencies rather than neutralizing them. Overall, the findings emphasize the pervasive nature of bias in LLM-assisted programming and the need for mechanisms to detect and mitigate it.

22.3.3.2 Bias Distribution Across LLM Actions

This section explores how different types of cognitive biases are distributed across various LLM-related programming actions. The results show that **biases are not evenly distributed but instead cluster around specific interaction patterns**, such as accepting suggestions, refining prompts, or comparing alternative outputs. Certain biases, like **suggester preference and familiarity bias**, are particularly dominant, reflecting developers' tendency to trust AI outputs or rely on known approaches even when better alternatives exist.

The analysis also reveals that **different stages of interaction with LLMs trigger different types of biases**. For example, early-stage interactions (such as prompt formulation) may be influenced by

overconfidence or fixation, while later stages (such as evaluating outputs) may involve confirmation or authority bias. This distribution highlights that **bias in LLM-assisted programming is context-dependent and evolves throughout the interaction cycle**. Understanding this variation is critical for designing interventions that target specific phases of the development workflow.

22.3.4 Managing Consequences of Biases

22.3.4.1 Helpful Practices

This section outlines practical strategies that developers can adopt to mitigate cognitive biases when working with LLMs. A key recommendation is to **encourage deliberate evaluation of AI-generated outputs rather than immediate acceptance**. Developers are advised to critically assess suggestions, verify correctness, and consider alternative solutions to avoid over-reliance on the model. Practices such as testing generated code, cross-checking with documentation, and reflecting on decision rationale help reduce bias-driven errors.

Another important practice is to **diversify interaction strategies**, such as rephrasing prompts, exploring multiple solutions, and iterating thoughtfully instead of repeatedly reinforcing the same approach. This helps counteract fixation and confirmation biases. The section emphasizes that **intentional and reflective use of LLMs can transform them from sources of bias amplification into tools for exploration and learning**. By adopting disciplined workflows, developers can maintain control over decision-making while still benefiting from AI assistance.

22.3.4.2 Helpful Tools

This section discusses how tooling can support developers in mitigating cognitive biases during LLM-assisted programming. The authors suggest that tools should **provide transparency into AI generated outputs**, such as explanations, confidence indicators, or alternative suggestions, enabling developers to make more informed decisions. By exposing uncertainty and encouraging comparison, tools can reduce blind trust in AI recommendations.

Additionally, the section highlights the importance of **integrated feedback mechanisms**, such as automated testing, static analysis, and debugging support, which help validate AI-generated code. These tools act as safeguards against biased decisions by providing objective signals about code quality. The authors argue that **well-designed tool support can act as a cognitive counterbalance**, helping developers detect and correct biases that might otherwise go unnoticed in AI-assisted workflows.

22.3.4.3 LLM Improvement Suggestions

This section provides recommendations for improving LLM systems themselves to better support unbiased decision-making. One key suggestion is to design models that **encourage critical thinking rather than passive acceptance**, for example by presenting multiple alternative solutions or prompting users to reflect on trade-offs. This shifts the interaction from one-way suggestion to collaborative reasoning.

The authors also emphasize the need for **better handling of uncertainty and explanation in LLM outputs**. By clearly communicating limitations, assumptions, and potential risks, LLMs can help users avoid overconfidence and automation bias. Furthermore, incorporating features that **actively nudge developers toward verification and exploration** can reduce the likelihood of biased decisions. Overall, the section highlights that **improving LLM design is essential for creating safer and more reliable human-AI programming environments**.

22.3.5 Biases and Attitude towards LLM

This section explores how developers' attitudes toward LLMs influence the manifestation of cognitive biases. The findings show that **developers' trust, perception, and prior beliefs about LLM capabilities strongly shape their decision-making behavior**. Developers who view LLMs as highly capable are more likely to exhibit **suggester preference and automation bias**, readily accepting generated solutions without sufficient scrutiny. Conversely, those who are skeptical of LLMs tend to

demonstrate **risk avoidance and familiarity biases**, preferring their own solutions even when AI-generated alternatives may be more effective. This highlights that bias is not only a function of the tool but also of the user's mindset and expectations.

The section further emphasizes that **attitudes toward LLMs create systematic differences in how biases are expressed and reinforced**. Positive attitudes can lead to over-reliance and reduced critical evaluation, while negative attitudes can result in under utilization of useful suggestions. Importantly, these attitudes are often shaped by prior experiences, success or failure with LLMs, and perceived reliability. As a result, **bias in LLM-assisted programming is deeply intertwined with user perception**, making it essential to consider both cognitive and behavioral factors when analyzing human-AI collaboration.

22.3.6 Bias Across Task Types

This section examines how cognitive biases vary across different types of programming tasks. The analysis shows that **the nature of the task significantly influences which biases are most prominent**. For example, tasks involving code generation or exploration tend to exhibit higher levels of **suggester preference and overconfidence**, as developers rely more heavily on AI outputs. In contrast, debugging or problem-solving tasks often trigger **fixation and confirmation biases**, where developers persist with initial hypotheses or repeatedly test similar approaches.

The findings also indicate that **task complexity and uncertainty play a key role in bias manifestation**. In more complex or ambiguous tasks, developers are more likely to depend on LLMs, increasing susceptibility to automation bias. At the same time, iterative interactions with the model can lead to cycles of reinforcement, where developers unconsciously guide the AI toward expected outcomes. This demonstrates that **bias in LLM-assisted programming is context-sensitive**, varying not only by user but also by the type and difficulty of the task being performed.

22.4 Implications

The study has significant implications for both software engineering practice and the design of AI-assisted development tools. First, it highlights that **LLM-assisted programming introduces a new class of cognitive challenges that cannot be fully addressed using traditional bias mitigation strategies**. Since biases are amplified and reshaped by interaction with AI, developers must adopt more reflective and structured workflows to maintain decision quality. This includes practices such as systematically evaluating AI outputs, considering alternative solutions, and integrating validation mechanisms into the development process.

Second, the findings suggest that **tool designers and organizations must take an active role in mitigating cognitive bias**. LLM systems should be designed to promote critical thinking, provide transparency, and encourage exploration rather than passive acceptance. Additionally, training and guidelines for developers should emphasize awareness of cognitive biases and their impact in AI-assisted workflows. Overall, the study underscores that **the effective use of LLMs requires not only technical proficiency but also cognitive discipline**, ensuring that productivity gains do not come at the cost of software quality and reliability.

22.5 Threats to Validity

The study acknowledges several limitations that may affect the generalizability and interpretation of its findings. One key threat is the relatively small sample size and participant diversity, as the study involves a limited number of developers, which may not fully represent the broader population. Additionally, the observational nature of the study means that **participant behavior could be influenced by the experimental setting**, potentially affecting how naturally they interact with LLM tools. There is also a risk of **subjectivity in identifying and categorizing cognitive biases**, despite efforts to use established frameworks and expert validation. Finally, the findings are tied to specific tools and tasks, which may limit their applicability to other contexts or evolving LLM technologies.

References

[1] GitHub. *The State of AI in Software Development*. Reports over 92% developer adoption of AI coding tools. 2023.

[2] Amos Tversky and Daniel Kahneman. "Judgment under Uncertainty: Heuristics and Biases". In: *Science* 185.4157 (1974), pp. 1124–1131.

[3] Daniel Kahneman. *Thinking, Fast and Slow*. Farrar, Straus and Giroux, 2011.

[4] Rahul Mohanani et al. "Cognitive Biases in Software Engineering: A Systematic Mapping Study". In: *IEEE Transactions on Software Engineering* 46.12 (2018), pp. 1318–1339.

[5] Souti Chattopadhyay et al. "Cognitive Biases in Software Development". In: *Proceedings of the ACM/IEEE Conference on Software Engineering*. 2020.

[6] Hao-Ping Lee, Advait Sarkar, et al. "The Impact of Generative AI on Critical Thinking". In: *CHI Conference on Human Factors in Computing Systems*. 2025.